This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

SystemVerilog Assertions

Course Version 5.1

Lecture Manual **Revision 1.0**

cadence®

This document is for the sole use of Nikhil Vijay Pawashe of Cadence

© 1990-2022 Cadence Design Systems, Inc. All rights reserved.

Printed in the United States of America.

Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

The publication may be used solely for personal, informational, and noncommercial purposes;

The publication may not be modified in any way;

Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement; and

Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence customers in accordance with, a written agreement between Cadence and the customer.

Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Table of Contents

SystemVerilog Assertions

Module 1 About This Course 5

Module 2 Assertion-Based Verification 11

Module 3 Simple Boolean Assertions 23

Module 4 Sequences 55

Module 5 Coverage 86

Module 6 Liveness Properties 104

Module 7 Property Reuse 115

Module 8 Auxiliary Code 131

Module 9 SVA Course Part 1 (Simulation and Formal Users) Conclusion 142

Module 10 Sequence Operations 150

Module 11 Advanced SVA Features 168

Module 12 Constructs Which Form Properties 186

Submodule Constructs of Some Pragmatic Value 189

Submodule Constructs of Little or No Pragmatic Value 203

Module 13 The Verification Completeness Problem 220

Module 14 SVA Course Part 2 (Simulation Users Only) Conclusion 241

Appendix A Quiz Solutions 249

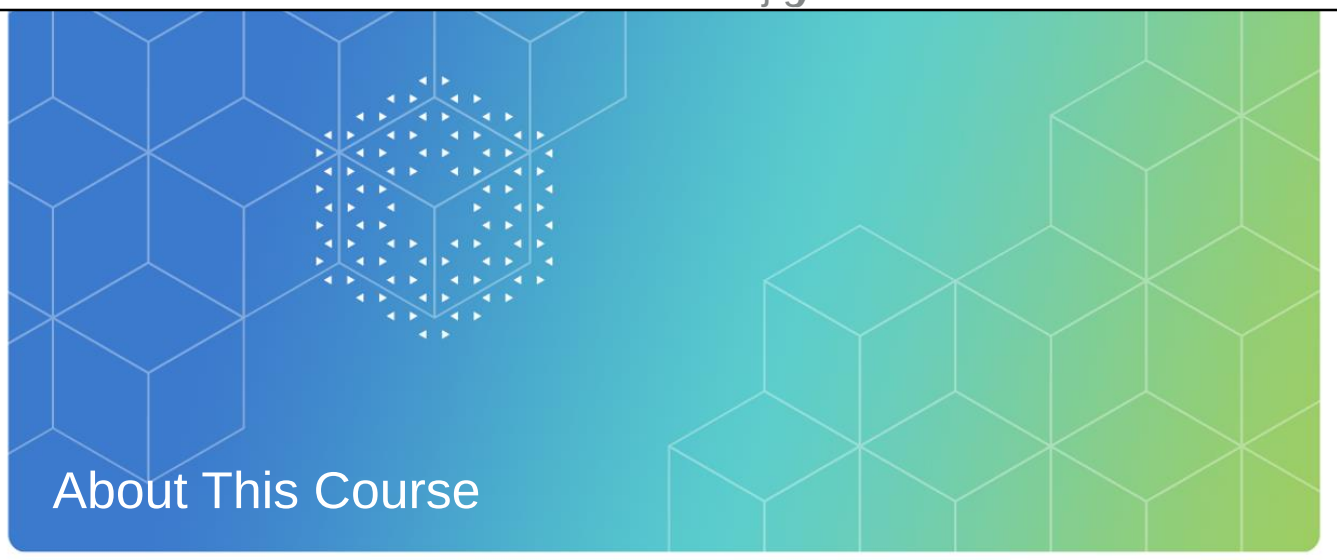
This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module 1
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Course Prerequisites

Before taking this course, you need to have already:

- One week's experience of writing Verilog or SystemVerilog
- Attended a [System]Verilog training course, for example, SystemVerilog for Design and Verification

All you need to know, for now, is how to:

1. Write a boolean expression in [System]Verilog
2. Describe a module
3. Define an event control, for example, @(posedge clk)

© Cadence Design Systems, Inc. All rights reserved.

There is no required prior knowledge of assertions or tools that use assertions in order to complete this course.

You will only need to know how to define Boolean expression in [System]Verilog and define clocking events like @(posedge clk).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What's Coming

This course teaches the latest standard at the date of writing

- IEEE 1800 SystemVerilog Language Reference Manual 2017

This course is effectively in 2 parts:

- Part 1: SVA for Formal and Simulation users
- Part 2: SVA for Simulation users only

Why the distinction?

Firstly it's important to understand that SVA is a language for describing design behavior (properties)

- This description is equally valid in both formal and simulation

To describe SVA properties that are efficient for formal requires a particular style

- Known as “Formal Friendly SVA” covered in detail in the Jasper Formal Fundamentals course

- Part 1 is SVA basic syntax and principles essential for both Formal and Simulation users
- Part 2 is SVA constructs which, while supported by formal tools, are not “Formal Friendly”
  - These constructs might be useful in simulation
  - It is also important to be aware of the limitations of those constructs

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Setting Your Expectations

What you will learn

- Basic and advanced SVA constructs
- Application issues, including:
  - Recommended coding styles
  - Guidelines for avoiding problems
  - Assertion organization and management
  - Reuse methodologies
- Practical verification tool use with SVA

Following the course

- Several weeks “hands-on” work to become experienced

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Course Philosophy

Many new concepts to learn

Keep an open mind

- This is very different to what you have done before
- It requires practice before full understanding is reached

Interactive classroom style

- Ask questions
- May take off line if too detailed!

Remember we are here to learn SVA, not a particular tool

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Course Agenda

Part 1 – Formal and Simulation Users

- Course Introduction
- Introduction to Assertion-Based Verification (ABV)
- Simple Boolean Properties
- Sequences
- Coverage
- Liveness Properties
- Property Reuse
- Auxiliary Code Introduction
- Part 1 Conclusions and Next Steps

Part 2 – Simulation Users

- Sequence Operations
- Advanced SVA Topics
- Constructs Which Form Properties
  - Ones of pragmatic use
  - All of the others
- Verification Completeness
- Part 2 Conclusions and Next Steps

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Software and Licenses

The course was developed for and Xcelium™ 2203, Indago™ 2203 and Jasper™ 2206

It should be also be compatible with earlier and later versions.

Part 1		
Product ID	**Product Name**	**License Feature Name(s)**
JGFVBASE	Jasper Formal Verification Platform	jasper_papp jasper_pcov jasper_pint
Part 2		
X300	Xcelium Single-Core	Affirma_sim_analysis_env Integrated_Metrics_Center Xcelium_Single_Core
29853	Indago Debug Analyzer App	Affirma_sim_analysis_env Indago_DA_App Indago_DA_wAPI

9 © Cadence Design Systems, Inc. All rights reserved.

cadence

In order to run the formal_profiler you will need an additional license feature:

JGFVADVP: JasperGold Advanced Platform Option to JGFVBASE: jasper_advp

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Become Cadence Certified by Earning a Digital Badge

Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
  - Expand career opportunities
- Professional credibility
  - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.



How do I register to take the exam?

- Log in to our Learning Management System to locate the exam in your transcript.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- Credly validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module 2
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

Aims

- Overview of concepts and practicalities of Assertion-Based Verification (ABV)

Topics

- What is an assertion?
- What is Assertion-Based Verification (ABV)?
- ABV concepts

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What Is an Assertion?

An embedded check-in code

During verification, assertions watch to see:

- If a specific condition occurs, or
- If a specific sequence of events occurs

Assertions generate warnings or errors when:

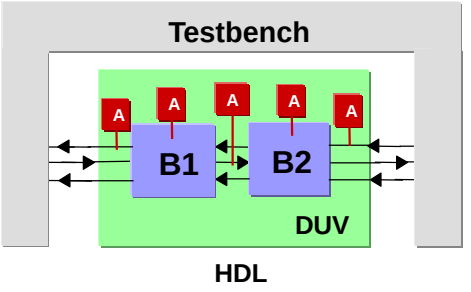
- A specified condition fails, or
- A specified sequence does not complete properly.

Assertions offer a way to significantly enhance productivity for designers

- Find bugs earlier and more easily

Checks without coverage are of little use

- We need to know what was exercised when the checks were performed
- This is fundamental to any verification methodology



13 © Cadence Design Systems, Inc. All rights reserved.

cadence

Definition: property – a collection of logical and temporal relationships between boolean expressions, sequential expressions, and other properties that represent a set of behaviors.

Definition: assertion – a statement that a given property is required to be true within a specific design – and a directive to verification tools to verify that it is true.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

What Is a Cover?

Firstly it is not a check which passes/fails

The motivation for writing covers is to ensure that our verification is thorough

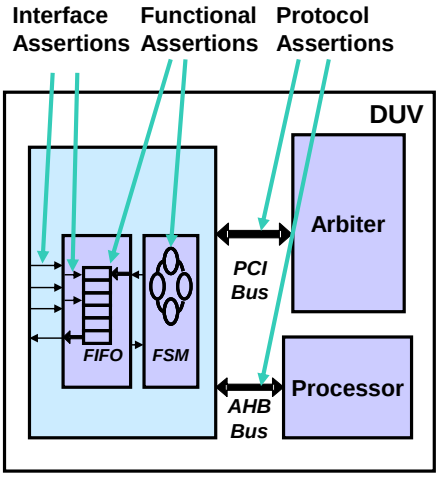
A cover is a sequence of normal design behavior which one would expect to observe during verification, for example:

- My FIFO buffer can get full and then empty again
- I had a PCI bus transaction that was interrupted by a reset
- I observed all interesting variations of AHB transfer

In simulation, a cover is a count of how many times the sequence was observed

In formal, a cover is a demonstration of that sequence occurring

- Normally the shortest trace



14 © Cadence Design Systems, Inc. All rights reserved.

Definition: property – a collection of logical and temporal relationships between boolean expressions, sequential expressions, and other properties that represent a set of behaviors.

Definition: assertion – a statement that a given property is required to be true within a specific design – and a directive to verification tools to verify that it is true.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Simulation Testbench

Testbench drives the DUV inputs, typically with randomized stimulus

SVA Assertions monitor and report:

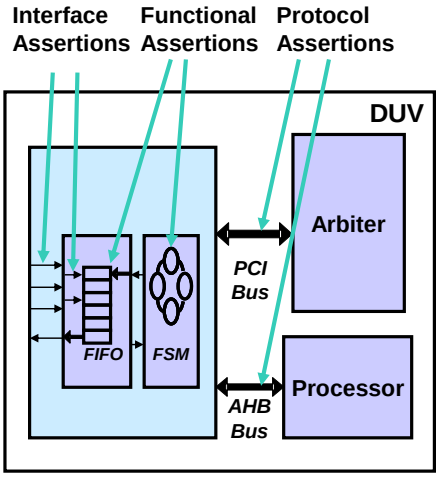
- Expected behavior
- Forbidden behavior
- Signal protocols

SVA Cover Statements monitor and report:

- A count of how many times specific sequences are observed

SVA Assumes are not relevant for simulation

- The simulator will automatically turn assumes into asserts



15 © Cadence Design Systems, Inc. All rights reserved.

cadence

Static verification is verification that does not require a verification engineer to think of what stimulus to apply to verify a design.

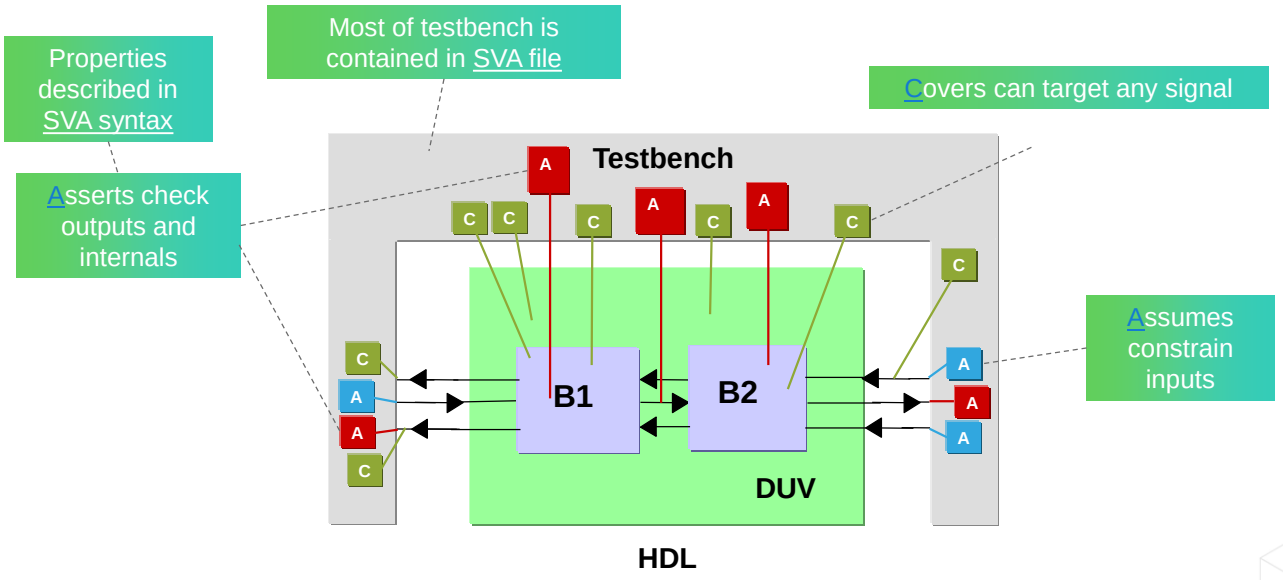
The design's "correctness" is verified using mathematical techniques. This is known as "formal verification." It is performed using symbolic proofs rather than applying simulation vectors. There are different kinds of "formal verification," Logic Equivalence Checking (LEC) and Functional Property Checking, for which there is no universally accepted name. In the present context, we are referring to Functional Property Checking.

Formal Verification (formal) is a fundamentally different paradigm to simulation. There are different considerations for effective ABV in formal vis-à-vis simulation. One of these is to ensure that the environment is set up correctly for the results of a formal proof to have any value. We will discuss how formal works and why assumptions are needed later in this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Formal Testbench



 © Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

What Is Assertion-Based Verification (ABV)?

Formalized and structured use of assertions to describe and check design characteristics

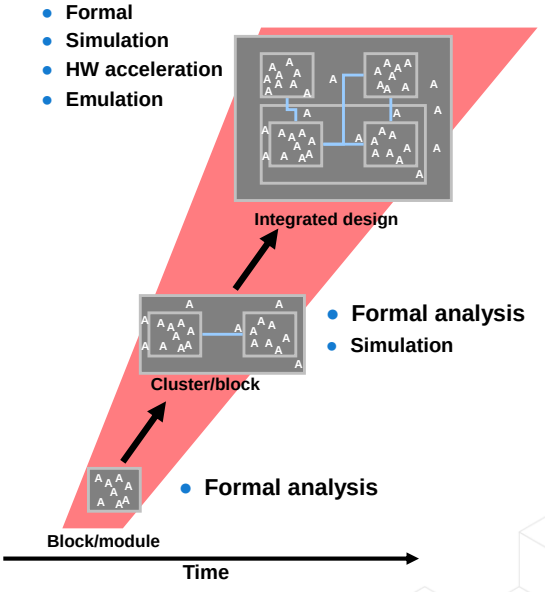
Integration of clean blocks and modules significantly reduces effort and debug time

Same assertions [re]used in:

- Formal
- Simulation
- Acceleration/emulation
- Cluster and full chip level

Unified ABV flow results in:

- Improved productivity and quality
- Overall reduction in verification time/effort



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Who Writes Properties and Why?

Who	Type of Assertion	Reason	Example
System Architects	Abstract properties	Ensure implementation meets functional spec	An asynchronous FIFO does not indicate full and empty at the same time
RTL Designer	Functional Property	Implementation has properties the RTL designer intended it to have	When Read and Write pointers both 0, the FIFO indicates empty
Verification Engineer	Functional Coverage	Ensure test stimulus exercises the corner cases as required by the test plan	Did you read from full FIFO, write to the last empty FIFO location, etc.
IP Designer	Correct use of IP	Indicate to IP user that the IP is not being used correctly	FIFO was read when it was empty

© Cadence Design Systems, Inc. All rights reserved.

Assertions are effective for many applications and at many levels.

Coverage is, strictly speaking, separate from assertions that properties are met, as you will see later in the course. However, SVA supports both assertions and coverage measurement, so both are mentioned here.

Note that for IP, assertions guard against misuse (and consequent complaints back to the IP provider).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Why Use SVA Properties?

Improves observability

- Automatically and constantly checks behavior
- Can detect hard-to-find bugs embedded deep in the design
- Can non-intrusively apply properties, even to legacy code
- Isolates problem close to source
  - No need to propagate error to outputs and “track-back” to source

Improves verification efficiency

- Concisely and unambiguously document design intent
- Detect bugs earlier in the design cycle
- Trap and exit on errors, saving simulation cycles
- Writing assertions helps designers to better understand their design
- Writing assertions encourages designers to do more debugging

Assertions can be embedded in code to “travel with” design from:

- Project to project – ideal for IP
- Tool to tool – ideal for design methodology integrity

19 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions can be placed anywhere in your HDL hierarchy, enabling you to monitor design behavior locally and highlight problems as soon as they occur from the source of the problem. Conventional HDL testbenches need a problem to propagate to the design outputs before it can be detected, and then the problem has to be tracked back through the design hierarchy to discover the source.

Assertions can help detect problems as testbenches are developed, allowing earlier debugging. Assertions can also be used to terminate simulation on critical errors, thus saving simulation time.

Writing assertions encourages designers to think more clearly about their designs. This can help highlight problems without specific assertions being written. Verification efficiency is improved by designers having a better understanding of their designs and carrying out more debugging of their code.

Embedded assertions travel with a design as it is used in different projects, warning against unexpected or incorrect use of the IP block.

Embedded assertions also travel with a design from tool to tool (for example, simulation to formal verification), avoiding error-prone re-entry of design behavior definitions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Issues with Properties

Quality of test stimulus for simulation is still necessary

- Assertions are only useful if exercised by the test stimulus
  - My properties are not violated on this simulation run but ...

Extremely complex behaviors can be represented using SystemVerilog assertions.

However, it is important not to get carried away using complex SVA constructs

- A dedicated syntax for assertions is great.
- It is easy to shoot yourself in both feet using complex SVA constructs.
  - The syntax is “sugared”*
- Sometimes writing simple auxiliary Verilog code can help in assertion definition and specification**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

*Sugaring of a syntax means that it looks very easy because there are very few characters used.

However, the reason for this is because difficult concepts have been hidden to allow complex behaviors to be described very concisely.

**This is known as Auxiliary code. It is just HDL code and is used purely for verification. We discuss this at length in a later chapter in the course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

More Issues with Properties

Assertions must be constructed with care

- Incorrect assertions can give misleading results
- Debugging an assertion can be difficult
- How do you know when enough assertions have been written?

Simulation overhead

Formal Verification capacity limitations (complexity)

Other issues, as you will see...

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Summary

Assertions can help reduce debugging time by identifying incorrect design behavior **when** and **where** it occurs

Assertion-Based Verification:

- Captures specifications
- Captures design assumptions
- Documents interfaces
- Provides white-box visibility in simulation
- Supports advanced verification technologies
  - Dynamic simulation
  - Functional coverage analysis
  - Static (formal) verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Static (formal) verification is a symbolic proof, i.e., a mathematical proof. There is no need to think of what stimulus to apply hence there is no testbench.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module 3
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Module Objectives

Aims

- To explain and demonstrate simple assertions based on boolean expressions

Topics

- Terminology
- Immediate assertions
- Concurrent assertions
- Boolean expressions
- Assertion structure
- Controlling evaluation
  - Clock operators
  - Default clocks
- Conditional assertions

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Terminology Introduction

Invariant

- A boolean condition that should always hold (namely, be true)
- For example, A and B are never both high

Temporal sequence

- A series of boolean conditions evaluated over time
  - Hence, a sequence of length 1 cycle is the same as a boolean
- For example, If `A` is high in one cycle, then `B` must be high in the next cycle, and `C` must be high cycle after that

Property

- A design behavior is defined as logical and temporal relationships between Boolean and sequential expressions

It's important to understand that a property and an assertion are not the same thing

An assertion is one example of a "Verification Directive"

- An instruction to the EDA tool to tell it what to do with the property

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

SVA Verification Directives

Property

Expression of design behavior

- A design behavior defined as logical and temporal relationships between boolean and sequential expressions

Verification Directive Keywords

What the tool should do with the expressed design behavior

- assert – An assertion
  - The tool must check the property is always true
- assume – An assumption/constraint
  - In formal – The tool must only allow the DUT and inputs to behave in ways that never violate the property
  - In Simulation – Has no meaning in this context, so it is automatically converted into an assertion
- cover – A cover expression
  - In formal – Demonstrate that it is possible for the expression to be true
  - In Simulation – Count how many times the expression is true
- restrict – A readability intent statement
  - In formal – Exactly the same as assume
  - In Simulation – Has no meaning; hence, is ignored

The stated design behaviors in the SVA properties do not care if we are a simulation or formal user

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

SystemVerilog Assertions (SVA) Overview

Defined as part of the SystemVerilog LRM (IEEE1800)

Very similar to Property Specification Language (PSL) IEEE1850 and the Temporal Language of Specman® ***e*** IEEE1647

Used to specify functional *properties* of a design

- A property is a general behavioral attribute used to characterize a design
  - What it should or should not do

SVA characteristics:

- Mathematically precise and computationally efficient
- Sufficiently expressive to specify “real world” design properties
- Uses Verilog syntax
- Embedded in functional code and ignored by synthesis

Supported by mainstream simulators and formal verification tools

 © Cadence Design Systems, Inc. All rights reserved.

cadence

A property is a general behavioral attribute used to characterize a design. It is similar to stating to a synthesis tool that the design must have the property that the clock can run at 250 MHz.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Immediate Assertions

An assertion as a procedural statement

- Embedded within a procedural block
  - Usually! deferred immediate assertions can also appear in concurrent code
- Only active within the block
- Similar to an if statement

```
always @(posedge clock)
  assert !(wr_en && rd_en);
```

Immediate Assertion

Instantaneous

- Checks a specific Boolean expression when executed in procedural code
- If expression evaluates to `Z`, `X` or `0`, assertion *fails*
- Error message reported when assertion fails

```
always @(posedge clock)
  if (wr_en && rd_en)
    $display("error");
```

Equivalent Verilog

28 © Cadence Design Systems, Inc. All rights reserved.

Immediate assertions are similar to if statements, but note that the assertion fails if the assertion expression evaluates to "untrue," that is, `Z`, `X`, or `0`.

When an assertion fails, a tool-specific message is written to the simulator transcript window containing the following minimum information:

- File name and line number of the assertion
- Assertion label (see next slide) or if unlabelled, the scope of the assertion
- Severity level of the assertion (see second next slide) – default *error*
- Simulation time at which the assertion failed

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Variants of Immediate Assertions

Simple immediate assertions

- Evaluation of the assertion uses the current (immediate) signal values when the line of code containing the assertion is evaluated

```
always @(posedge clock)
  assert !(wr_en && rd_en);
```

Deferred immediate assertions

- Evaluation of the assertion uses the signal values after signal updates in later “regions” of a timeslot
- Allows one to deal with glitches on combination signals in simulation
- Formal is cycle based so deferred immediate assertions are not required

```
always @(posedge clock)
  assert #0 !(full && empty);
always @(posedge clk2)
  assert final (K == L);
```

Google search for the phrase “What is a deferred Immediate Assertion”

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Limitations of Immediate Assertions

Instantaneous boolean check only

Extra code is required around assertion for complex or temporal checks

- Substantial extra code is required to check even a simple protocol

Complexity becomes comparable to that of design

Requires as much debugging as design

Poor readability as design intent is not obvious

However, tools can discern that a check was intended

- Allows verification management tools like Cadence® vManager to recognize a check was specified

```
@(posedge CE);
for (i = 0; i < 16; i++)
begin
  @(posedge SCLK or negedge CE);
  assert (CE)
  else begin // short pulse
    $error("short CE pulse");
    break;
  end
...
```

```
always @(posedge clk)
  case (state)
    `state1:  ...
    `state2:  ...
     default:
       adef: assert(1'b0)
         $warning( "case default");
  endcase
```

Assertion just used to report problem

© Cadence Design Systems, Inc. All rights reserved.

Immediate assertions are useful, are ignored by synthesis tools, and should be used liberally in RTL code.

However, as they are procedural statements checking instantaneous boolean conditions, they usually have to be surrounded by extra code to control when the assertion is actually evaluated. Commonly this extra code is substantial and requires debugging and verification, just like design code.

For complex checks, the immediate assertion is reduced to simply reporting the error rather than detecting it, leaving the assertion with little advantage over using a *$display* statement.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Concurrent Assertions

Used to specify functional *properties* of a design

- A property is a general behavioral attribute used to characterize a design
  - What it should or should not do…

Standalone rules that describe design behavior

They are not procedural code

Evaluated as separate threads in simulation

Formal typically evaluates each assertion in isolation

These are what are used in practice and will be the focus for the remainder of this course

 © Cadence Design Systems, Inc. All rights reserved.

Concurrent assertions are statements inside of hierarchical blocks, for example *modules*.

They describe a standalone behavior which is evaluated in parallel with all other assertions and design behavior.

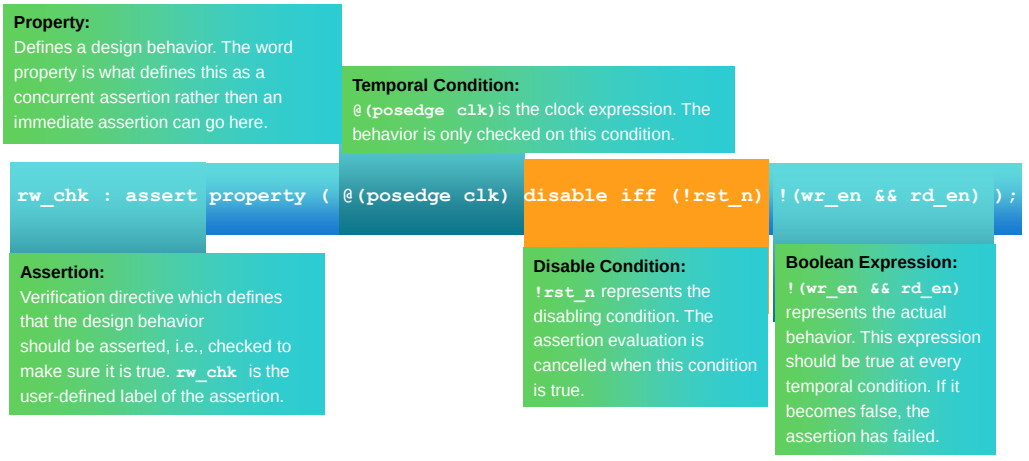
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Concurrent Assertion Structure

Design Behavior

- Signals `wr_en` and `rd_en` are never both high at the positive edge of `clk`



 © Cadence Design Systems, Inc. All rights reserved.

cadence

The temporal condition defines when the behaviour is actually checked. In this example, it is not important what *wr_en* and *rd_en* are doing except at the negedge of some clock signal named *clock*.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Creating a Simple Property

Define a design behavior as a Verilog boolean expression

Any expression allowed in an `if` condition can be used, including:

- Functions
- Out of module references
- Logical and relational operators

```
(EN1 | EN2);
```

```
((A + B) >= 42) && (C | D);
```

Subject to normal Verilog restrictions, particularly:

- Case sensitivity
- Naming rules

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Naming and Asserting the Property

Two options:

1. Label and assert the property in one statement
   This single statement is both the declaration and instantiation of the property

```
ASSERT1 : assert property (EN1 | EN2);
```

2. Name the property and assert in a separate (labeled) statement
   - Property can be used as a building block in other properties
   - Property can have arguments
   - Allows a more flexible approach

Declaration

```
property PROP1;
  (EN1 | EN2);
endproperty
```

Instantiation

```
ASSERT1 : assert property (PROP1);
```

Caution

Brackets for named property in assertion

Tip

Always label Assertions or the tool will assign a meaningless name

Each property instance has its own unique hierarchical name

34 © Cadence Design Systems, Inc. All rights reserved.

Assertions can be defined in 1 of 2 ways:

Single statement assertion – define the boolean condition as an un-named property and assert the property directly. The assertion statement should be labeled so that the assertion can be referenced by verification tools.

Property declaration – declare a property to define the desired behavior and assert the property in a separate statement. The property must have a name and the assertion statement can be labeled. The assertion label or property name can be used by the verification tools to identify the assertion.

The property name or assertion label must be a legal Verilog name that does not conflict with other names in the Verilog namespace.

The second, 2-statement form is more flexible for complex assertions. Separately declaring and naming properties allows the properties to be used as building blocks in more complex assertions, or to be separately used for dynamic assertions, functional coverage or formal verification.

In this course, you use the 2-statement form throughout the course and assume the property is separately asserted if not explicitly shown.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Clocking the Property

Properties must be clocked

- Does not necessarily have to be in the properties declaration

Evaluation of clocked properties controlled by clock edge expression

- Attached to property via clock operator @
- Same syntax as procedural sensitivity lists in RTL Verilog
- Any variable or combination of variables can be used for clock...

```
property PCLK1;
  @(posedge CLK) (EN1 | EN2);
endproperty
```

```
property PCLK2;
  @(EN1 or EN2) (EN1 | EN2);
endproperty
```

```
property PCLK3;
  @(negedge ADDR_EN) ADDR <= 7;
endproperty
```


© Cadence Design Systems, Inc. All rights reserved.

SystemVerilog allows any boolean expression to be used for a clock operator, allowing a property to be evaluated on a complex boolean condition.

Typically RTL synthesis-style clock edge expressions are used to define clock conditions

A property must be clocked by something, but this can come from places other than the property declaration, as you will see later.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Counterintuitive Clock Behavior

Possible to describe properties that don't behave as you may expect

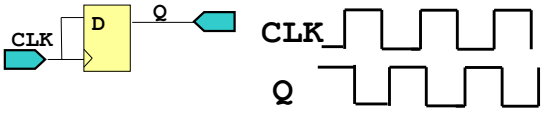
- For example, if using the same signal in clocking expression and in the property definition

LRM is unambiguous but behavior can be counter intuitive



This property fails, always!

Caution




```
P5: assert property (@(posedge clk) clk );
```

LRM defines regions where signals are sampled and where they are updated

The behavior is the same in both simulation and formal

36 © Cadence Design Systems, Inc. All rights reserved.

Yes, @(posedge clk) you may expect that clk has the value '1'.

However, the properties shown always fail in both formal tools and simulators.

LRM defines where values of signals are sampled and updated.

Think as if you were considering the clock and D inputs of a D-type flip-flop and then you will be OK.

If you connect the clock signal to both D input and the clock input of a FF, then the OP will be the inverse of the clock as the value at D is "sampled" before the clock changes.

If you use the same signal in both the property and clocking expression, then maybe you need to consider if you are doing the correct thing.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Clocked Property Evaluation

Objects sampled *before* clock edge

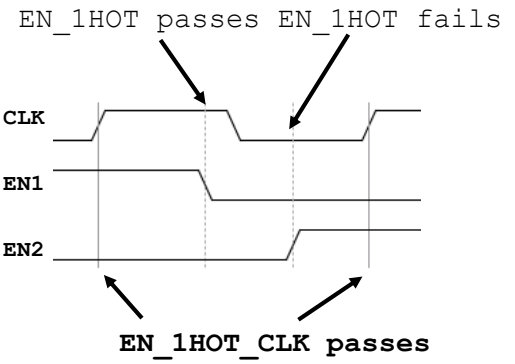
- More details later...

`EN_1HOT` evaluated when `EN1` or `EN2` change value

- 2 evaluations, 1 failure

`EN_1HOT_CLK` evaluated on positive edge of `CLK`

- 2 evaluations, 2 passes




```
property EN_1HOT;
  @(EN1 OR EN2) (EN1 | EN2);
endproperty

property EN_1HOT_CLK;
  @(posedge CLK) (EN1 | EN2);
endproperty
```

37 © Cadence Design Systems, Inc. All rights reserved.

Objects in a property are sampled before the edge on the clocking variable. This is covered in more detail in the Advanced SVA Features module.

The most obvious effect of this is that the following is always true:

```
@(posedge CLK) !CLK;
```

However, in general, it is best to avoid using the same signal in the clocking and boolean parts of a property.

Asynchronous properties should be clocked on changes of any signals in the property expression.

Synchronous properties are clocked on the rising or falling edge of a design clock or other signal.

Notice that the failure of `EN_1HOT` occurred sometime after that “bad” behavior started. Indeed if you end the simulation before `EN2` changes from 0->1, then you would never know that there was a problem.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

Consequences of Counter Intuitive Clock Behavior

At cycle 1 we evaluate the property because EN1 changed from 1 to 0

We evaluate using the preponed value of EN1, which is 1, hence the property passes

At cycle N we evaluate the property because EN2 changed from 0 to 1

We evaluate using the preponed value of EN2, which is 0, hence the property fails

From a verification perspective there are several problems with this description

- We don't find out that we have a failure until cycle N
- Cycle N might be in 1 million cycles from the start of the "bad behavior"
- Cycle N may never occur, namely EN1 or EN2 never change again
- We miss that bug!

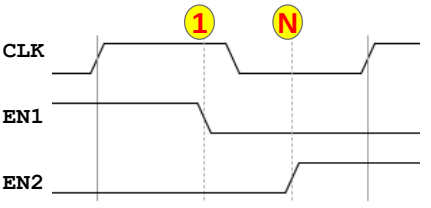
Important rule of thumb

- Do not use the same signal as your property clock and also in the definition of the behavior of the property
- Otherwise there are many things to understand for results not to hide bugs

```
property EN_1HOT;
  @(EN1 OR EN2) (EN1 | EN2);
endproperty
```



Don't write a property like this



38 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What If We Want to Evaluate “Asynchronous” Properties?

Then we must use an appropriate property clock

We could use some existing clock in our design

Otherwise we have to create own clock

```
logic MY_PROP_CLK = 0;
always @(EN1 or EN2)
  MY_PROP_CLK <= !MY_PROP_CLK
```

```
property EN_1HOT_CLKD;
  @(MY_PROP_CLK) (EN1 | EN2);
endproperty
```

Now we see the assertion fail at the exact time/cycle the “bad behavior” is observed

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Should I Use posedge or negedge of the DUT Clock for Properties?

If your design uses `@(posedge clock)` then all of your properties should as well

For simulation this removes the possibility that a GLS assertion fails due to delay annotations

- For example, if sampling on the `negedge clock` then any signal delayed by more than ½ cycle could cause a false negative

For formal it means that the tool can ignore all activity on the negedge clock

- This significantly reduces proof times

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

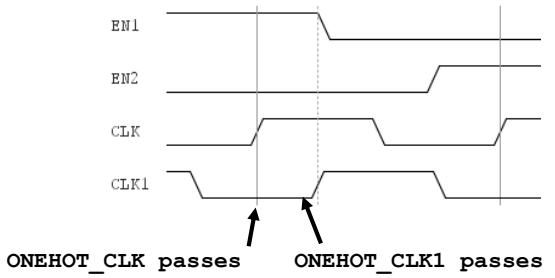
This document is for the sole use of Kuhn Vajay Praveen of Cadence

Default Clock Statement

Painful to specify clock operator for every assertion

Use the default clock block

- Applies to every assertion in the same scope without a specific clock
- Only 1 default clock per scope allowed
- SVA allows for definition of global clocks but these are almost never used
- Google search "SVA clocking explained"




```
default clocking DEFCLK @(posedge CLK);
endclocking

property ONEHOT_CLK;
   (EN1 | EN2);
endproperty

property ONEHOT_CLK1;
  @(posedge CLK1) (EN1 | EN2);
endproperty
```

41 © Cadence Design Systems, Inc. All rights reserved.

cadence

Clocking blocks are a new feature of SystemVerilog, aimed at defining timing and synchronization for testbenches. They are covered in more detail in the SystemVerilog Verification module.

A clock for a SystemVerilog assertion can be declared or inferred in many different ways. You will explore these in greater detail in the advanced chapters of this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Assertion Placement

Placed in a hierarchical block

- Normally a module or interface

Usually placed as a concurrent statement outside of a procedure

- Can be placed inside procedural code
  - Possible issues – Google search "What is the Difference Between a Concurrent SVA Property in Procedural Code and an Immediate Assertion"

Will cause compilation errors with non-SystemVerilog tools

- Disable with `ifdef` conditional compilation
- Allows assertions to be added to existing Verilog models

```
module mod1
  (input logic CLK,
   ...
   output logic EN1, EN2);

'define SVA_ENABLE

'ifdef SVA_ENABLE
  property PROP1;
    @(posedge CLK) (EN1 | EN2);
  endproperty

  ASSERT1: assert property (PROP1);
'endif
...
```

42 © Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog allows concurrent assertions to be placed in any hierarchical block, i.e., Module, Interface, Program block, Package, Clocking block or Compilation unit scope.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

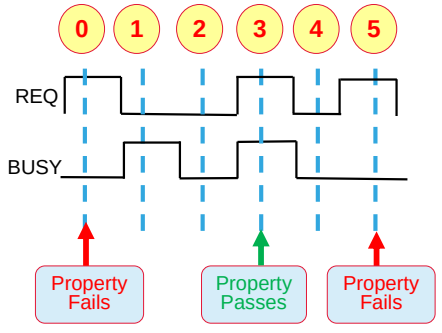
Same Cycle Implication |->

Assertion checks may only be valid under certain conditions

Use logical *if* implication operator **|->**

```
expr1 |-> expr2;
```

- **If** `expr1` is true, **then** `expr2` must be true at the *same* evaluation point




```
property SCI;
  @(posedge CLK)
   REQ |-> BUSY;
endproperty
```


© Cadence Design Systems, Inc. All rights reserved.

At each evaluation point (falling edge of `CLK` in this examples) the *if* condition is checked (`REQ` = logic 1; `ACK` = logic 0). If the condition is false, the assertion is ignored for this clock cycle. If the condition is true, then the *then* condition is checked (`BUSY` = logic 1) in the same evaluation cycle. If the *then* condition is true, then the assertion passes. If the *then* condition is false, the assertion fails.

Both the *if* and *then* conditions must be expressions that return a boolean value.

The *if* condition is also known as the *enabling* condition, and the *then* as the *fulfilling* condition.

This implication operator is called the overlapping implication operator.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

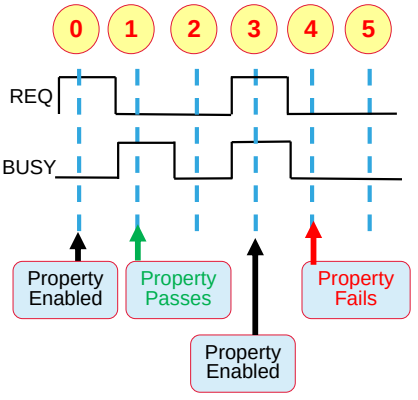
Next Cycle Implication `|=>`

Use `|=>` operator to check a *then* condition in the cycle after an *if* condition

If `expr1` is true, **then** `expr2` must be true at the *next* evaluation point

```
expr1 |=> expr2;
```

```
property NCI;
  @(posedge CLK)
   REQ |=> BUSY;
endproperty
```



44 © Cadence Design Systems, Inc. All rights reserved.

At each evaluation point (falling edge of `CLK` in these examples) the first condition is checked (`REQ` = logic 1; `ACK` = logic 0). If the condition is false, the assertion is ignored for this clock cycle. If the first condition is true, then the second condition is checked on at the *next* evaluation point (`BUSY` = logic 1 at the next falling edge of `CLK`). If the second condition is true, then the assertion passes. If the second condition is false, the assertion fails.

The first condition is also known as the enabling condition. The second condition is known as the fulfilling condition. If the enabling condition is true, then the fulfilling condition must be true for the property to pass.

This implication operator is called the nonoverlapping implication operator.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

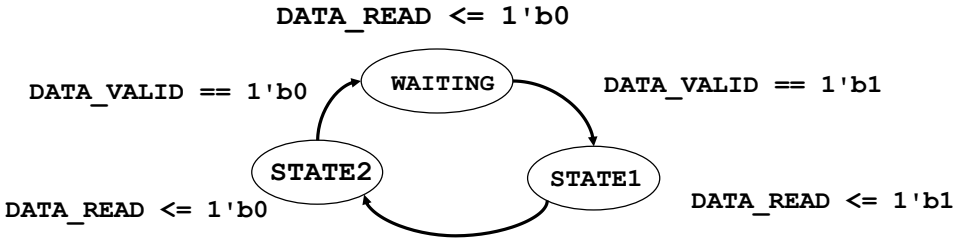
Simple FSM Example

Simple FSM

- `DATA_VALID` is an input
- `DATA_READ` is an output

To write SystemVerilog assertions for verifying FSM:

- Use `|=>` to verify state transitions from current state and inputs
- Use `|->` to verify outputs from current state



© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

FSM Properties

Properties must be asserted to be checked!

```
property FSM1;
  @(posedge CLK) ((STATE == WAITING) && DATA_VALID) |=> (STATE == STATE1);
endproperty

property FSM2;
  @(posedge CLK) (STATE == STATE1) |=> (STATE == STATE2);
endproperty

property FSM3;
  @(posedge CLK) ((STATE == STATE2) && ~DATA_VALID) |=> (STATE == WAITING);
endproperty

property OP_ACTIVE;
  @(posedge CLK) (STATE == STATE1) |-> DATA_READ;
endproperty

property OP_INACTIVE;
  @(posedge CLK) ((STATE == STATE2) | (STATE == WAITING)) |-> ~DATA_READ;
endproperty

AST_FSM_CHK1 : assert property (FSM1);
// ....and so on
```

 © Cadence Design Systems, Inc. All rights reserved.

Properties `FSM1` to `FSM3` check the state transitions. These properties only check that the FSM changes state on the correct input values. They do not check that the FSM does not change state on any other input value. For example, in `FSM1`, if the current state is waiting and `DATA_VALID` is false, then the enabling condition of the property does not complete, and the property does not become active. However, if the state incorrectly changes to `STATE1`, this is not detected as an error. Further properties may be required to check for incorrect transitions, e.g.,

```
property FSM1A ;
  @(posedge CLK) ((STATE == WAITING) && !DATA_VALID) |=>
    (STATE == WAITING);
endproperty
```

The other properties check that the output is active only in the correct state. These properties will detect incorrect outputs due to errors in the output decoding, but will not detect incorrect outputs from the FSM as a result of being in the wrong state.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Assertion Overlapping

Assertions are checked at every evaluation point

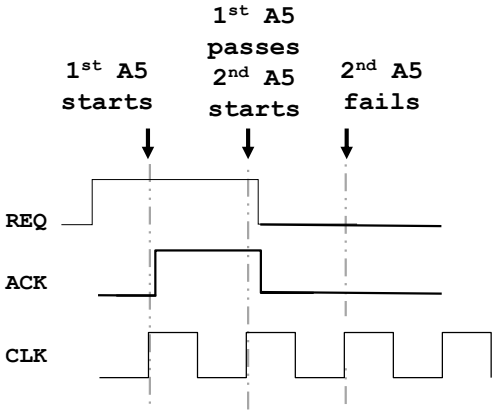
If the start condition is true, a new assertion is triggered

Many copies of an assertion can be active simultaneously

- `REQ` true for two cycles triggers two assertions

More on overlapping later...

```
property A5;
 @(posedge CLK) REQ |=> ACK;
endproperty
```



Caution

Assertions must be carefully written to avoid unexpected failures from overlapping

 © Cadence Design Systems, Inc. All rights reserved.

The enabling condition of every assertion is checked on every evaluation cycle. If the enabling condition is true, then one copy of the assertion becomes active at the evaluation cycle. If the enabling condition is also true at the next or future evaluation cycle, then another copy of the assertion becomes active. If unintentional, this can lead to confusion and possible false failures as multiple, overlapping copies of a single assertion are active.

Assertions defined with level-sensitive enabling conditions are a common cause of overlapping. In the example above, as `REQ` is true for 2 evaluation cycles, the enabling condition (`REQ`) triggers 2 copies of the assertion. For this example, a single `ACK` fulfils one of the active assertions, but the other fails.

Overlapping is a particular issue for assertions defined using sequences. The next chapter provides more detail.

The above assertion could be made edge-sensitive by using a sequence or a built-in function like `$rose`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

$rose and $fell

Built-in functions for detecting value change

Useful for making properties edge-triggered

- One way to avoid overlapping

  `$rose(expr)`

- True if `expr` has changed value and is 1 in the current cycle

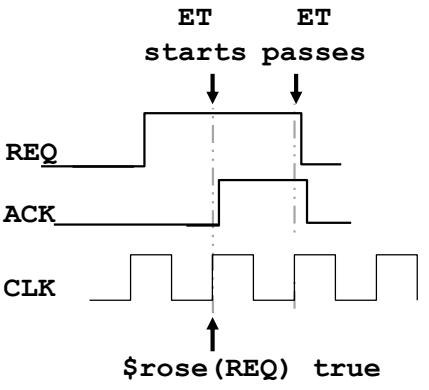
  `$fell(expr)`

- True if `expr` has changed value and is 0 in the current cycle

Other built-in functions useful for simple boolean assertions...

$rose() is also true for X->1 or Z->1 changes

```
property ET;
 @(posedge CLK)
  $rose(REQ) |=> ACK;
endproperty
```



Caution

© Cadence Design Systems, Inc. All rights reserved.

Using `$rose` or `$fell` to make the enabling condition of a property edge-triggered rather than level-sensitive is one easy way to avoid overlapping assertions.

This course provides other methods later.

Note `$rose(A)` is not the equivalent of `(!A ##1 A)`,

it is the equivalent of `(A != 1'b1 ##1 A)`

(`##` syntax explained in chapter on Sequences)

i.e., it is a transition to `1'b1` from any other value, not only from `1'b0` to `1'b1`.

© Cadence Design Systems, Inc. All rights reserved.

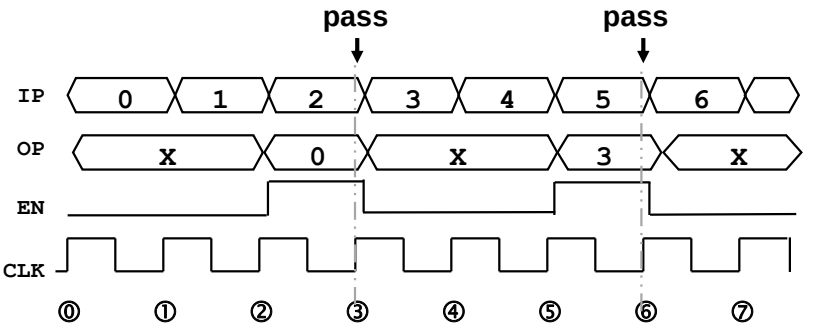
This document is for the sole use of Kuhn Vajay Praveen of Cadence

$past

Returns the value of A from a previous evaluation cycle

- N defaults to 1, i.e., last cycle

```
$past(A, N)
```




```
property PAST1;
  @(posedge CLK) EN |-> (OP == $past(IP,2));
endproperty
```


© Cadence Design Systems, Inc. All rights reserved.

Past has 1 mandatory parameter, the expression whose past value one desires, and 3 optional ones.

Only one of the optional parameters is shown.

Note that the mandatory parameter can be any expression, for example (C && D) || E

See the IEEE 1800-2009 for details.

© Cadence Design Systems, Inc. All rights reserved.

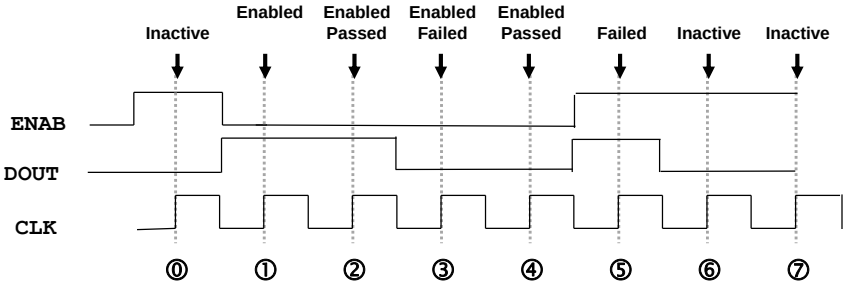
This document is for the sole use of Kuhn Vajay Praveen of Cadence

$stable()

Returns a boolean `true` if `expr1` has the same value as it did in the previous cycle

$stable(expr1)

```
property STABLE1;
  @(posedge CLK) ~ENAB |=> $stable(DOUT);
endproperty
```



 © Cadence Design Systems, Inc. All rights reserved.

The same behavior can be described with the built-in `$past()` function.

```
property SAME;
   @(negedge CLK)
   ( ~ENAB |=> (DOUT == $past(DOUT))  );
endproperty
```

In contrast to `$past()`, stable does not have an argument to specify the number of cycles over which the object has been stable.

IEE2009 adds a function $changed(expr) which returns true if expr changed from the previous cycle.

This is the logical inverse of $stable(expr).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

$countones() and $isunknown()

`$countones()`

- Returns an integer representing the number of 1's in `expr1`

`$countones(expr1)`

- Any value other than 1 ignored

```
property CHK_ONEHOT;
  @(posedge CLK)
  ($countones(GRANT_VEC) == 1 );
endproperty
```



Caution

Function can be used as an integer for example

`$isunknown()`

- Returns a boolean `true` if <u>any</u> bit in `expr1` is `x` or `z`

`$isunknown(expr1)`

```
property CHK_UNKNOWN;
  @(posedge CLK)
  (~EN |-> $isunknown(TS_BUS));
endproperty
```

Check that if `~EN`, then some bits of `TS_BUS` are driven to 'X' or 'Z'

51 © Cadence Design Systems, Inc. All rights reserved.

The argument of either function is any reg, wire or net type or any word in a memory .

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

$onehot() and $onehot0()

`$onehot` returns a boolean `true` only if *exactly* one bit in `expr1` is `1`

`$onehot0` returns a boolean `true` only if, *at most*, one bit in `expr2` is `1`

$onehot(expr1)

$onehot0(expr1)

```
property OH;
  @(posedge CLK)
  ($onehot(GRANT_VEC));
endproperty

property OH_ZERO;
  @(posedge CLK)
  ($onehot0(GRANT_VEC));
endproperty
```

GRANT_VEC	OH	OH_ZERO
"XX1Z"	Pass	Pass
"0000"	Fail	Pass
"0100"	Pass	Pass
"0Z00"	Fail	Pass
"1111"	Fail	Fail

52 © Cadence Design Systems, Inc. All rights reserved.

cadence

The argument is any reg, wire or net type or any word in a memory.

If you wanted to check for a “real” one hot, i.e., GRANT_VEC having the following set of values, “0001”, “0010”, “0100”, “1000”.

Then you can combine `onehot` and `isunknown` like so:

```
property COMB;
  @(posedge clk)
  $onehot(GRANT_VEC) && (!$isunknown(GRANT_VEC)) ;
endproperty
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Summary

SystemVerilog assertions are based on Verilog boolean conditions

Constructing a simple SVA assertion is straight-forward

- Write a boolean condition using Verilog syntax
- Define when the condition is checked with a default or explicit clock
- Define when the condition should not be checked through a `disable iff`
- Name the condition using the `property` statement
- Assert the property using an `assert` statement

Built-in functions can help

- Avoiding overlapping
- Common checks

Conditional checks are the first step toward more complex assertions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

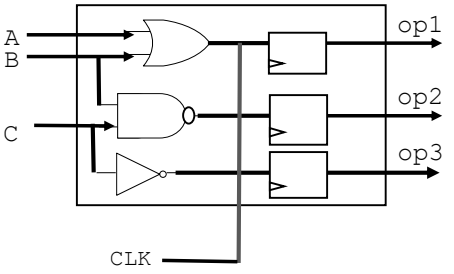
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Quiz

Write assertion(s) to check the behavior of the circuit:

- Don’t be concerned about reset for this example
- Use the posedge of CLK to clock the properties



 © Cadence Design Systems, Inc. All rights reserved.

Hopefully, this quiz gets you in the mindset of writing assertions.

Don’t worry if it looks harder than you thought; this is the feeling most people have initially.

It’s recommended to try and forget about procedural code like Verilog, C/C++, VHDL,

Knowing when you have written enough assertions to cover all desired behavior is a very difficult task.

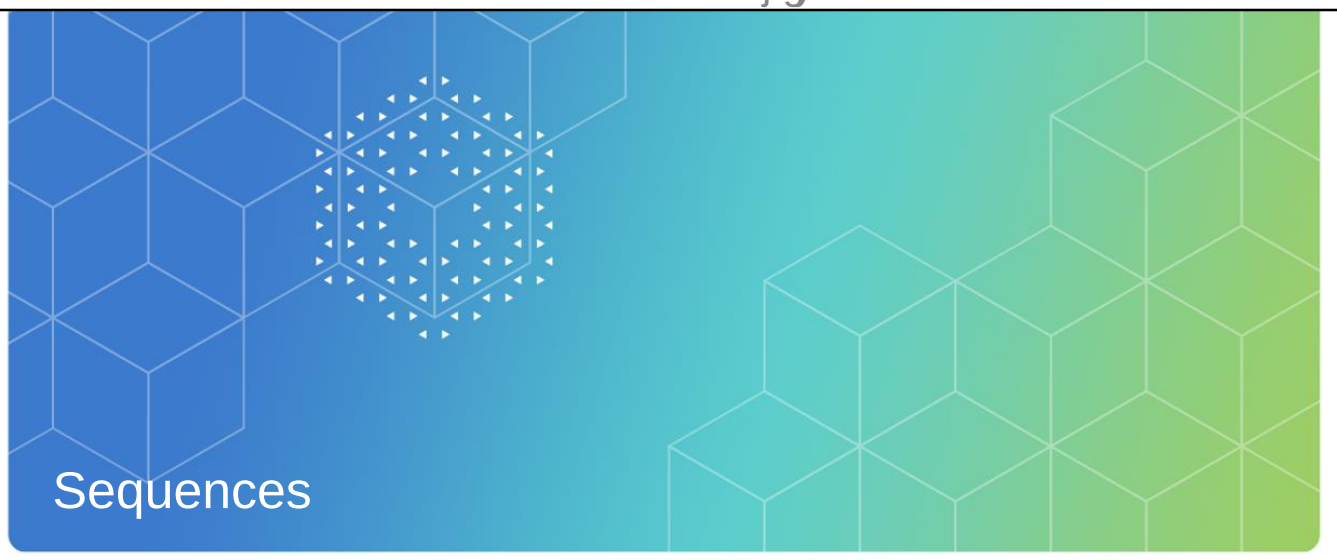
In fact, it is so hard that tools cannot do this for you automatically.

This is called the *completeness* problem and is applicable to verification in general, not just using properties or formal verification. This course provides more detail later.

Often with assertions to specify completely the behavior which you want to be checked, you may need to check the output value in the context of previous inputs and the next output value in the context of the current inputs.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module	**4**
Revision	**1.0**
Version	**5.1**

Estimated Time:

- **Lecture**
- **Lab**

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

Aim

- Explain sequences and show sequence-based assertions

Topics

- Introduction to sequences
- Conditional sequence operators
- Repetition operators
- Complex assertion construction with named sequences

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Sequences

Temporal properties are described using sequences

- Series of boolean equations
- Each cycle separated by `##N`

Sequences are building blocks of properties

Normally sequences are defined as unclocked

They inherit their parents clock, for example, when used in an assertion

Wide range of powerful operators and features to construct complex sequences

- if-then implication
- Replication
- Named sequences
- Composition operators

```
sequence MYSEQ;
 A ##1 B ##1 C;
endsequence
```

```
AST_SEQ1: assert property
(@(posedge CLK) START |=> MYSEQ;
```

MYSEQ inherits the assertion AST_SEQ1's clock

© Cadence Design Systems, Inc. All rights reserved.

Simple boolean assertions are instantaneous. To define complex design properties, you need to define temporal properties where boolean equations are evaluated over successive cycles. Sequences let us do this.

is the cycle delay operator. It joins separate boolean equations into a sequence where the equations are examined on successive evaluation cycles.

© Cadence Design Systems, Inc. All rights reserved.

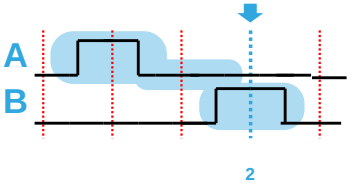
This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Other Sequence Examples

- Sometimes it is necessary to capture sequence of events in properties
- Most sequences are described using the `##` operator

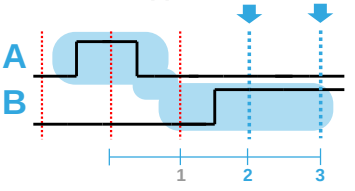
`A ##2 B`

"A happens then exactly 2 cycles later B happens"



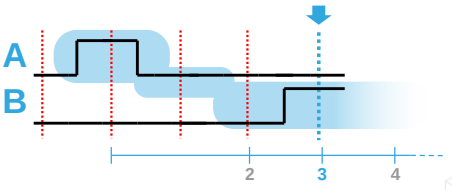
`A ##[1:3] B`

"A happens then 1 to 3 cycles later B happens"



`A ##[3:$] B`

"A happens then 3 or more cycles later B happens"



- **It important to remember that a sequence of length 1 cycle is a boolean**

58 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Sequence Implication

Conditional sequences:

- *If* sequence `SEQ_A` completes, *then* sequence `SEQ_B` must occur
- If enabling condition starts but does not complete, property does not fail

```
SEQ_A |-> SEQ_B;
```

Two implication operators:

`|->`

- If `SEQ_A` completes, assertion looks for `SEQ_B` to begin in the *same* cycle
- Known as overlapping sequence implication

```
SEQ_A |=> SEQ_B;
```

`|=>`

- If `SEQ_A` completes, assertion looks for `SEQ_B` to begin in the *next* cycle
- Known as non-overlapping sequence implication

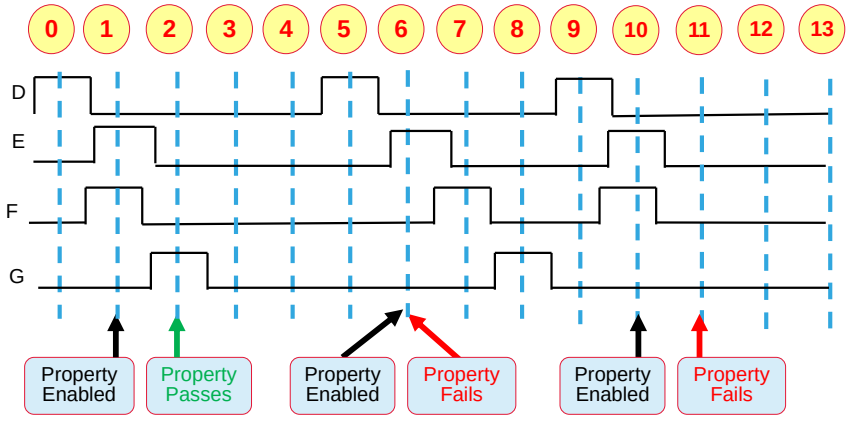
© Cadence Design Systems, Inc. All rights reserved.

The implication operators for simple boolean assertions can also be used for sequence assertions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Overlapping Sequence Implication Example




```
property OVRLAP_SEQ;
  @(posedge clk) D ## E |-> F ##1 G;
endproperty
```

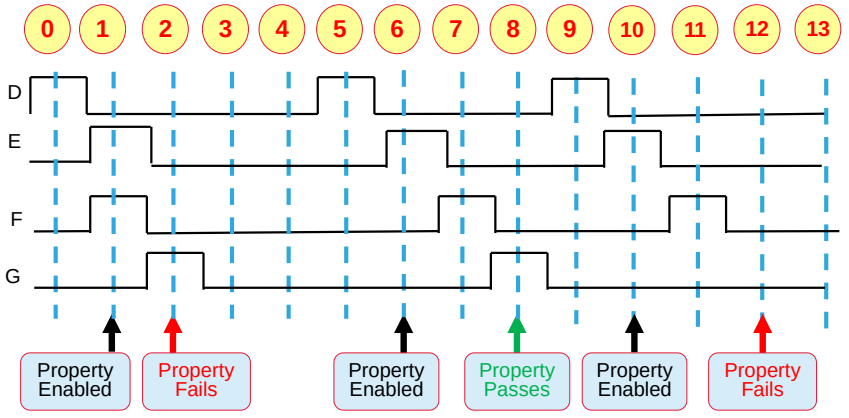

© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Non-Overlapping Sequence Implication Example




```
property NON_OVRLAP_SEQ;
  @(posedge clk) D ## E |=> F ##1 G;
endproperty
```


© Cadence Design Systems, Inc. All rights reserved.

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Conditional and Unconditional Properties

Properties are either:

- Unconditional using instantaneous boolean expressions
- Conditional using implication operators
  - *If* enabling condition, *then* fulfilling condition

Avoid unconditional sequence properties

- Requires `(A ##1 B #1 C)` to be true at *every* evaluation point
- This means:
  - A must be true at every cycle
  - B must be true at the second and all consequent cycles
  - C must be true at the third and all consequent cycles

unconditional

```
property UNCOND;
  @(posedge clk) (COUNT > 10);
endproperty
```

conditional

```
property COND;
  @(posedge clk) A |=> C ##1 D;
endproperty
```

if — enabling sequence — then — fulfilling sequence

```
property AVOID;
  @(posedge clk) A ##1 B ##1 C;
endproperty
```

equivalent to:

```
(1'b1 |-> (A ##1 B ##1 C);
```

62 © Cadence Design Systems, Inc. All rights reserved.

SystemVerilog allows sequence properties to be defined like this:

```
@(posedge CLK)(A ##1 B ##1 C);
```

However, remember that properties are evaluated on every cycle. On the first cycle, a copy of the assertion is triggered and A must be true. On the second cycle B must be true to keep the first assertion true, but we also trigger a second copy of the assertion, which requires A to be true in the second cycle also. In the third cycle, C must be true for the first assertion, B must be true for the second, and A true again for a new third copy of the assertion. Therefore A, B and C must be true for every evaluation point.

Properties are rarely written in this way. Condition assertions using implication operators are much more useful when using sequences.

Terminology: the enabling sequence or condition is also referred to as the *Antecedent* or cause. The fulfilling sequence/condition is also referred to as the *Consequent* or effect.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Never Properties

Often easier to specify bad behavior than every good behavior, for example:

- Count should never be greater than 10
- Should never see `BUSY` followed by `GRANT`

Use the `not` keyword in front of the bad behavior

Do not use with implication operators

- Only with booleans or sequences
- Although it would be syntactically correct

```
property NEVER_BOOL;
  @(posedge clk) !(COUNT > 10);
endproperty
```

```
property NEVER_SEQ;
  @(posedge clk)
            not (BUSY ##1 GRANT);
endproperty
```

a	b	AVOID Result	not AVOID Result
0	0	Pass	Fail
0	1	Pass	Fail
1	0	Fail	Pass
1	1	Pass	Fail



Caution

```
property AVOID;
  @(posedge clk) not (A |=> B) ;
endproperty
```

Not intuitive what this actually means

© Cadence Design Systems, Inc. All rights reserved.

Using implication with `not` operators falls outside of what is known as a property simple sub-set.

This basically means properties which can be evaluated easily by a simulator where time progresses linearly from left to right.

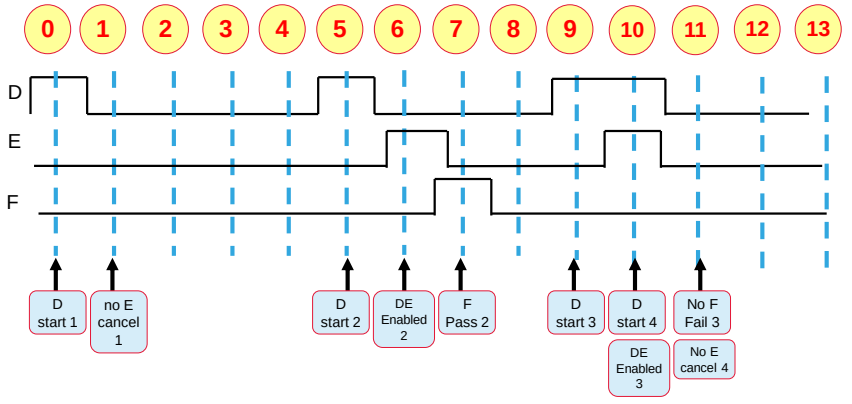
Think of it as similar to the synthesisable subset of Verilog.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prakash of Cadence

Analysis of Overlapping Sequence Property Evaluations

```
property SQA;
  @(posedge CLK) (D ##1 E) |=> F;
endproperty
```



64 © Cadence Design Systems, Inc. All rights reserved.

Remember that the enabling sequence on the left side of the implication operator must complete before the fulfilling sequence on the right side is checked.

Remember also that assertions can overlap.

At cycle 1, `A` is true so copy 1 of `SQA` becomes active. At cycle 2, the second condition of the enabling sequence, `B`, is false so copy 1 of `SQA` terminates.

At cycle 3, `A` is true so copy 2 of `SQA` becomes active. At cycle 4, `B` is true and copy 2 of `SQA` becomes enabled. `A` is false at cycle 4 so another copy of `SQA` does not become active. At cycle 5, the fulfilling condition `C` is true and copy 2 of `SQA` passes.

At cycle 6, `A` is true and copy 3 of `SQA` becomes active. At cycle 7, `B` is true so copy 3 becomes enabled. `A` is still true at cycle 7, so copy 4 of `SQA` becomes active.

At cycle 8, `C` is not true, so copy 3 fails. Also, `B` is not true, so copy 4 terminates.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

$rose and $fell in Sequences

$rose and $fell can be used in sequences too

Useful for making sequences edge-triggered and avoiding overlapping

`$rose(expr)`

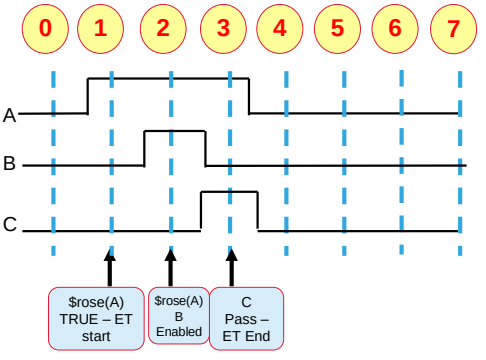
- True if `expr` has changed value and is 1 in the current cycle

`$fell(expr)`

- True if `expr` has changed value and is 0 in the current cycle

Other built-in functions covered later...

```
property ET;
  @(posedge CLK)
  $rose(A) ##1 B |=> C;
endproperty
```



© Cadence Design Systems, Inc. All rights reserved.

Using `$rose` or `$fell` to make the enabling condition of a property edge-triggered rather than level-sensitive is one easy way to avoid overlapping assertions.

This course shows other methods later on.

Note `$rose(A)` is not the equivalent of `(!A ##1 A)`.

It is the equivalent of `(A != 1'b1 ##1 A)`.

For example, it is a transition to `1'b1` from any other value, not only from `1'b0` to `1'b1`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Disabling Properties

To terminate an assertion when a condition is true, use `disable iff`

If `expr` is true at any point, then the property evaluation is terminated (passes)

Evaluation of `disable iff` is asynchronous, i.e., the clock is irrelevant

Disable expression must be bracketed `()`

Useful for cancelling multicycle sequence properties

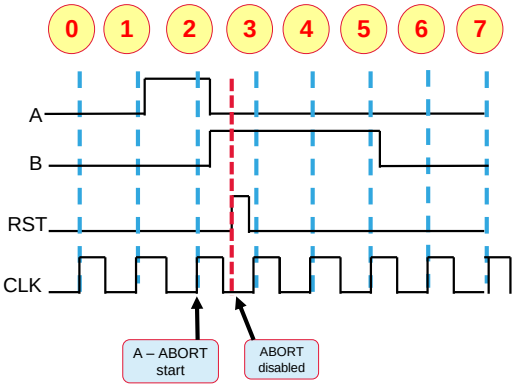
- For example bus transactions which are interrupted by a reset

```
property ABORT;
  @(posedge CLK) disable iff (RST)
    A |=> B ##1 C ##1 D;
endproperty
```

Caution

Disabled properties pass!

```
property NAME;
  @(clocking) disable iff (expr)
   property;
endproperty
```



© Cadence Design Systems, Inc. All rights reserved.

Disable is essential for terminating long sequence properties if design conditions change, for example. In this case, a reset occurs.

If the disable condition (`RST`=logic 1) is true, then the assertion is disabled (terminated) immediately, regardless of how the property is clocked.

You can view the disable as being asynchronous from the property sampling clock.

Note: Depending upon which tool you are using, disabled property may be defined as a pass, just as if the property had successfully completed. They are not counted as a fail. This can possibly lead to misleading results when a simulator reports the number of design properties that have passed or failed.

The reason for this is that in Formal Verification (where assertion languages were first used), the absence of a failure is deemed as a pass.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Default Disable

IEEE 1800-2009 introduced a default disable

Similar concept to default clocking

`default disable iff` only applies to the scope in which it is defined

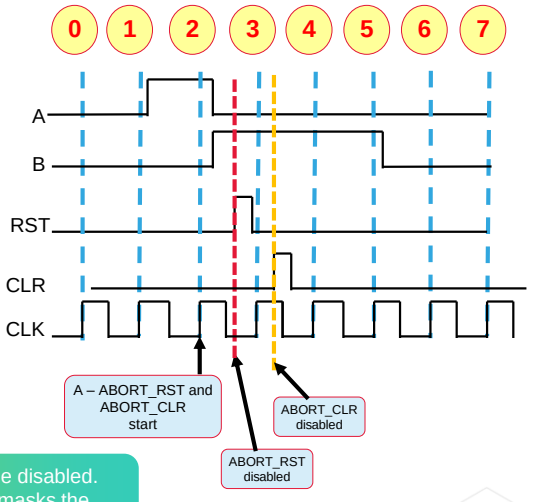
Explicit `disable iff` overrides the default

```
default disable iff (RST);
property ABORT_RST;
  @(posedge CLK)
    A |=> B ##1 C ##1 D;
endproperty

property ABORT_CLR;
  @(posedge CLK) disable iff (CLR)
    A |=> B ##1 C ##1 D;
endproperty

property NO_DISABLE;
  @(posedge CLK) disable iff (1'b0)
    A |=> B ##1 C ##1 D;
endproperty
```

Can never be disabled. Effectively masks the `default disable iff`



67 © Cadence Design Systems, Inc. All rights reserved.

Just as specifying the same clocking expression for every property would be painful, especially if you wanted to change that clock, then specifying the same disable condition is also.

The behavior is very similar to that of default clocking. A default disable clause cannot have a name, unlike default clocking.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Synchronous Abort Operators

`disable iff` is asynchronous

- In simulation this will likely lead to problems on reset signals which have glitches

For synchronous abort we choose whether property should pass or fail when abort condition occurs

`sync_accept_on`

- For example, if we observe `A ##1 B && CLR ##1 !C` then the property is deemed to have passed on the cycle where CLR occurred

```
ABORT_PASS_ON_CLR: assert property @(posedge CLK)
          A |=> sync_accept_on(CLR)  (B ##1 C ##1 D);
```

`sync_reject_on`

- For example, if we observe `A ##1 B ##1 C && CLR ##1 D` then the property is deemed to have failed on the cycle where CLR occurred

```
ABORT_PASS_ON_CLR: assert property @(posedge CLK)
          A |=> sync_reject_on(CLR)  (B ##1 C ##1 D);
```

There are also asynchronous forms of these, `accept_on` and `reject_on`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

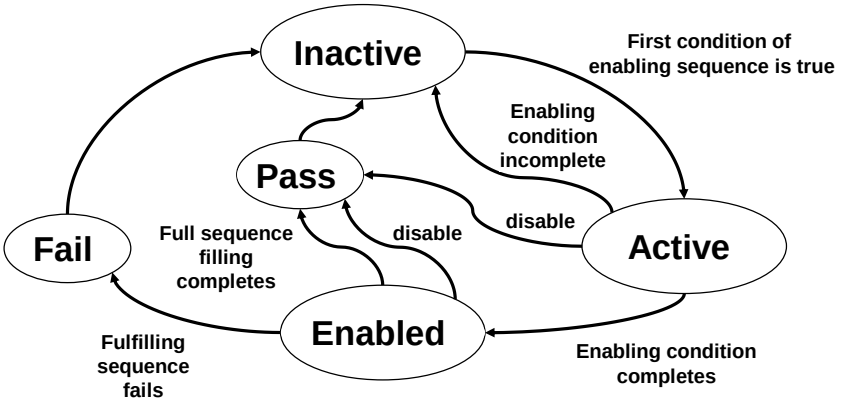
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Assertion Status

An assertion can be in one of several states:




```
@(clocking) disable iff (dis_expr) enable |=> fulfill;
```

69 © Cadence Design Systems, Inc. All rights reserved.

Assertions start in the inactive state. At each evaluation cycle, the enabling condition (or the first condition of an enabling sequence) is checked. If the condition is true, a copy of the assertion becomes active. In the active state, the assertion checks the enabling condition or sequence. If the enabling condition(s) fails, then the assertion terminates. If the enabling condition(s) complete, then the assertion is enabled. In the enabled state, the assertion checks the fulfilling condition(s). If the fulfilling conditions fail, then in the evaluation cycle the failure was detected, the state of the assertion is fail, after which it becomes inactive. If the fulfilling condition(s) are true, then the assertion state is pass in the evaluation cycle the final fulfilling condition becomes true. An abort can terminate an active or an enabled assertion.

Remember the enabling condition of every assertion is checked on every evaluation cycle; therefore, multiple versions of a single assertion can exist in different states. This can make management and debugging of assertions difficult.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Cycle Delay Repetition

`##` can be used with any static integer to define a number of cycle delays

After `expr1`, count `N` evaluation points, after which `expr2` must be true

```
expr1 ##N expr2
```

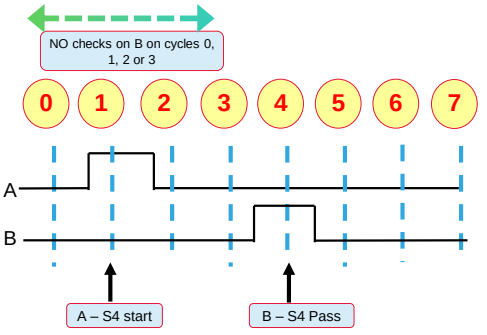
`##0` is allowed (and useful)

- Zero delay ("fusion")

```
property S4;
  @(posedge CLK)
   A |-> ##3 B;
endproperty
```

```
property S4_EQUIV;
  @(posedge CLK)
   A |=> ##2 B;
endproperty
```

This property is exactly equivalent to S4 but is slightly less intuitive to count the cycles



70 © Cadence Design Systems, Inc. All rights reserved.

Cycle delay `##` just counts evaluation cycles without checking any conditions. Any static integer value can be used with the operator. This can be very useful for defining a number of cycle delays between conditions.

Note:

`##0` = current evaluation cycle

`##1` = next evaluation cycle

`##2` = next but 1 evaluation cycle

etc.

Zero delay is also referred to as fusion or overlapping, for example.

```
(A ##1 B) ##0 (C ##1 D)
        = (A ##1 B && C ##1 D)
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

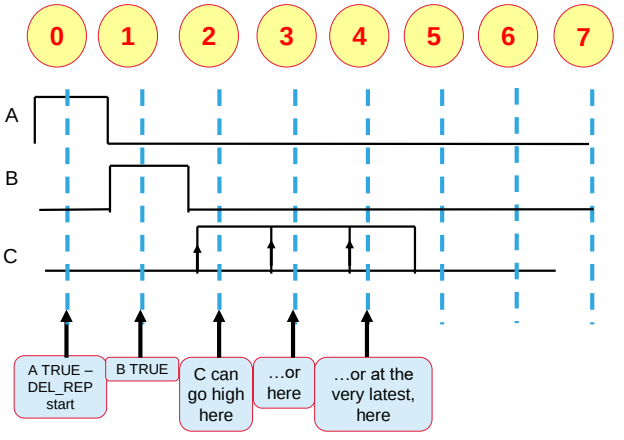
Cycle Delay Repetition Ranges

`##` can also be used with a range of integers

- Range of cycle delays between conditions

```
expr1  ##[min:max] expr2
```

After `expr1`, `expr2` must be true between `min` and `max` cycles later




```
property DEL_REP;
  @(posedge CLK)
   A |=> B ##[1:3] C;
endproperty
```


© Cadence Design Systems, Inc. All rights reserved.

Min and max must both be static integers, i.e., defined with literals, constants or constant expressions.

Min must be less than or equal to max.

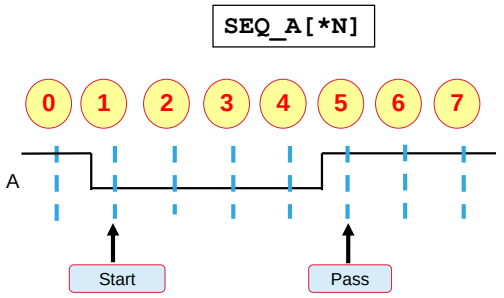
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Consecutive Repetition

Repeated, consecutive sequences can be defined using `[*N]`

- Consecutive repetition operator
- `SEQ_A` repeated N times



For example, A is never low for more than 4 cycles:

instead of this...

```
property SEQ_COUNT;
  @(negedge CLK) !A ##1 !A ##1 !A ##1 !A |=> A;
endproperty
```

write this...

```
property CONSEC_REP;
  @(negedge CLK) !A[*4] |=> A;
endproperty
```

72 © Cadence Design Systems, Inc. All rights reserved.

Consecutive repetition allows an individual condition or a complete sequence to repeat, consecutively, a set number of times. For example:

`(A[*4])` is equivalent to `(A ##1 A ##1 A ##1 A)`

`((A ##1 B)[*2])` is equivalent to `(A ##1 B ##1 A ##1 B)`

The count value `N` must be statically computable, i.e., known at compilation time and not defined by a variable or expression.

© Cadence Design Systems, Inc. All rights reserved.

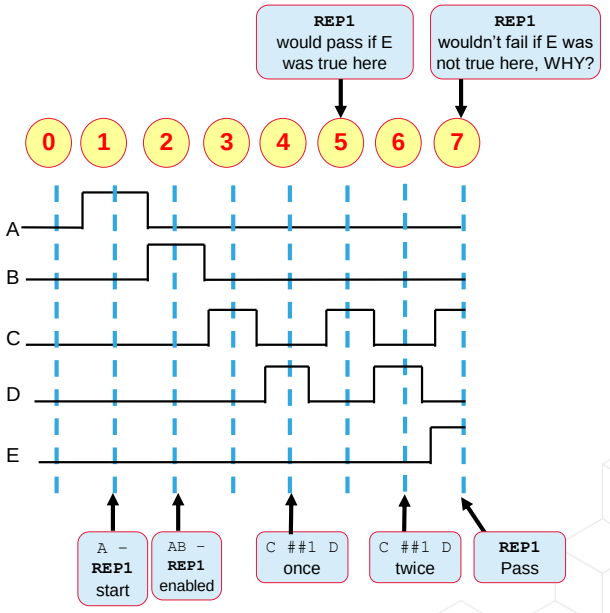
This document is for the sole use of Kuhn Vajay Praveen of Cadence

Consecutive Repetition with Ranges

For a variable number of repetitions, a range can be defined

- `SEQ_A` will repeat from `min` to `max` times
- Remember that a sequence of length 1 cycle is a boolean

```
sequence SEQ_CD;
  C ##1 D;
endsequence
property REP1;
  @(posedge CLK) A ##1 B |=>
                 SEQ_CD[*1:3] ##1 E;
endproperty
```



73 © Cadence Design Systems, Inc. All rights reserved.

For a varied number of consecutive repetitions, a range can be defined. Both bounds of the range must be statically computable (like a constant expression) and must be defined in an ascending direction.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Jayaraj Pawashe of Cadence

Consecutive Repetition: Special Ranges

Lower bound can be 0

- No minimum number of repetitions
- Sequence may be absent

```
B ##1 A[*0:2] ##2 C
```

This is known as an empty sequence because it contains a term (A[*0]) which is not required to occur. Google search for Empty Sequences in SVA Explained

```
(B ##1 A[*0] ##2 C) or (B ##1 A[*1] ##2 C)  or (B ##1 A[*2] ##2 C)
```

```
using the LRM rule (empty ##n seq) is equivalent to (##(n-1) seq) for n>0
```

```
(B ##1 ##1 C) or (B ##1 A[*1] ##2 C)  or (B ##1 A[*2] ##2 C)
(B ##2 C)     or (B ##1 A[*1] ##2 C)  or (B ##1 A[*2] ##2 C)
```

Upper bound can be $ (infinity)

- No maximum number of repetitions
- Sequence may never end

```
A[*2:$] ##1 B
```

```
(A ##1 A ##1 B) (A ##1 A ##1 A...)
```

74 © Cadence Design Systems, Inc. All rights reserved.

Consecutive ranges using $ must be used with caution. Unless additional information is supplied to apply limits to the sequence (see the Sequence Composition topic) an infinite sequence may never complete.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Nonconsecutive Repetition

Repeated, *nonconsecutive* sequence expressions can be defined using `[=N]`

Nonconsecutive means `not necessarily` consecutive

`expr1` must occur N times

When `A` is high, then from the same cycle, there must be 2 cycles of `B` and then `C`

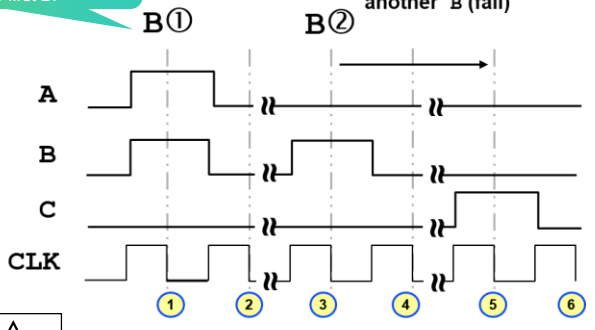
```
property NONCONSEC_REP;
  @(negedge CLK)
  A |-> B[=2] ##1 C;
endproperty
```

IMPORTANT : The sub-term B[=2] is true from the second occurrence of B until the *cycle before* the 3rd occurrence of B

`expr1[=N]`

B did not need to occur the same cycle as A. We can wait any number of cycles for the first B.

`B[=2]` remains active until `C` (pass) or another `B` (fail)



Caution

No restriction on number of cycles between the first and second B
No restriction on number of cycles between second B and C

75 © Cadence Design Systems, Inc. All rights reserved.

A sequence defined with `[=N]` does not terminate after `N` occurrences. It remains active until the next condition in the sequence occurs. Therefore there can be any number of cycles between the `[=N]` boolean condition and the following boolean.

The nonconsecutive repetition operator defines a “window” for a number of repetitions of a condition. In this example, between `A` and `C`, there must be 2 occurrences of `B` only. After the second occurrence of `B`, there can be any number of cycles before `C`. This is a weak property, i.e., if `C` never occurs, the property will not fail as long as there are only 2 occurrences of `B` after `A`. Also, if there are never 2 more occurrences of `B`, then again, the property will not complete.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Go-To Repetition

The go-to repetition operator `[->N]` is similar to the `[=]` operator

However `expr1[->N]` ends on the Nth occurrence of `expr1`

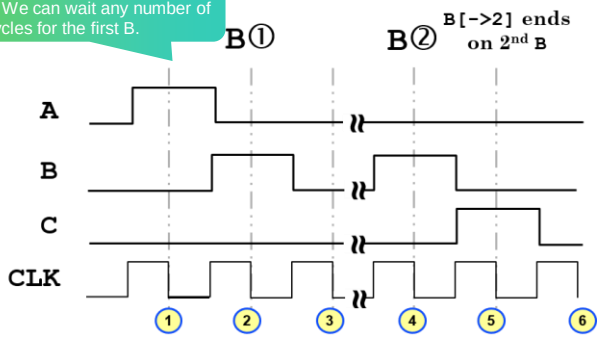
When A is high, then from the same cycle, there must be 2 cycles of B before C, with C immediately following the second B

```
property GOTO_REP;
  @(negedge CLK)
  A |-> B[->2] ##1 C;
endproperty
```

IMPORTANT : The sub-term B[->2] ends on the second occurrence of B

expr1[->N]

B doesn't need to occur the same cycle as A. We can wait any number of cycles for the first B.



Caution

No restriction on number of cycles between the first and second B
Last occurrence of B must be followed immediately by C

76 © Cadence Design Systems, Inc. All rights reserved.

A sequence defined with `[->N]` terminates after `N` occurrences. The next condition in the sequence must occur immediately on the next cycle.

In this example, after `A`, there must be 2 occurrences of `B` before `C`. These can be nonconsecutive and take any number of cycles to complete. However, `C` must occur in the cycle immediately following the second occurrence of `B`. This is a weak assertion, i.e., if 2 occurrences of `B` are never seen, the property will not fail.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Go-To and Nonconsecutive Ranges

Ranges can be used with both operators

- From `A`, there must be between one and 2 occurrences of `B` followed by `C`

```
A ##1 B[=1:2] ##1 C
```

- From `D`, there must be between 3 and 4 occurrences of `E` followed by `F`. `F` must immediately follow the 3rd or 4th occurrence of `E`

```
D ##1 E[->3:4] ##1 F
```

Upper bound of both operators can be `$`

- Use with care!

```
A ##1 B[=0:1] ##1 C
```

Lowest bound of can be 0

- No minimum number of occurrences

```
( A ##1 C)  or (A ##1 B[=1] ##1 C)
```

You will see examples with waveforms on the next slide ...

```
D ##1 E[->0:1] ##1 F
```

```
( D ##1 F)  or (D ##1 E[->1] ##1 F)
```

77 © Cadence Design Systems, Inc. All rights reserved.

cadence

`expr1[->]` defines a sequence that terminates with the first occurrence of `expr1`.

© Cadence Design Systems, Inc. All rights reserved.

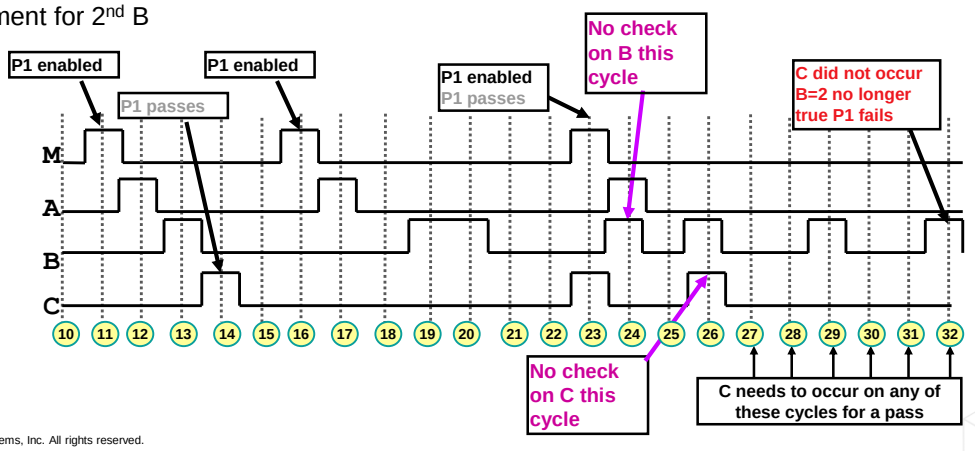
This document is for the sole use of Kuhn Vajay Praveen of Cadence

Nonconsecutive Repetition Range Example

This property states:

- No limit on # cycles between A and 1st B
- No limit on # cycles between 1st and 2nd B
- No limit on # cycles between 2nd B and C
- No requirement for 2nd B

```
property P1;
  ( M |=> A ##1 B[=1:2] ##1 C);
endproperty;
```



78 © Cadence Design Systems, Inc. All rights reserved.

cadence

An occurrence of C on the last cycle shown where the 3rd occurrence of B would still be a pass, as B[=2] was true in the previous cycle.

© Cadence Design Systems, Inc. All rights reserved.

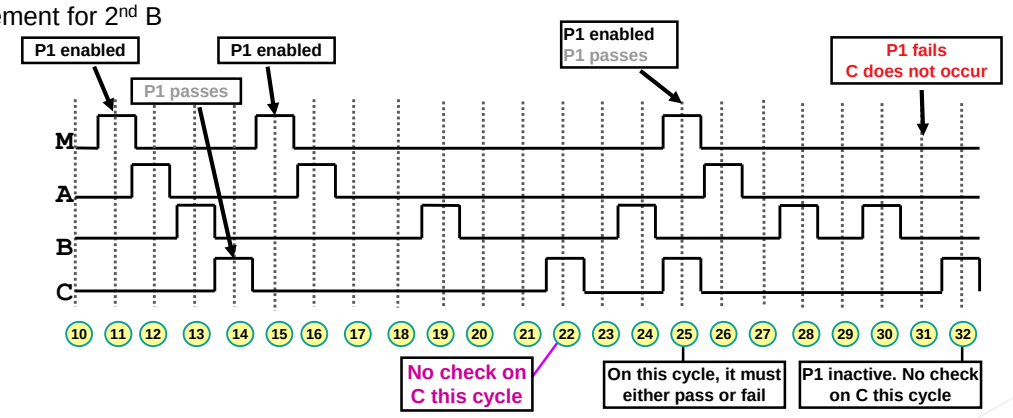
This document is for the sole use of Kuhn Vajay Prasad of Cadence

Go-To Repetition Range Example

This property states:

- No limit on # cycles between A and 1st B
- No limit on # cycles between 1st and 2nd B
- C **must** occur on cycle after 1st or 2nd B
- No requirement for 2nd B

```
property P1;
  ( M |=> A ##1 B[->1:2] ##1 C);
endproperty;
```



© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Praveen of Cadence

Repetition Shorthand

- Shorthand for common sequence repetitions
- Can also be used in cycles range delays
- Need to use IEEE 1800-2009 switch or later in tool

`myseq[*0:$]`	Zero or more repetitions of myseq	`myseq[*]`
`myseq[*1:$]`	One or more repetitions of myseq	`myseq[+]`
`A ##[0:$] B`	B in the same cycle as A or any future cycle after A	`A ##[*] B`
`A ##[1:$] B`	B in cycle following A or any future cycle after A	`A ##[+] B`
1800-2005, 1800-2009 1800-2012, or 1800-2017		Only 1800-2009 1800-2012, or 1800-2017

© Cadence Design Systems, Inc. All rights reserved.

This shorthand notation is exactly the same as IEEE 1850, Property Specification Language (PSL).

Hence, this further reduces the differences in semantics and the learning curve between SVA and PSL.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Property Abstraction

Properties should *not* duplicate the Verilog implementation

Properties should be created *from* the specification *using* the implementation

- Not vice-versa

Properties should be defined alongside stimulus in a test plan

- What are you testing?
- How are you going to test it?
  - Stimulus
- How do you check that you have tested it?
  - Coverage
- How are you going to verify results?
  - Assertions

Avoid Copying RTL Verilog

```
...
if (ADD)
  C <= A + B;  // Add A & B

property CHECK_ADD;
  @(posedge CLK)
   ADD |=> (C ==(A+B));
endproperty

A1 : assert property (CHECK_ADD);
```

81 © Cadence Design Systems, Inc. All rights reserved.

cadence

Blindly duplicating RTL code in SystemVerilog Assertions can mean:

- Errors in the RTL are duplicated in SystemVerilog Assertions.
- Assertions are written at such a low level they become ineffective. (As in the example above – the add is not going to fail, so why check it?)

Indeed, who would write the comment //Add A & B? No one because its too obvious.

Ideally assertions should be defined as part of a formal test plan.

Note that the snippet code is not accurately translating RTL to SVA. The correct SVA is: ADD |=> (C == ($past(A)+$past(B)));

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Incorrectly Specifying Assertions

Properties can be redundant

```
property OC1; (A != B); endproperty
property OC2; (B != C); endproperty
property OC3; (C != A); endproperty
```

If OC1 and OC2 fail then so will OC3

This doesn't mean OC3 couldn't tell us something we wouldn't otherwise know if OC1 and OC2 pass

Properties can potentially conflict

```
property CON1; !A; endproperty
property CON2; !B; endproperty
property CON3; (A || B); endproperty
```

- A simulator or formal tool would not warn if these were assertions
- A formal tool would warn if these were assumptions

82 © Cadence Design Systems, Inc. All rights reserved.

cadence

Incorrectly specifying can lead to duplicated or redundant assertions that are explicitly or implicitly covered by previous assertions.

Incorrectly specifying can also lead to conflicting assertions, where assertions will always fail.

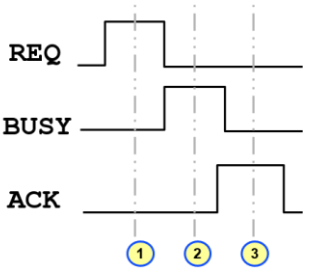
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

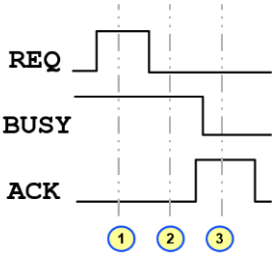
Under-Specifying Assertions

Write this sequence to detect this waveform

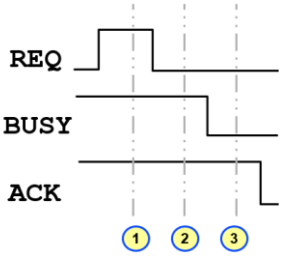
```
(REQ ##1 BUSY ##1 ACK)
```



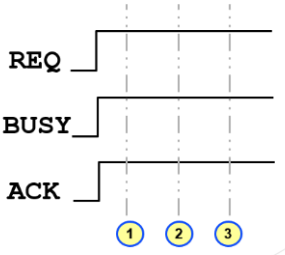
However this sequence also satisfies this…



and this ...



and even this ..



83 © Cadence Design Systems, Inc. All rights reserved.

cadence

The above sequence checks `REQ` on the first cycle, `BUSY` on the next and `ACK` on the last. `REQ` is not checked after the first cycle. Likewise, `BUSY` and `ACK` are not checked outside of their respective cycles. The sequence may look straight-forward, but there are a large number of waveforms that satisfy the sequence. If all these are acceptable, then there is no problem. If, however, some of these sequences are unacceptable, then you must define the sequence more strictly.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Summary

Sequences allow complex, multi-cycle properties to be easily expressed

- Boolean expressions separated by cycle delay `##N`

SVA has several operators and features to aid definition of multi-cycle and repeated sequences

- Ranged and un-ranged repetition operators
  - Consecutive repetition `[*N]`
    - Applies to sequences and individual boolean expressions
  - Nonconsecutive repetition `[=N]`
    - Applies to individual boolean expressions
  - Go-to repetition `[->N]`
    - Applies to individual boolean expressions

Ranges can be defined for cycle delay and repetition operators

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

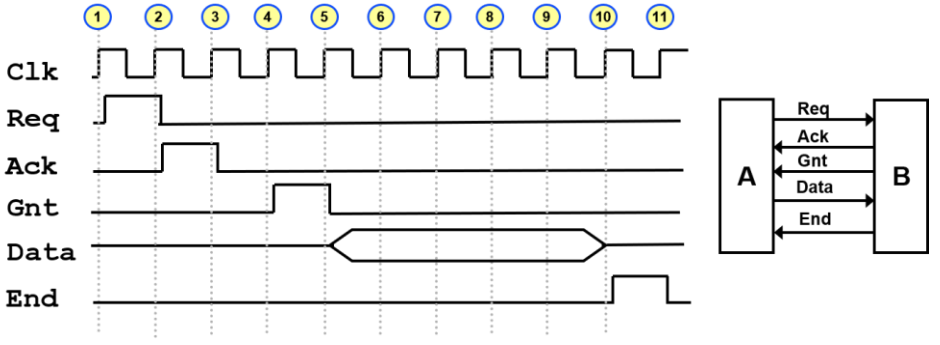
This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Quiz

After REQ in one cycle followed by ACK in the next what must then occur is:

- GNT must be low in the same cycle as ACK, and goes high between 1 and 2 cycles *after* ACK goes high
- END must remain low for 5 CLK cycles after GNT as DATA clocked in
- END then goes high

Write an assertion to check the behavior described above



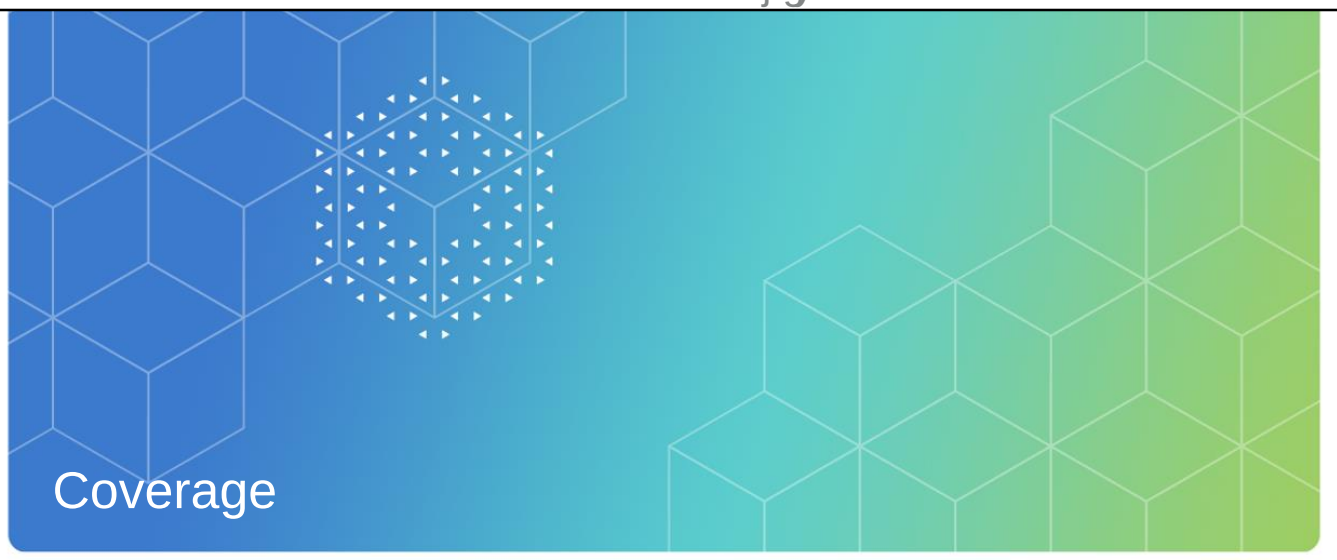
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module 5
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

Aim

- Describe the application of SVA to sequence-based coverage

Topics

- What is coverage?
- Structural and functional coverage
- Cover directive
- Simple example
- Using coverage to debug assertions

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Analyzing Effectiveness of Test Data

- How effective are test vectors?
- Basic measurement is code coverage, for example:
  - How many times each line of code executed by given test data
  - Unexecuted code => holes in test coverage => undetected bugs

```
always @(posedge clk or posedge rst)
    20    if (rst)
     1      op <= 8'h00;
    19    else
    19      case (opcode)
    19        ADD : op = a + b;
    *0*       SUB : op = a - b;
            ...
endcase
```

Statement Coverage

process never stimulated with `opcode = SUB`

© Cadence Design Systems, Inc. All rights reserved.

Code coverage identifies lines of code that are not executed for a given set of test vectors.

Untested code shows holes in test coverage. Untested code may contain bugs that have not yet been detected. In addition, untested code may represent redundant logic or un-testable code.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Functional Coverage Metrics (Review)

Structural metrics like code coverage are static

- Check every code branch exercised
- Check every value of an object applied
- No design knowledge required to define this check

Functional metrics use "dynamic" coverage

- Check sequence of values applied to an object
- Check sequence of events (transactions) has occurred
- Design knowledge and intent required to define this check

Functional coverage analyzes design use scenarios

- Complementary to code coverage

```
typedef enum
  {add, sub, divide} t_opcode;
```

Static value coverage

```
add      ☑
sub      ☑
divide   ☑
```

Dynamic sequence coverage

```
add;sub;divide       ☑
add;divide;sub       ☑
add;add;add;add      ☑
sub;sub;sub;divide   ☒
```

89 © Cadence Design Systems, Inc. All rights reserved.

cadence

Code coverage is static. It will check that every line of code or every opcode has been tested, but it will not check that sequences of opcodes have been tested. This is important as our design may have a problem where 2 `SUB`s followed by a *DIVIDE* causes an overflow error.

Code coverage is a basic measure, checking that every line of code has been exercised. Functional coverage, as the name suggests, checks if the functionality of the design has been exercised by the stimulus. You cannot exercise every combination and sequence of events in a design in simulation, but you can identify which critical combinations to exercise to provide confidence in the stimulus.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Defining Functional Coverage

Powerful language needed to specify coverage criteria

- Define legal or illegal sequences
- Trigger and terminate coverage
- Monitor sequences over time

SystemVerilog Assertions can do this!

```
typedef enum
  {add, sub, divide} t_opcode;
t_opcode opcode;
```

```
sequence opsub; opcode == sub; endsequence
sequence opdiv; opcode == divide; endsequence
...
sequence opseq1;
  opsub ##1 opsub ##1 opsub ##1 opdiv;
endsequence
```

Sequence coverage

add;sub;divide	☑
add;divide;sub	☑
add;add;add;add	☑
sub;sub;sub;divide	☒

 © Cadence Design Systems, Inc. All rights reserved.

Specifying the sequences to be checked during functional coverage can be difficult. Commercial functional coverage tools use proprietary languages to handle this. But SVA has the ability to specify complex multicycle functional sequences in a concise, powerful language.

SVA can be applied to functional coverage definitions as well as dynamic assertion checking.

Note that SystemVerilog has another form of coverage based on covergroups. This is a data-based coverage approach and is covered in the Cadence® SystemVerilog Verification module.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Cover Directive

The `assert` directive is not suitable for coverage

Use the `cover` directive

- Counts number of times a sequence occurs

Cover can be used with either sequences or properties

- Declaring sequences for use in both assertions and coverage is very efficient
- Coverage must be clocked!

```
sequence opsub; opcode == sub; endsequence
sequence opdiv; opcode == divide; endsequence
sequence opadd; opcode == add; endsequence

sequence opseq1;
  opsub ##1 opsub ##1 opsub ##1 opdiv;
endsequence

C1 : cover property ( @(negedge clk) opsub ##1 opdiv ##1 opadd);
C2 : cover property ( @(negedge clk) opseq1);
```

Sequences almost always defined without a clock

opseq1 clock inherited from cover C2

© Cadence Design Systems, Inc. All rights reserved.

The assert directive is not suitable for coverage. If you want to check the coverage of 3 *SUBs* followed by a *DIVIDE*, attaching an assert directive to the sequence only reports errors when the sequence does not complete.

SVA has the *cover* directive. Cover can be applied to sequences (as above) or properties.

Named or explicit sequences, or any combination of these, can be used in a cover statement.

The sequence cover records how many times the sequence started and how many times it was successfully completed.

Property cover records how many times the property started, failed, and succeeded. If a property is asserted, a good simulator should record property coverage automatically without the need for separate coverage.

Note: Be careful on clocking. Sequences are usually unclocked as the clock is defined in the property where the sequence is used. In the absence of a default clock, an explicit clock must be defined in the cover statement.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Cover in Simulation Versus Formal

In simulation `cover property (my_property)` means:

- From what was observed in a specific simulation count how many times `my_property` was observed to complete

In formal `cover property (my_property)` means:

- Show me an example of `my_property` completing
- This is normally the shortest possible trace

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Cover Statement

Statements can be attached to cover directives for simulation

- Executed when cover completes
- Multiple completes in same cycle gives multiple executions of statement

```
sequence TRANSFER;
  (GntA or GntB) ##1 (Busy throughout Reset[->1]);
endsequence

C3 : cover property (
     (@(posedge CLK) TRANSFER)
     $display("Test 34 found: reset during transfer");
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Named cover statements, like assertion properties, are much easier to manage and debug.

An appropriate default clock should be defined if an explicit clock is not included.

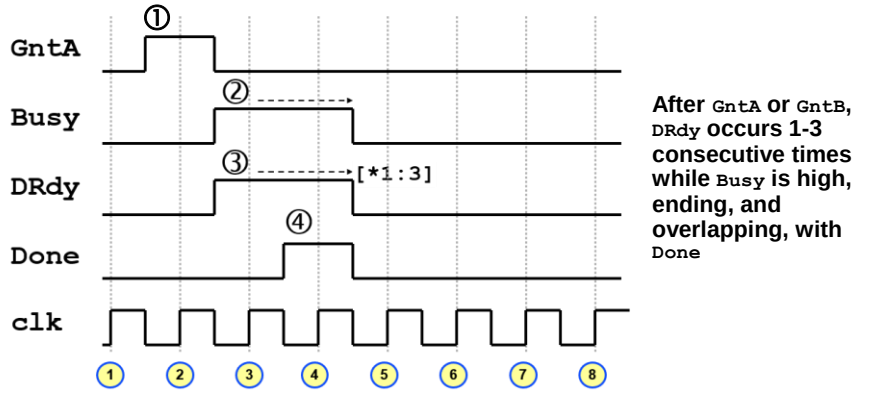
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Bus Protocol Example 1

Test when a transfer includes between 1 and 3 successive data ready cycles

```
C4 : cover property
  ( (GntA || GntB) ##1 (Busy throughout DRdy[*1:3]) ##0 Done );
```



After `GntA` or `GntB`, `DRdy` occurs 1-3 consecutive times while `Busy` is high, ending, and overlapping, with `Done`

94 © Cadence Design Systems, Inc. All rights reserved.

This cover directive monitors for transfers involving 1, 2, or 3 successive “data ready” cycles.

An appropriate default clock should be defined as an explicit clock is not included.

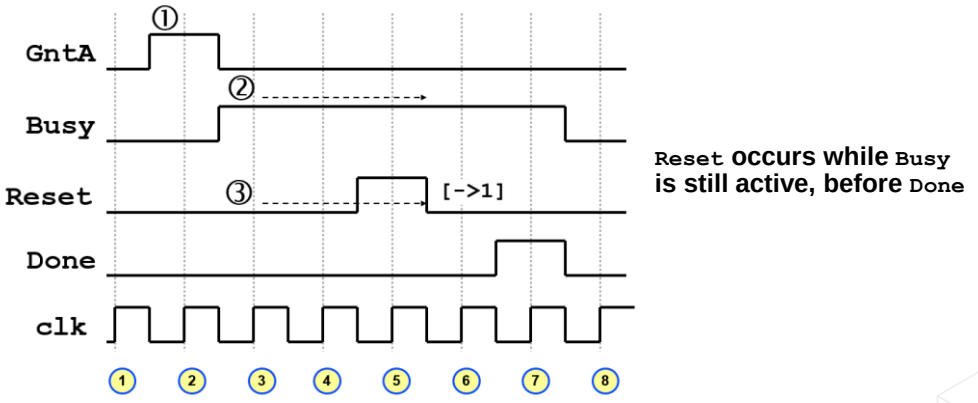
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Bus Protocol Example 2

Test the case in which a transfer is interrupted by a bus reset

```
C6 : cover property
 ( (GntA || GntB) ##1 (Busy throughout Reset[->1]) );
```



Reset occurs while Busy is still active, before Done

95 © Cadence Design Systems, Inc. All rights reserved.

cadence

Another coverage scenario

Here you are using the `[->]` operator to match an occurrence of `Reset` that happens while `Busy` is high.

Remember `Reset[->]` defines a sequence that finds and ends with the first occurrence of `Reset`.

So `(Busy[*] intersect Reset{->])` defines a sequence where `Busy` is high up to and including the first occurrence of `Reset`, when the sequence terminates.

An appropriate default clock should be defined, because an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Using Coverage to Debug Assertions

Managing and debugging assertions with long or alternate enabling sequences can be difficult

Need feedback on:

- Start of enabling sequence found
- Enabling sequence does not complete
- Which parts of alternate enabling sequence are found
- Assertion passes
- Assertion fails

Tool support for this varies...

Coverage can help

```
property P1;
  A ##1 B ##1 (C or D) ##1 E |=> END);
endproperty
C0: cover property
  (A);
C1: cover property
  (A ##1 B);
C2: cover property
  (A ##1 B ##1 C);
C3: cover property
  (A ##1 B ##1 D);
C4: cover property
  (A ##1 B ##1 C ##1 E);
C5: cover property
  (A ##1 B ##1 D ##1 E);
```


© Cadence Design Systems, Inc. All rights reserved.

Coverage can also help in debugging long-sequence assertions.

In the first example, a long enabling sequence also contains alternative sequences. Systematic coverage can check each stage of the enabling sequence if the assertion is not becoming active in simulation, or can check that each alternative sequence does occur.

An appropriate default clock should be defined as an explicit clock is not included.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Cover Groups

General SystemVerilog construct not used solely in SVA

- Defined with keywords `covergroup` and `endgroup`

Widely used in simulation testbenches but can also be used in formal

Property Table

Filter on name

	Type	Name	
✔	Cover	cover_item_covergroup_cg_inst_cg1	coverpoint_packet_type_bin_auto[TYPE1]
✔	Cover	cover_item_covergroup_cg_inst_cg1	coverpoint_packet_type_bin_auto[TYPE0]
✔	Cover	cover_item_covergroup_cg_inst_cg1	coverpoint_has_crc_bin_auto[1]
✔	Cover	cover_item_covergroup_cg_inst_cg1	coverpoint_has_crc_bin_auto[0]

```
covergroup cg @(posedge clk);
  packet_type: coverpoint my_packet.packet_type;
  has_crc: coverpoint my_packet.has_crc;
  cross_type_crc: cross packet_type, has_crc;
endgroup

cg cg_inst_cg1 = new;
```

JasperGold® automatically creates one property (bin) for each possible value of the signal

97 © Cadence Design Systems, Inc. All rights reserved.

cadence

Covergroups are supported in JasperGold

- Language support limitations are documented in the Command Reference Manual

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

What Does Cover Property Mean?

In **simulation** the answer seems so intuitive one never stops to consider it

Count how many times I observe A followed by B

```
C10 : cover property ( A ##1 B);
```

Count how many times I observe A followed by B

```
C11 : cover property ( A |=> B);
```

Do they really mean the same thing?

Until the 2012 LRM the answer was ambiguous
To most simulators and formal tools, the answer was YES

© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What Does Cover Sequence Mean?

Cover sequence was introduced in the 2009 LRM

The argument can only be a sequence, i.e., no implication allowed

- Indeed, just as PSL does

Count how many times I observe A followed by B

```
C10 : cover sequence ( A ##1 B);
```

Illegal syntax in 2009 and later

```
C11 : cover sequence ( A |=> B);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What Changed In the 2012/2017 LRM?

The Formal Syntax of LRM 2012 makes it explicit what this means

```
C12 : cover property ( A |=> B);
```

Show/count a trace of the property “passing/completing”

This means either:

1. A followed by B
2. A not occurring (remembering what an implication means)

However, 2. is not very useful

Recommendation: If what you want to cover is a sequence `(A ##1 B)` then use:

```
C13 : cover sequence ( A ##1 B);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Backward Compatibility Issue with `cover` Verification Directive

Prior to the 1800-2009 LRM the only way of writing a cover directive was like this:

```
C_W_IMP : cover property ( A |=> B);
C_SEQ   : cover property ( A ##1 B);
```

Unfortunately, many people wrote cover directives with implication in them, i.e., like `C_W_IMP`

- …and they kept on writing them like that

This leads to undesirable consequences from LRM 1800-2012 onwards, as we have seen

This begs the question, “Why cause backward compatibility problems?”

The answer is in order that one can write an infinite length cover which can do things like:

- “Show (cover) that it is always *possible* for signal B to go high in the future”
- This is not the same as saying “B must always go high in the future” (which would be an assertion)
- For more information, see the next slide in this module or Google search for:
  - Can I Cover Infinity With SVA Properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Can I Cover Infinity in SVA?

Because it allows one to specify something really useful in formal

```
property CNT_ALWAYS_INCS_OR_WRAPS_P;
 ( 1[+] |-> (( CNT_VAL == ($past(CNT_VAL) + 1)) || (CNT_VAL == 0)) );
endproperty

ast_INCS_OR_WRAPS : assert property  (CNT_ALWAYS_INCS_OR_WRAPS_P);
cov_INCS_OR_WRAPS : cover  property  (CNT_ALWAYS_INCS_OR_WRAPS_P);
```

The assertion will fail any cycle the count doesn't increment or reset

- For example, perfectly valid behavior like the counter not enabled on this particular cycle

The cover tells me that it is **ALWAYS POSSIBLE** to keep counting, unless it is reset, without it being a requirement

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sure you would need to check the counter value being zero is expected for various scenarios as well.

This will only work as described with the Tcl command:

elaborate -weak_cover_property

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Summary

Assertions only catch bugs that occur during verification

- Conditional assertions must be enabled before they can fail

Coverage Monitors ensure that verification is thorough

- Monitors check that all interesting behavior is exercised
- Functional coverage is more effective than code coverage

SVA includes both

- Assert Directives (assertions, i.e. checks)
- Cover Directives (coverage monitors)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions are half the battle – they catch bugs that occur

Stimulus is the other half of the battle – the right stimulus is necessary to trigger bugs

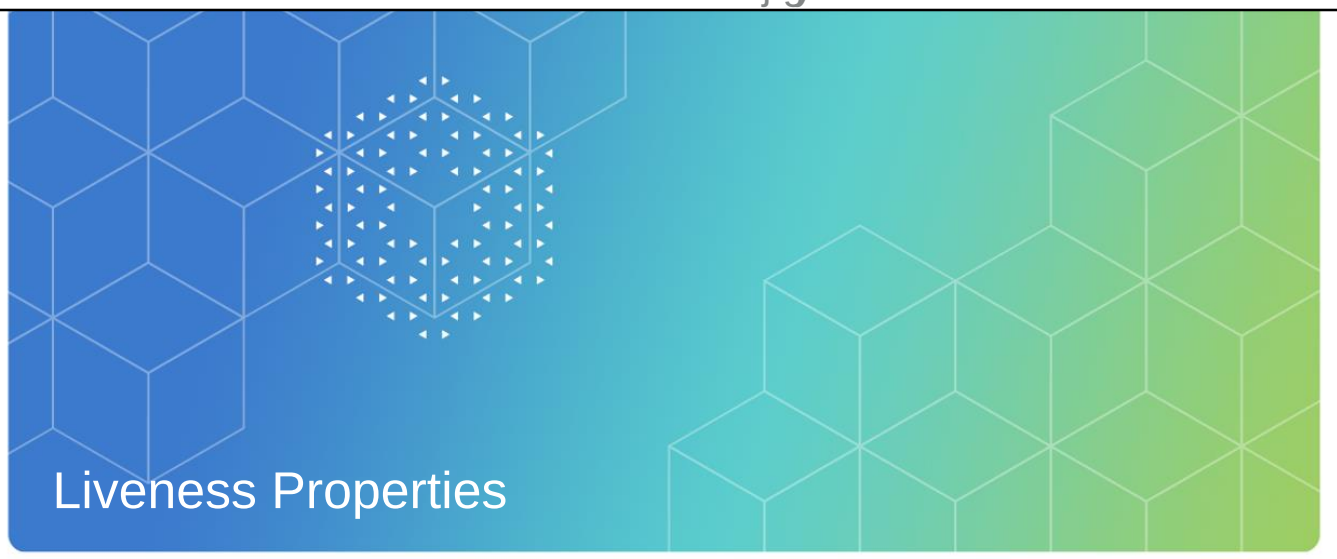
Coverage is a measure of whether we've stimulated a design thoroughly, to trigger all bugs

Coverage Monitors observe the design during simulation and report what behavior is exercised

The "cover" directive creates a Coverage Monitor, using the same behavioral description features as used for assertions

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence



Module 6
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Module Objectives

After this module, you will

- Write strong and weak properties recognizing when its appropriate to use one or the other
- Identify whether one obtains strong or weak properties depending upon LRM version and default strength
- Define a strong liveness property independent of LRM version
- Define a fairness constraint
- Identify in the LRM strong operators from weak operators

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Strong and Weak Properties

1800-2009 introduced weak and strong properties

Strong and weak relates to how failures of properties can be shown

Strong creates Liveness properties

Weak creates Safety properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Liveness Compared with Safety Properties

Liveness property informal definition:

"A property where `at least` one way in which it could fail is demonstrated by an infinite waveform"

Or to put it another way:

"Claims that something `good` happens, eventually"

Has no meaning in simulation

- Can't simulate for an infinite number of cycles

In contrast, `safety` properties are bound in time

- Failure can only be demonstrated by a finite waveform
  - Hence, all failures can be shown in simulation
- In other words, claims nothing `bad` happens

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Understanding Liveness Properties

`SAFETY1` holds if and only if:

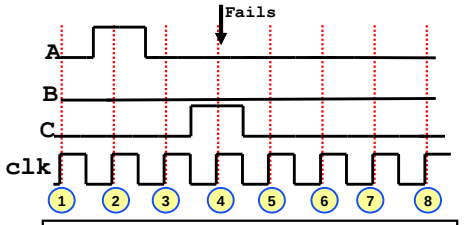
- `A` can never occur
- `A` occurs and then `B` occurs before `C`
- `A` occurs but `B` or `C` never occur
  - Makes no requirement for `B` or `C` to occur

```
SAFETY1:assert property (
    @(posedge clk)
      A |-> weak(!C[*] ##0 B) );
```

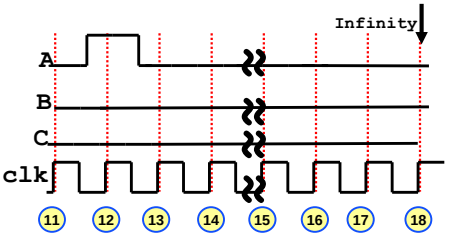
`LIVENESS1` holds if and only if:

- `A` can never occur
- `A` occurs and then `B` occurs before `C`
  - `B` must occur or property fails

```
LIVENESS1 :assert property (
    @(posedge clk)
      A |-> strong(!C[*] ##0 B) );
```



SAFETY1 and LIVENESS1 fail



SAFETY1 pass – LIVENESS1 fail

108 © Cadence Design Systems, Inc. All rights reserved.

cadence

This property example is known as mixed liveness.

It has both infinite length and finite length counter examples.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Default Strength

In 2005 LRM the concept of weak does not exist

From 2009 LRM onwards, if strong or weak are omitted the default strength depends upon usage

The default is weak for these verification directives

- `assume`
- `assert`

The default is strong for these verification directives

- `cover`
- `restrict`

Properties shown are equivalent in 1800-2009

```
SAFETY1 :assert property(
  @(posedge clk)
     A |-> (!C[*] ##0 B) );
```

=

```
LIVENESS1 :assert property (
 @(posedge clk)
    A |-> weak(!C[*] ##0 B) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Standard Incompatibilities

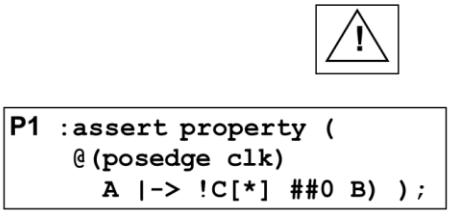
Default strength means different behavior in 2005 and 2009

In IEEE 1800-2005 the property fails

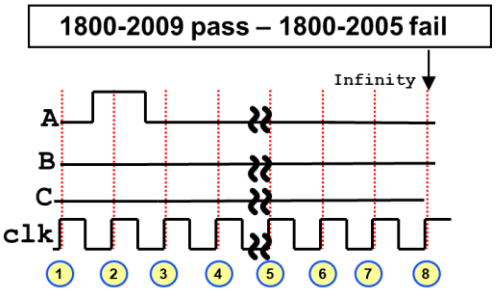
- IEEE 1800-2005 only had strong semantics

In IEEE 1800-2009 the property passes

- Default strength for assertion is weak



```
P1 :assert property (
    @(posedge clk)
      A |-> !C[*] ##0 B) );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Linear Temporal Logic (LTL) Operators

There were many new operators added in IEEE-2009

Most do exactly the same as their PSL namesake

Cannot define anything you couldn't already do with sequences

- Just makes it easier and more intuitive (and in many cases that is very questionable)

There are strong and weak forms

Strong forms prefixed with `s_`

Strong		Weak	
`s_until`	`s_eventually`	`until`	`eventually`
`s_until_with`	`s_implies`	`until_with`	`implies`
`s_always`	`s_nexttime`	`always`	`nexttime`

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

`eventually` Versus `s_eventually`

These keywords only exist in IEEE-2009 or later

`eventually` **must** have a bounded range, i.e., can't use `$` as upper bound

`s_eventually` may or may not have a range

- The range may or may not be bounded

In virtually every case, it is not of pragmatic use for any form of `eventually` to have a bound

Hence, `s_eventually` with no bound is used almost all of the time

```
W_EVEN : assert property (
  REQ |-> eventually[1:3] GNT);
```
≡
```
P1 :assert property (
  REQ |-> weak(##[1:3] GNT) );
```

```
S_EVEN_BND : assert property (
 REQ |-> s_eventually[1:3] GNT);
```
≡
```
P2 :assert property (
 REQ |-> strong(##[1:3] GNT) );
```

```
S_EVEN_UBND : assert property (
 REQ |-> s_eventually GNT);
```
≡
```
P2 :assert property (
 REQ |-> strong(##[0:$] GNT) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

LTL Operator Example

Assumption requirement at every `posedge clk`:

- `STALL` input to FSM must be false at some point in the future

You are stating that you do not want to be shown a property fail by formal tool holding STALL true forever

This is known as a Fairness constraint

All Properties shown are equivalent

- Only if correct standard release is followed

1800-2005

```
P2005 :assume property (
    @(posedge clk)
        1'b1[0:$] ##1 !STALL );
```

≡

1800-2009

```
P2009_A :assume property (
    @(posedge clk)
        s_eventually !STALL) );
```

≡

```
P2009_B :assume property (
    @(posedge clk)
        strong( 1'b1[0:$] ##1 !STALL ));
```

1800-2009

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Summary

- The 2009 standard changes behavior of some liveness assertions
- Essential that:
  - You understand the difference
  - You know which version of the LRM that you are compiling with
- Using `s_eventually` is the most intuitive way to express the desired behavior unambiguously and independent of the LRM version

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Property Reuse

Module 7
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

In this module, you

- Explore some of the language features and methodologies to facilitate reuse of properties
- Explore parameterized sequences and properties binding

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Sequence Arguments

Declare common sequences using formal arguments

Map actual to formal arguments at instantiation

- Tools check expanded sequence is legal

Can be instantiated inside property or another sequence

```
sequence N_CYCLE_RESP(n, p);
  (!p[*1:n-1] ##1 p);
endsequence
```

instantiation

```
( $rose(req) ##1 N_CYCLE_RESP(10,ack) ##1 done )
```

result after text expansion

```
( $rose(req) ##1 !ack[*1:9] ##1 ack ##1 done )
```

117 © Cadence Design Systems, Inc. All rights reserved.



A common sequence that applies to different signals can be defined as a named sequence with arguments. Different objects can then be associated with the sequence arguments when used. Integer values can be passed as arguments, as in the example above, to vary the number of cycles checked by the sequence.

An actual argument can replace an:

- Identifier
- Expression
- Event control expression

An actual argument can be $ (infinite upper range for repetition), such as:

```
N_CYCLE_RESP(ack, $)
```

When using parameters, you do not need to pass everything that you want to use in the sequence definition as a parameter to the sequence.

You still have visibility of all HDL signals in the same scope as the sequence is defined without them being explicitly passed as parameters.

However, you must be aware that reusability of a parameterized sequence is limited if you define a sequence using items that are not passed as parameters.

This is because you would be relying on local signals with specific names existing in all places where you want to reuse the sequence.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Property Arguments

Note the signal CLK is not passed explicitly as an argument. This relies upon what is known as “side effects”. This is BAD practice. It just works as long as there is a signal named CLK in the local scope. One should pass everything as an argument for reuse.

parameterized property

```
property ResultAfterN (START, N, RESULT, STOP);
  @(posedge CLK) disable iff (STOP)
  START |=> ##N RESULT;
endproperty
```

instantiation

```
A1 : assert property (ResultAfterN(WRITE_REQ, 3, ACK, CANCEL));
```

result after text expansion

```
A1 : assert property (
  @(posedge CLK) disable iff (CANCEL)
           WRITE_REQ |=> ##3 ACK);
```

Recommended for you to research in the LRM:

- Properties can have more kinds of arguments than sequences, e.g., property arguments can be events or other properties
- Sequence and property arguments can be defined as being typed and having a direction
- Google Search for “SVA Local Variables Practical Examples” video from 6:00 mins onwards

118 © Cadence Design Systems, Inc. All rights reserved.

A common property that applies to different signals can be defined as a named property with arguments. Different objects can then be associated with the property arguments when instantiated. Integer values can be passed as arguments, as in the example above, to vary the number of cycles checked by the sequence.

An actual argument can replace an:

- Identifier
- Expression
- Event control expression

An actual argument can be $ (infinite upper range for repetition), such as:

When using parameters, you do not need to pass everything that you want to use in the sequence definition as a parameter to the property.

You still have visibility of all HDL signals within the same scope, because the property is defined without them being explicitly passed as parameters.

However, you must be aware that reusability of a parameterised property is limited if you define a property using items that are not passed as parameters.

This is because you would be relying upon local signals with specific names existing in all places where you want to reuse the property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vijay Praveen of Cadence

Generating Assertions

`generate` can be used to create multiple or conditional assertions

- Verilog2001 statement

Useful for properties with arguments

- Here `for generate` creates an assertion for each bit of a vector

Conditional, case and nested generates can also be used

Remember `for generate` requires:

- Loop variable declared as a `genvar`
- A `begin` label to uniquely identify each generated assert
- Advised to label each generate block

```
reg [7:0] REQ, ACK;

property VRES (N);
  @(posedge CLK)
  $rose(REQ[N]) |=> ACK[N];
endproperty

genvar i;
generate
  for (i=0; i<8; i=i+1)
  begin :genvres
    A1 : assert property (VRES(i));
  end
endgenerate
```

```
….genvres[0].A1
….genvres[1].A1
…
….genvres[7].A1
```

Each generated assertion with have a unique hierarchical name

119 © Cadence Design Systems, Inc. All rights reserved.

cadence

Generates create multiple instances of:

- Modules, primitives, variables, nets, tasks, functions, continuous assignments, initial blocks and always blocks, assertions and properties.

New key words

- `generate`, `genvar`, `endgenerate`

Three forms:

- For generate
- If/else generate
- Case generate

For generates must be labeled.

- All instances and declarations have unique names based on generate label.

For generate loop variable must be declared as with genvar.

Tools usually automatically generate labels for unlabelled generate blocks. It's not guaranteed that it'll stay the same after modification of the SVA code.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Verification Components (Checkers)

Verification components (VCs):

- Are generic modules that encapsulate only verification code for reuse, e.g. properties and auxiliary code
- Provide an infrastructure for reusable properties for commonly used interfaces and components
- Just as with UVCs in a UVM testbench, instance and `bind` to where they are needed in the DUT

We could use the SystemVerilog `checker` construct instead of a normal SystemVerilog module

If the VC ports are all logic signals then we can literally text replace the word “module” with “checker”

```
checker fr_cnt_chkr (
        input logic  CLK,
        input logic  rst_n,
       input logic  CNT_ENB,
        input logic[2:0]  CNT_VAL
      );
default clocking DEFCLK @(posedge CLK1);
endclocking
  C1 : cover property (
     CNT_ENB && CNT_VAL == 3'b101);
  ast_cnt_ne_6 : assert property (
         CNT_VAL != 6 );
endchecker
```

```
module fr_cnt_chkr (
        input logic  CLK,
        input logic  rst_n,
       input logic  CNT_ENB,
        input logic[2:0]  CNT_VAL
      );
default clocking DEFCLK @(posedge CLK1);
endclocking
  C1 : cover property (
     CNT_ENB && CNT_VAL == 3'b101);
  ast_cnt_ne_6 : assert property (
         CNT_VAL != 6 );
endmodule
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Should I Use a Module or Checker for My Verification Components?

At first reading, SystemVerilog Checkers seem to have advantages over modules, for example:

- An EDA tool knows that a checker is there purely for verification purposes
- This allows, for example, checkers to be hidden in the design hierarchy diagram

Checkers can have more types of arguments than a normal SystemVerilog Module, for example:

- Can infer clock and reset from context in which they are used
- Can be instantiated in procedural code
- Can have properties, events, sequences, etc., in port list

However, SystemVerilog checker constructs are rarely used for some basic practical reasons.

- Checkers cannot have parameters
- The “extra” features complicate verification and in practice are of little pragmatic use

It is recommend to continue using SystemVerilog modules for your VCs for reuse

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Assertion Binding

`bind` statement can literally be placed anywhere

- Including on a line by itself in a separate file

Instantiates a VC in instance(s) of a hierarchical design module

Works the same both with modules or SystemVerilog `checkers`

Advantages

- No changes to design module
- Easier to modify and update

Can bind to all or specific instances of a design module

- To bind to every instantiation of `mone`

```
bind mone mone_prop...
```

- To bind only to instantiation `m1` of `mone`

```
bind mone:m1 mone_prop...
```

```
module mone_prop(input a,b,clk);
  property P1;
    @(posedge clk) $rose(a) |=> b;
  endproperty
  A1 : assert property (P1);
endmodule
```

Verification Component

```
module mone(input logic clk, req,
            output logic ack,
             ...);
  ...
endmodule
```

Design module

```
module top;
...
  // instantiate design
  mone m1 (clk, req1, ack1...);
  mone m2 (clk, req2, ack2...);

  bind mone mone_prop mp1(req,ack,clk);
                    ...
```

Signal names in design module scope

Testbench

Design module | Verification Component | Instantiation name | Design module variables

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Bind allows you to insert an instantiation of a VC (or indeed any module) into any design module in the hierarchy, by adding a bind construct to, typically, the topmost module in a design.

Hence, you can add modules into a design hierarchy from the testbench without having to edit the design modules.

Syntax for bind:

- `Bind`
- `mone` //design module name/instance
- `mone_prop` //VC
- `mp1` //instantiation of assertion module
- `req,ack,clk);` //map to variables in design module

You can bind a VC to a specific instantiation of a design module by using the instantiation name instead of the module name:

- `bind m1 mone_prop mp1(req,ack,clk);`

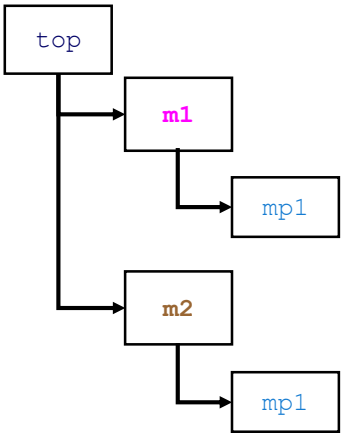
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

How Binding Works

Equivalent to an instantiation of the VC inside `top.m1` and `top.m2`

- Linked into design by way of hierarchical path names




```
module mone_prop(input a,b,clk);
  property P1;
    @(posedge clk) $rose(a) |=> b;
  endproperty
  A1 : assert property (P1);
endmodule
```

Verification Component

```
module mone(input logic clk, req,
            output logic ack,
             ...);
...
endmodule
```

Design module

```
module top;
...
  // instantiate design modules
  mone m1 (clk, req1, ack1...);
  mone m2 (clk, req2, ack2...);

  bind mone mone_prop mp1(req,ack,clk);
...
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Bind creates an implicit instantiation of a VC and links this to all instances, or a specific instance, of a design module via hierarchical variable names

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Applying Properties Conditionally

Use Verilog `ifdef` statement

You might want to conditionally include these items:

- Cases for Formal
- Abstract Properties for Formal
- Sanity Checks

You might want to conditionally exclude these items:

- EDA tools which don't support SystemVerilog Assertions
- When verifying at a higher level and these properties are already proven
- ..and so on

```
module mod1
  (input logic CLK,
   ...
   output logic EN1, EN2);

`define SVA_ENABLE

`ifdef SVA_ENABLE
  property PROP1;
    @(posedge CLK) (EN1 | EN2);
  endproperty

  ASSERT1: assert property (PROP1);
`endif
                    …
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

SystemVerilog allows concurrent assertions to be placed in any hierarchical block, including Module, Interface, Program block, Package, Clocking block or Compilation unit scope.

There are several clever techniques in formal for abstracting the proof of a property to something much easier for the tool to deal with.

These techniques would not be valid for simulation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What Properties Tell You

Properties specify some desired behavior

- Behavior of the DUT – Assertions
- Behavior of DUT's environment – Assumptions

These say nothing about whether you use Formal or Simulation

Hence, they should be reusable with either tool

This is mostly true, but you need to be aware of the exceptions

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

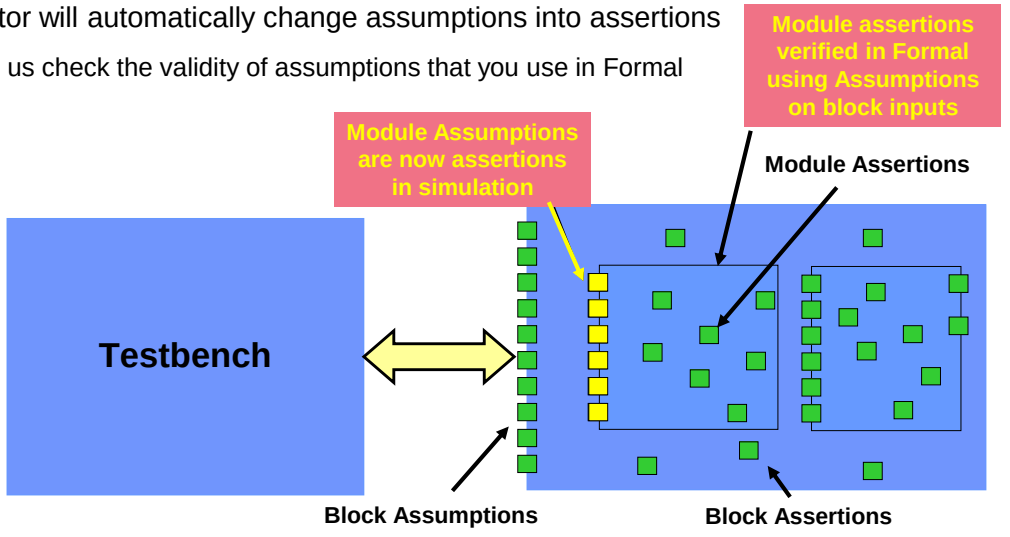
This document is for the sole use of Kuhn Vajay Praveen of Cadence

Simulation of a Block

Both assertions and assumptions are verified in simulation

Simulator will automatically change assumptions into assertions

- Lets us check the validity of assumptions that you use in Formal



126 © Cadence Design Systems, Inc. All rights reserved.

During simulation, the block and module assumptions are checked to insure that the assumptions used during Formal Analysis still hold during simulation.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Cases Only Applicable to Formal

You want to formally verify some `write` functionality of the FIFO

To simplify the formal verification task, add `pin constraints`

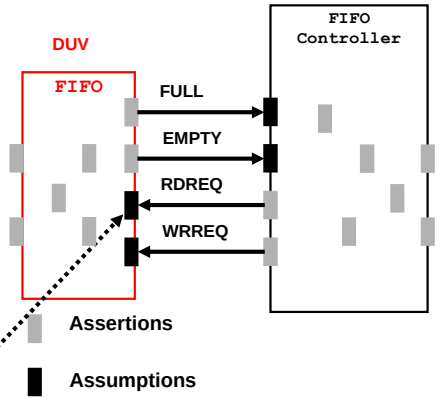
- For example, hold `RDREQ` at `'0'`
- Deliberate overconstraining to reduce state space
- This is a temporary measure to make verification more convenient

This is called a `case` in Formal Analysis

In simulation, this property will certainly fail

Keep `cases` separate from normal assumptions, so they can be easily disabled

```
ASM_NOREAD: assume property @(posedge CLK)(!RDREQ);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Nondeterministic Constants

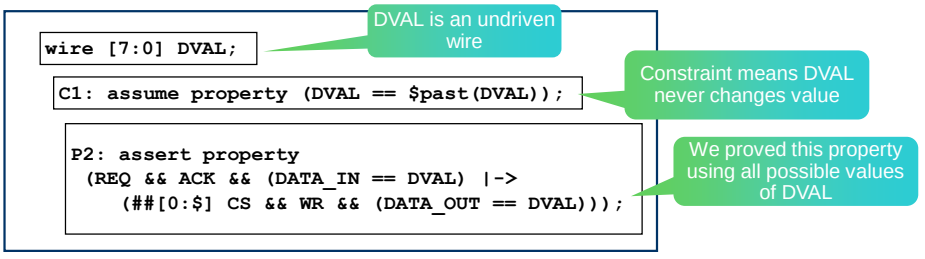
Conceptually formal tool chooses all possible values for DVAL

The value chosen remains the same during the proof

Nondeterministic constants have no meaning in simulation

In simulation DVAL value will be deterministic

- Initial value determined by [System]Verilog LRM



 © Cadence Design Systems, Inc. All rights reserved.

This will seem very strange to you if you have never used Formal before.

The Formal tool can use any value for the undriven wire DVAL which will result in the property failing.

However the constraint means that once a value has been chosen for DVAL, then that value never changes during verification.

So conceptually the tool considers all possible values of DVAL.

C1: clock definition is omitted to simplify example.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Formal Complexity Issues

SVA in Simulation is intuitive and generally WYSIWYG

- Simulator builds a state machine to check property

Formal has techniques with which simulation cannot cope with

- For example, FSM deadlock
- SVA has no constructs to deal with this but PSL does

Formal has to perform exhaustive proofs

- With simulation all you know is what you tested works
- Properties which are OK for simulation may vastly increase Formal Complexity
  - Hence Formal Run times

Especially true of assumptions

- Having state (repetition) on RHS of implication very costly for assumptions

```
property P1;
   ( @(posedge CLK)
          START |=> POP[->1] ##0 (TAG==2 or (BUSY[*1:$] ##1 (!BUSY && SKIP))));
endproperty
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Summary

You now have some powerful constructs for property reuse

- Parameterized sequences and properties
  - Allow use of common sequence definitions on different signals
- Binding
  - Easier association of assertions with existing models

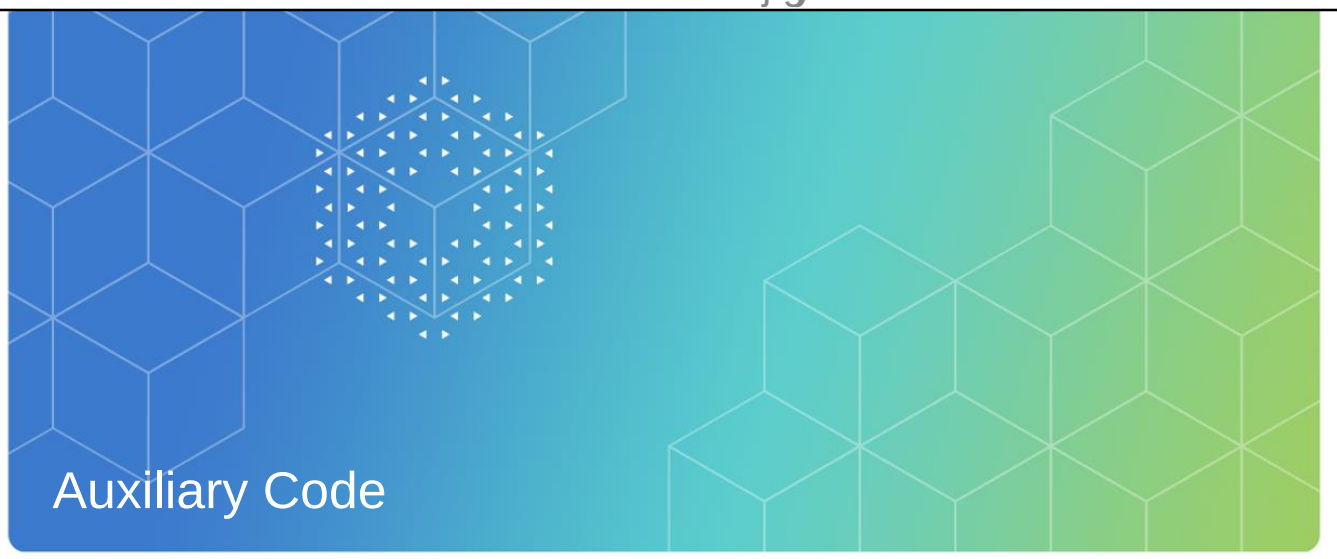
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence



Module 8
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

In this module you will learn how to:

- Create Auxiliary Code (HDL helper code) to assist when writing properties
- Recognize problems that cannot be solved by SVA alone

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Example: An Overlapping Behavior Requirement

Required request/grant interface behavior:

- If you have a REQ, then you must have a GNT within 2 to 8 cycles
- Requests can overlap
  - For example, you can have another request before the first grant received
- Each REQ must have a corresponding GNT
- No GNT without a REQ

This initially seems easy in SVA

Why won't it work as planned?

```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

You have a different situation, which looks easy to solve with SVA.

The cause of the problem is the overlapping nature of the REQ/GNT handshake behavior.

This makes it impossible to discern from handshake from the other, because you need to consider things in a global sense.

If you can guarantee that there are no overlapping transactions then this is easy.

You can even write an assertion to check that there were no overlapping transactions, for example:

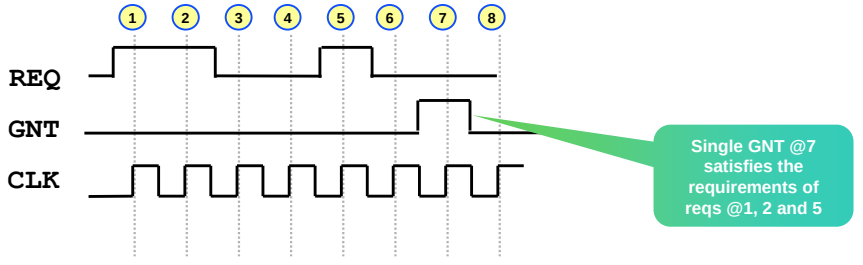
```
REQ |=> !REQ && !GNT[*0:$] ##1 GNT && !REQ
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Overlapping REQ/GNT: Bad Implementation

- Assertion languages have no way of describing what you want
- A single occurrence of GNT can satisfy multiple requests
- You need to model using Auxiliary HDL code (Aux code)
  - This will also be much more efficient for formal




```
A1: assert property(@(posedge clk) REQ |-> ##[2:8] GNT));
```


© Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

What Is Auxiliary Code?

Auxiliary code has 3 main purposes

- Make it easier to write properties
- Make the problem easier for the tool to evaluate
- Do things which assertion languages cannot

Location of Aux code

- In Verification Components(VC's) /Checkers
- Normal Verilog modules which have only inputs
  - VC's are merely a container for Aux code, assertions, assumptions, cover statements, i.e., only verification code
  - Connected to the DUV, even legacy code, using the SV `bind` statement
  - Just instance and connect them where you need them
    - Just like UVCs in a UVM testbench
- Embedded in RTL
  - Surrounded with synthesis pragmas or `` `ifdef ``
  - Not recommended

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

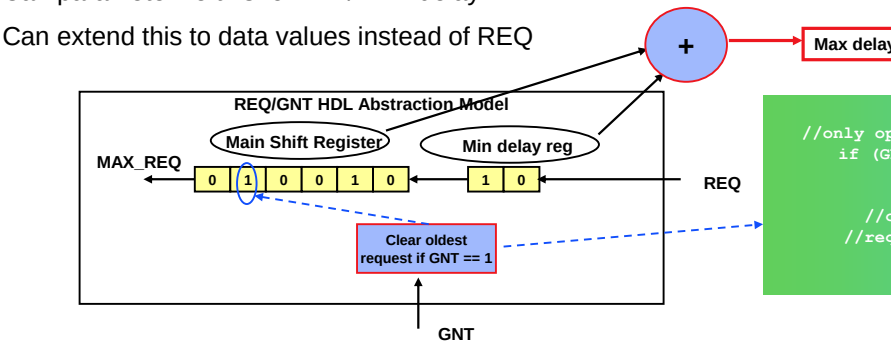
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

REQ/GNT Abstraction Model

Requirement of the Aux code model:

- If you have a request you must receive a GNT within 2-8 cycles
- 2 represents the minimum delay for the GNT response
- 8 represents the maximum delay for the GNT response
- Can parameterize this for MIN/MAX delay
- Can extend this to data values instead of REQ




```
for (i=7; i >= 2; i--)
//only operate on MAIN_SHIFT_REG elements
if (GNT && (SHIFT_REG[i] == 1'b1))
//remove oldest request
SHIFT_REG[i] <= 1'b0;
//only remove a maximum of one
//request from shift reg per clock
break;
end
```

AUX Code

```
A1: assert property ( MAX_REQ |-> GNT);
A2: assert property ( GNT |-> $countones(MAIN_SHIFT_REG) != 0);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Note that for this to work in the detailed implementation, SHIFT_REG[0] and the REQ signal would have to be assigned at the same clock, for example:

```
Always @(posedge CLK) begin
   REQ   <= NEXT_REQ;
   SHIFT_REG[0] <= NEXT_REQ;
end
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Overlapping REQ/ACK Handshake: Requirement

A new requirement now…

Every REQ must have an ACK sometime later

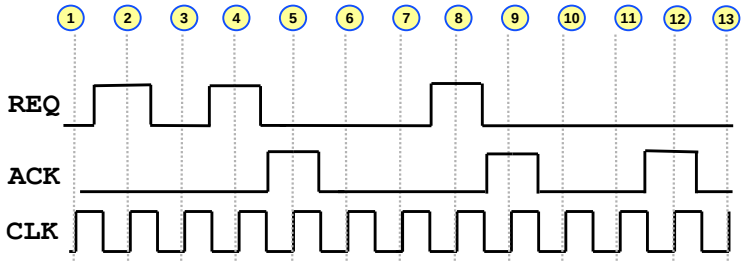
Maximum of 5 outstanding requests allowed

The ACK can occur any time

- There does not have to be an ACK before another REQ
- No limit on number of cycles from REQ to ACK

No ACK without REQ

How can you write this with SVA?



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

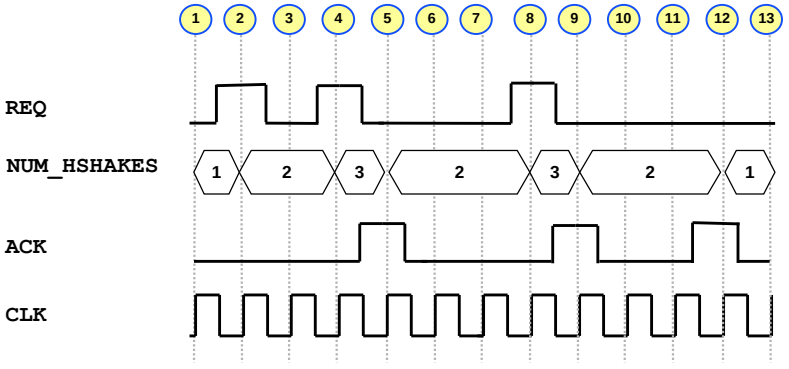
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Overlapping REQ/ACK Handshake Solution

No way to write this in just SVA

You need Aux code to do the *global counting*



Use the smallest counter size possible, not an integer (32 bits)

Start unsigned count at 1

```
reg[2:0]   NUM_HSHAKES;
always @(posedge CLK or posedge
                 RESET)
                   begin
                 if   (RESET
              NUM_HSHAKES
                   else
                   begin
               if (REQ & !ACK)
        NUM_HSHAKES<= NUM_HSHAKES+ 1;
             if   (ACK & !REQ)
        NUM_HSHAKES<= NUM_HSHAKES- 1;
                   end
                 end
```

AUX Code

```
LESS_THAN_6_REQS: assert property ( ;
@(posedge CLK)   ( NUM_HSHAKES !=7) );
        endproperty

NEVER_MORE_ACKS_THAN_REQS : assert property (
@(posedge CLK) (NUM_HSHAKES != 0) );

NEVER_ACK_WITHOUT_PREV_REQ : assert property (
        @(posedge CLK)
    (REQ && ACK |=> NUM_HSHAKES != 1) );
```

Q1) What would be good cover statements to write?

```
C1: cover property (
      @(posedge CLK)
(NUM_HSHAKES ==6) ##[+] (NUM_HSHAKES ==1) ) ;
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Praveen of Cadence

SVA Alone Cannot Solve All Problems

One-shot properties – Properties which we only want to check once

- Unlike PSL, SVA does not have the semantics for this

Requirement:

- From the beginning of verification we require A to be high up to and including the cycle B occurs
- After B occurs we don't care about what either A or B is doing

```
ONESHOT_A_UNTIL_B: assert property (
  @(posedge CLK) ( !B_SEEN |-> A) );
```

```
reg B_SEEN;
always @(posedge CLK or negedge
                RSTn)
                 begin
              if  (!RSTn)
               B_SEEN <= 0;
              else if (B)
             B_SEEN <= 1'b1;
                 end
```

AUX Code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Aux Code Versus SVA

A common question for beginners to formal is “What is the point of SVA if I have to write more RTL?”

- Aux code will just be more work

SVA is a very concise way to describe complex behavior

That is a source of strength and weakness, a double edged sword

- Even with as few as 6 characters one can create an SVA property that takes a long time to understand

This module was about efficient SVA coding and Aux code is an essential part of that

It is a naive view:

- Try and describe everything that needs checking in a single assertion
- SVA encompasses all your needs
- Trying to perform timing checks, e.g., pulse width, with SVA

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Summary

Writing complex SVA is counter-productive

- Its hard for the tool to evaluate
- Its hard for other engineers to understand

Almost every problem will require Aux code

- Allows the problem to be abstracted
- Does not create overlapping copies of itself
- More verbose but easy to understand
- Typically checks more behavior than the intended complex SVA
- Easier to write covers and even allows the tool to automatically infer covers

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

SVA Course Part 1 (Simulation and Formal Users) Conclusion

Module 9
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

What Have We Learned?

Motivation and methodology of using ABV

How to define sequential and boolean properties

Language features and methodologies for property reuse

- Including from formal to simulation and vice-versa

Motivation and methodology of defining coverage with SVA

How to correctly define Liveness and Safety properties

Motivation and methodology of using Auxiliary (HDL helper) code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

SVA Issues

SVA syntax looks easy but the application is not

- The syntax is “sugared,” it is meant to look easy
- The concepts are hard

Not all common behaviors can be checked with SVA alone

Aux code often required to:

- Make properties simpler to express
- Do things which assertion languages cannot express
- Reduce “complexity” for Formal

Special Techniques can be applied when using Formal

- These are not compatible with Simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

SVA Part 1 (Formal and Simulation Users) Summary

At this stage if you are a formal user then you have all you need to start using formal

- There is no need to continue to Part 2 of this course for formal only users

If you are a Simulation user then you also have what you need to write meaningful properties and start using simulation

However, Simulation users may benefit from continuing to Part 2 of the course and learning some more advanced SVA constructs

These may be useful in simulation but are rarely useful in Formal

- Indeed they are probably counter-productive in Formal

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Where Next?

SystemVerilog Language Reference Manual

- `www.ieee.org`
- `IEEE 1800-2012`

Textbooks

- Formal Verification: An Essential Toolkit for Modern VLSI Design
  - Erik Seligman
- SystemVerilog Assertions Handbook
  - Cohen *et al*

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

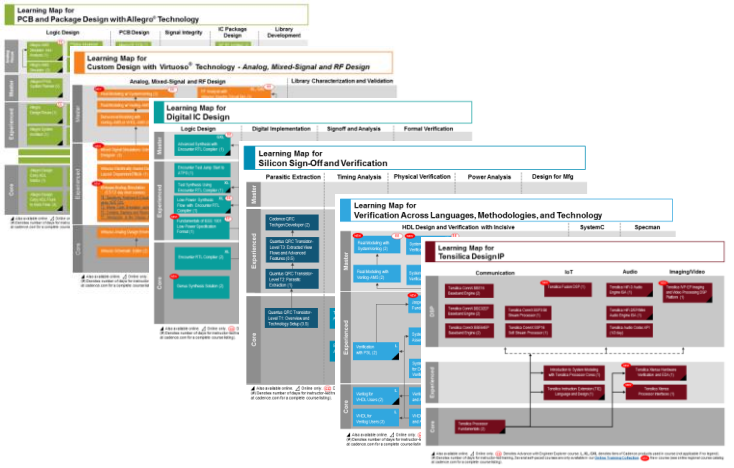
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click here to see all our courses in each technology area and the recommended order in which to take them.



 © Cadence Design Systems, Inc. All rights reserved.

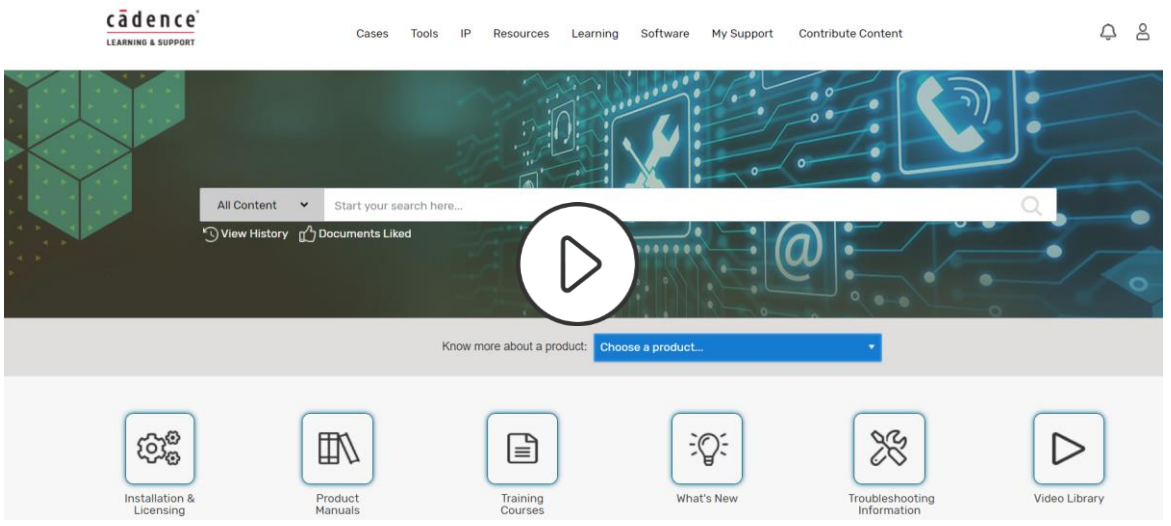
Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Cadence Learning and Support



Customer Support now includes over 2000 product/language/methodology videos ("Training Bytes")!

148 © Cadence Design Systems, Inc. All rights reserved.

Click the play button in the figure on this slide to view the demo of Cadence Learning and Support.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion and a Digital Badge

Thank you!

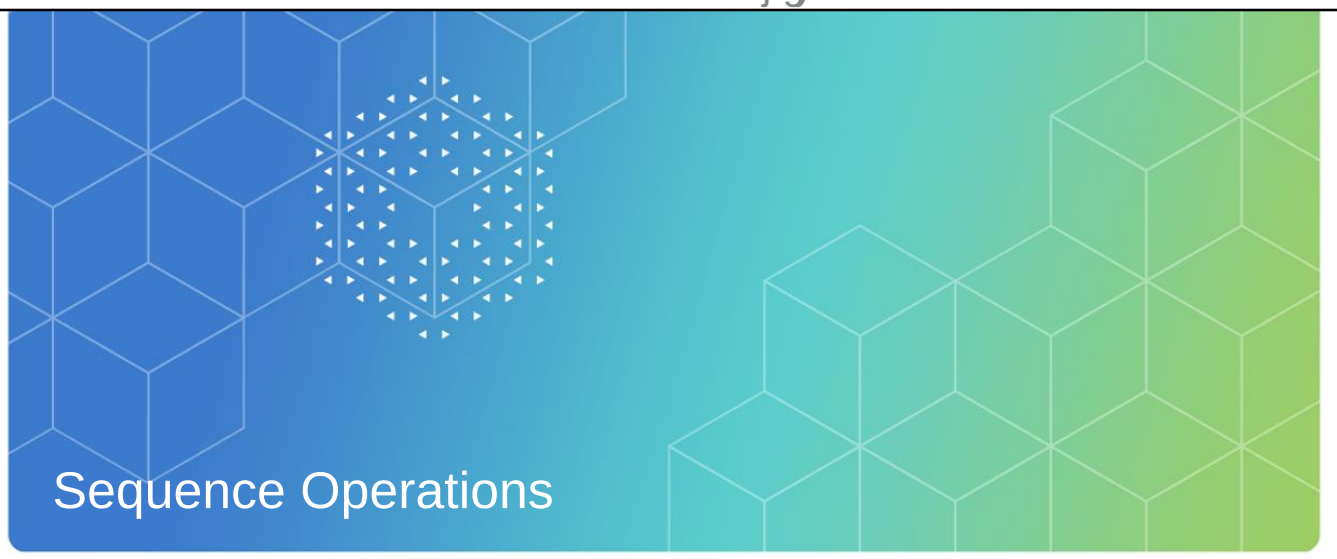
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Module 10
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Module Objectives

Aims

- Advanced sequence operators and features to make sequence and property definition easier

Topics

- Simple named sequences
- Sequence Clocking
- Sequence operators
  - Sequence or
  - Sequence and (non-length matching)
  - Sequence intersect (length matching)
  - Sequence throughout
  - Sequence within
  - first_match()

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Simple Named Sequences

Common sequences or boolean conditions can be declared separately with a unique name

Named sequences used as building blocks for more complex properties or sequences

Sequences can be parameterized

- See later...

Sequence `SABC` defines a sequence of A followed by B followed by C

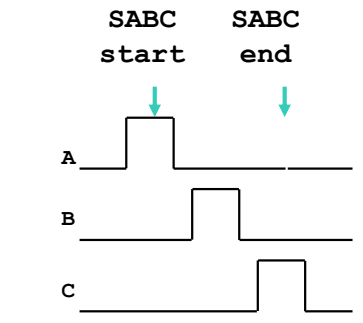
Sequence `ENAB` is a boolean condition

- A sequence of length 1 cycle

```
sequence ENAB;
 (COUNT > 10);
endsequence
sequence SABC;
  (A ##1 B ##1 C);
endsequence
property PS;
    ENAB |=> SABC;
endproperty
```

```
sequence NAME;
 (<sequence>);
 endsequence
```

```
sequence NAME;
 (<expression>);
 endsequence
```



 © Cadence Design Systems, Inc. All rights reserved.

A fundamental approach to describing complex assertions is to break up complex properties into smaller sequences, define these separately as named sequences and build up a definition for the complex sequence using these as building blocks. The sequences are easier to understand, require fewer lines of code and can be used in more than one complex sequence.

Named sequences can be considered as “subprograms” for sequence definition. They can even be parameterized (see topic on Advanced SystemVerilog Assertions).

Property PS states that if count is greater than 10 (the 1 cycle long sequence ENAB has completed), then from the next cycle the sequence A ##1 B ##1 C must start and subsequently complete.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Sequence Clocking

Sequence declarations do not normally contain clocks

- Clocking is inherited from the property or assertion where the sequence was used
- Allows different clocking for same sequence

Sequence can contain clocks

- Property can inherit clock from sequences

Unclocked properties are not allowed

Sequence and property clock can differ

- Multiclock properties are difficult to understand

property clocking

```
sequence S1;
  (A ##1 B);
endsequence
property P1;
  @(negedge CLK3) D |=> S1;
endproperty
property P2;
  @(posedge CLK4) S1 |=> C;
endproperty
```

Sequence without clock

Sequence clock inherited from parent property

sequence clocking

```
sequence S3;
  @(posedge CLK1) D;
endsequence
sequence S2;
  @(posedge CLK2) A ##1 B;
endsequence
property P1;
  S3 |=> S2;
endproperty
```

Multi-clocked property

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Sequences can have their own clock. However, if a property where a sequence is used also specifies a clock which is different, then it is a multiply-clocked property. Rules for the evaluation of such properties are complex.

You will see other details of multiply-clocked properties later in this course.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

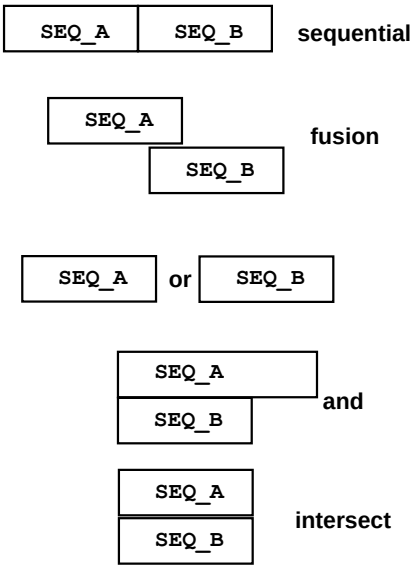
Operators Which Form Relationships with Other Sequences

Cycle delay ## allows sequential sequences

- Also permits fusion where sequences overlap by 1 cycle

Use these operators to define alternative and concurrent sequences

- or
  - 1 or more alternative sequences hold
- and
  - Concurrent sequences start, but need not complete in the same cycle
  - Non-length matching
- intersect
  - Concurrent sequences start and must complete in same cycle
  - Length matching



 © Cadence Design Systems, Inc. All rights reserved.

cadence

So far the cycle delay operator ## has allowed us to define serial sequences. Other SystemVerilog sequence operators allow us to define alternate and concurrent sequences. All these help in creating complex properties.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Sequence Fusion

Where sequences overlap, they can be defined with zero cycle delay

`SEQ_B` starts in the same cycle as `SEQ_A` completes

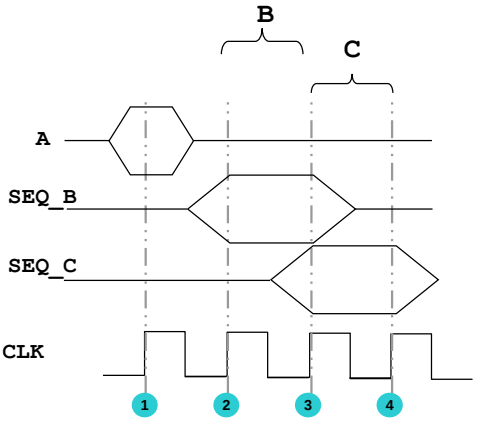
```
SEQ_A ##0 SEQ_B
```

Extremely useful where SEQ_A is of variable or unknown length

```
sequence SEQ_B;
  ...
endsequence

sequence SEQ_C;
  ...
endsequence

property SEQ_FUSION;
  @(posedge CLK)
    A |=> SEQ_B ##0 SEQ_C;
endproperty
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

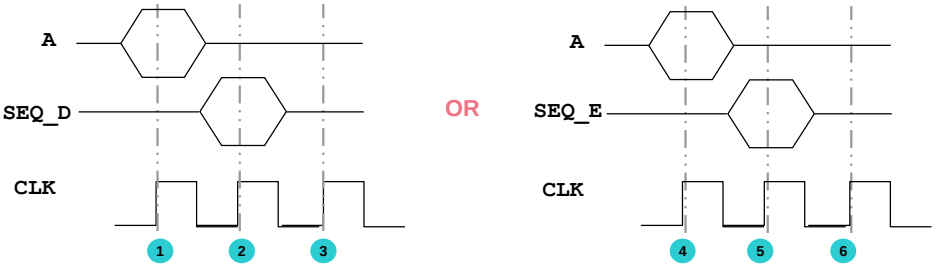
Sequence Or

Defines alternative sequences

Either `SEQ_D` or `SEQ_E` holds at the current cycle

```
(SEQ_D) or (SEQ_E)
```

```
property SEQ_OR;
  @(posedge CLK)(A) |=> ((SEQ_D) or (SEQ_E));
endproperty
```



156 © Cadence Design Systems, Inc. All rights reserved.

cadence

SEQ_B and SEQ_C happen to be sequences of length one cycle in this example.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Sequence And

Defines concurrent sequences

Sequences to start at the same time

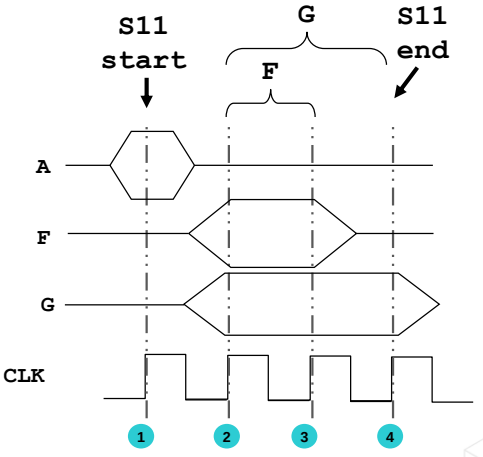
Sequences may finish in *different* cycles

- Non-length matching

`SEQ_F` and `SEQ_G` start in the same cycle, but can end in a different cycles

```
property S11;
   @(posedge CLK) (A) |=> (F and G);
endproperty
```

```
(SEQ_F) and (SEQ_G)
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Sequence Intersect

Sequences must start *and* finish in the same cycle

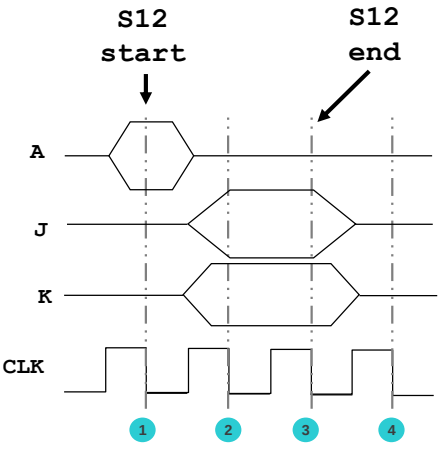
- Length matching `and`

`SEQ_J` and `SEQ_K` start in the same cycle and end in the same cycle

- Useful for placing cycle limits on otherwise "open-ended" sequences

```
property S12;
   @(negedge CLK)
   (A) |=> (SEQ_J intersect SEQ_K);
endproperty
```

```
(SEQ_J) intersect (SEQ_K)
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Sequence Operator Examples

1. From EN1, DPULSE must occur between 4 and 8 times over the next 15 cycles.

```
EN1 |=> (DPULSE[=4:8]) intersect (1'b1[*15])
```

2. If START occurs, then on the next POP either TAG is 2 or, the BUSY signal is turned on and when it is turned off, the signal SKIP is on.

```
START |=>
   POP[->1] ##0
  (TAG==2) or (BUSY[*1:$] ##1 (!BUSY && SKIP) );
```

 © Cadence Design Systems, Inc. All rights reserved.

A default clock must be declared.

1. Nonconsecutive repetition is used to look for 4 to 8 occurrences of DPULSE. Nonconsecutive repetition is open-ended, but the sequence is matched to a cycle count of 15 using length matching `and` to put a cycle limit on the occurrences of DPULSE.
2. If START is true, then look for a rising edge on POP. You use `##0` to check that either TAG=2 or the BUSY sequence starts in the *same* cycle as POP is high. Note that the `!POP[*0:$]` term is open-ended, meaning it is not required that there be a next POP. Also `BUSY[*1:$]` means the BUSY sequence does not have to complete.

© Cadence Design Systems, Inc. All rights reserved.

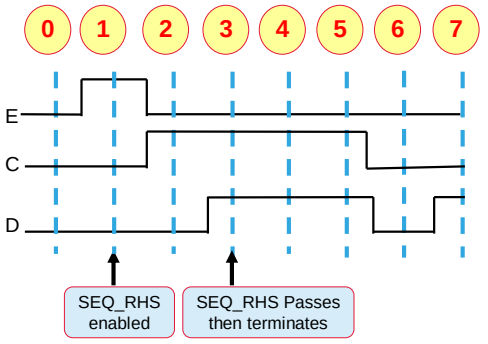
This document is for the sole use of Kuhn Vajay Praveen of Cadence

first_match Operator

If one finds oneself in a position where it looks like `first_match()` is the solution then it probably indicates a lack of understanding of the requirements

```
sequence SEQ_CD;
  C[*1:3] ##1 D;
endsequence
property SEQ_RHS;
  E |=> SEQ_CD ;
endproperty
property SEQ_RHS_EQUIV;
  E |=> first_match(SEQ_CD) ;
endproperty
```

There is an implicit first_match of any sequence on RHS of an implication operator. Hence both properties are equivalent



For more information Google search for "SVA first_match Operator: Why Doesn't PSL Have One?"

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

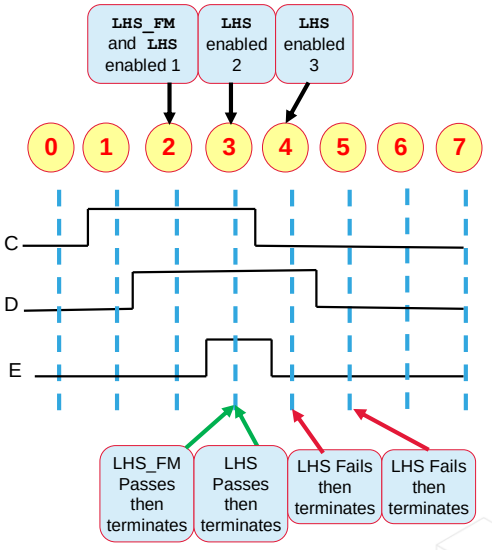
first_match Operator: Removing Undesired Failures

`first_match()` is only relevant when used on LHS of implication operator

```
sequence SEQ_CD;
  C[*1:3] ##1 D;
endsequence

property LHS_FM;
 first_match(SEQ_CD) |=> E;
endproperty

property LHS;
  SEQ_CD |=> E;
endproperty
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Sequence Throughout

Defines that a boolean expression is true for the duration of a sequence

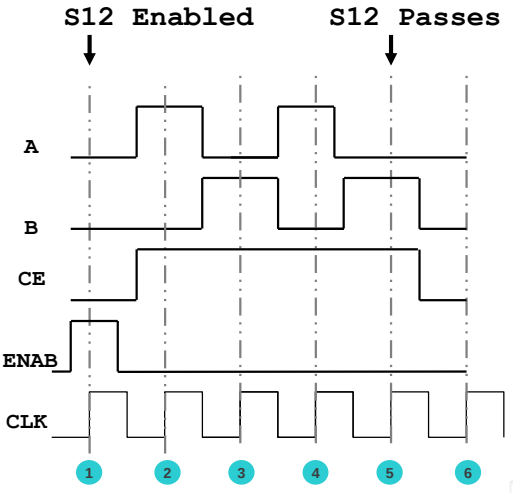
- `expr1` is true on every cycle of `SEQA`
- Equivalent to:

`(expr1)[*0:$] intersect (SEQA)`

```
sequence SAB;
  (A ##1 B);
endsequence

property S12;
  @(posedge CLK)
  ENAB |=>
  (CE) throughout (SAB[*2]);
endproperty
```

```
(expr1) throughout (SEQA)
```



162 © Cadence Design Systems, Inc. All rights reserved.

`expr1` must be true for every cycle of `SEQA`.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Sequence Within

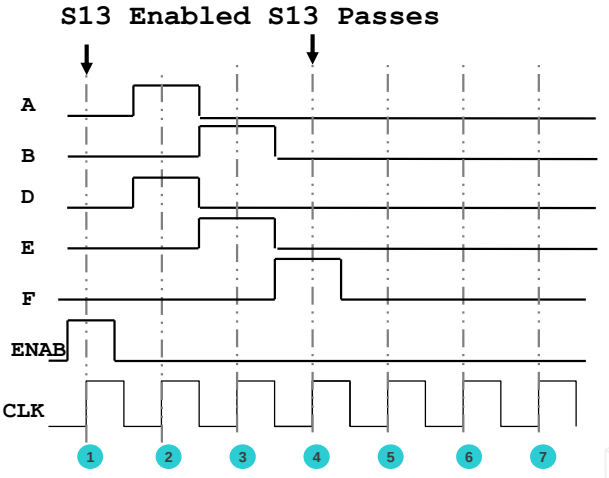
Defines that one sequence is enclosed by another

- SEQA begins at or after SEQB begins
- SEQA completes at or before SEQB ends

```
(SEQA) within (SEQB)
```

```
sequence SAB;
  (A ##1 B);
endsequence
sequence SDEF;
  (D ##1 E ##1  F);
endsequence

property S13;
  @(posedge CLK)
  ENAB |=>
  (SAB) within (SDEF);
endproperty
```



163 © Cadence Design Systems, Inc. All rights reserved.

SEQA is enclosed by SEQB. The resulting sequence begins and ends with SEQB. SEQA must start at or after the cycle in which SEQB starts. SEQA must complete before or in the same cycle as SEQB completes.

within is equivalent to:

```
(1[*0:$] ##1 SEQA ##1 1[*0:$]) intersect (SEQB)
```

i.e. a sequence composed of any number of cycles (including zero) followed by SEQA followed in turn by any number of cycles (including zero) is length matched (intersect) with SEQB.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Within Example : Ack Response Inside a Window

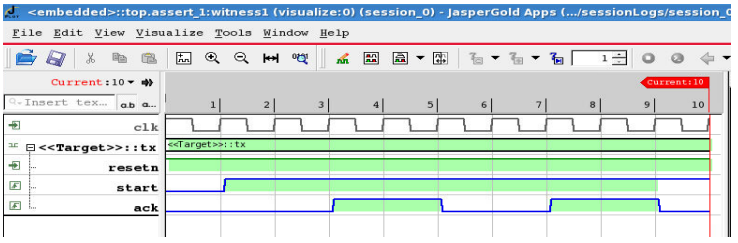
Requirement:

- If I observe the rise of `start` then I must observe between 1 and 3 `ack`'s within a 10 cycle window
- This 10 cycle window begins in the same cycle in which I detect a rise of `start`
- The `ack`'s need to occur in consecutive cycles but at any time in the 10 cycle window

```
S_ACK_WIN_10: assert property (
          @(posedge CLK) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

Looks like a sensible property at first glance.

However, if I ask JasperGold® to cover the property (`witness`), then this is what I see:



It is not expected design behaviour to see 4 acks in the 10 cycle window. **Why does Jasper show the design behave like this?**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

By default JasperGold will cover the left hand side of implication only, namely $rose(start) in our S_ACK_WIN_10.

If we wish to observe a cover of the entire property passing, known as a "witness," then we need to use the elaborate command option:

elaborate –create_related_covers witness

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Why the `within` Property Doesn't Work

The LRM states that these 2 properties are equivalent:

```
S_ACK_WIN_10: assert property (
          @(posedge CLK) $rose(start) |-> ack[*1:3] within 1'b1[*10] );
```

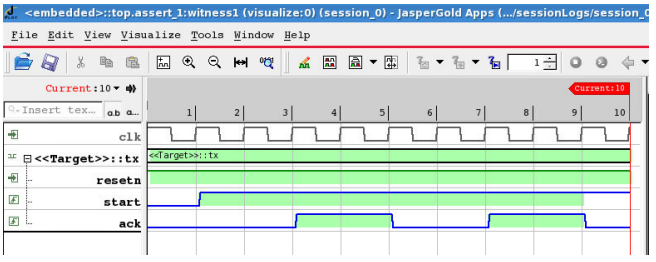
```
S_ACK_WIN_10_EQUIV: assert property (
          @(posedge CLK) $rose(start) |-> (1[*0:$] ##1 ack[*1:3] ##1 1[*0:$]) intersect 1'b1[*10] );
```

Hence a single `ack` anywhere in the window satisfies all requirements and `ack` can have any other behaviour in the remaining 9 cycles of the window

Easy to make mistakes while trying to understand what `intersect` and `within` do

Auxiliary code is easier to understand for such "`count occurrences in a window`" type properties

- Its also easier for the tool



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Limit on Number of Occurrences in a Given Window

A different requirement of Aux Code model to demonstrate the point:

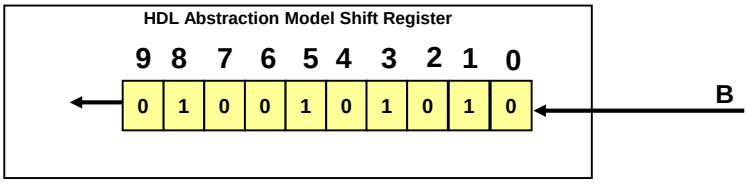
- No more than 3 occurrences of B in any 10 cycles

```
A1: assert property (not strong( (1 [*10]) intersect (B[=4]) ) );
```

Property A1 meets the requirement but …

- Is not particularly easy to conceive or understand – Why do we need strong?
- Is not that easy for the tool to evaluate

Remodel with a simple shift register and a simple boolean property




```
A2: assert property ($countones(my_10_elem_shift_reg) != 4);
```

166 © Cadence Design Systems, Inc. All rights reserved.

Could make these properties assumptions for formal:

- REQ and GNT will be driven by formal tool.
- Following rules of properties and HDL model.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Quiz

1. Write sequences to describe the following properties:
   a. 4 nonconsecutive occurrences of `PULSE` in 10 cycles
   b. `GO` is followed by either `ERR` or 3 to 7 consecutive cycles of `ENB`
2. Write an assertion for the following specification:

A sequence beginning with `STRT`, followed by 6 consecutive occurrences of `PUSH`, during which `KILL` does not occur, must be followed by 6 nonconsecutive occurrences of `POP` before `STOP` can occur.

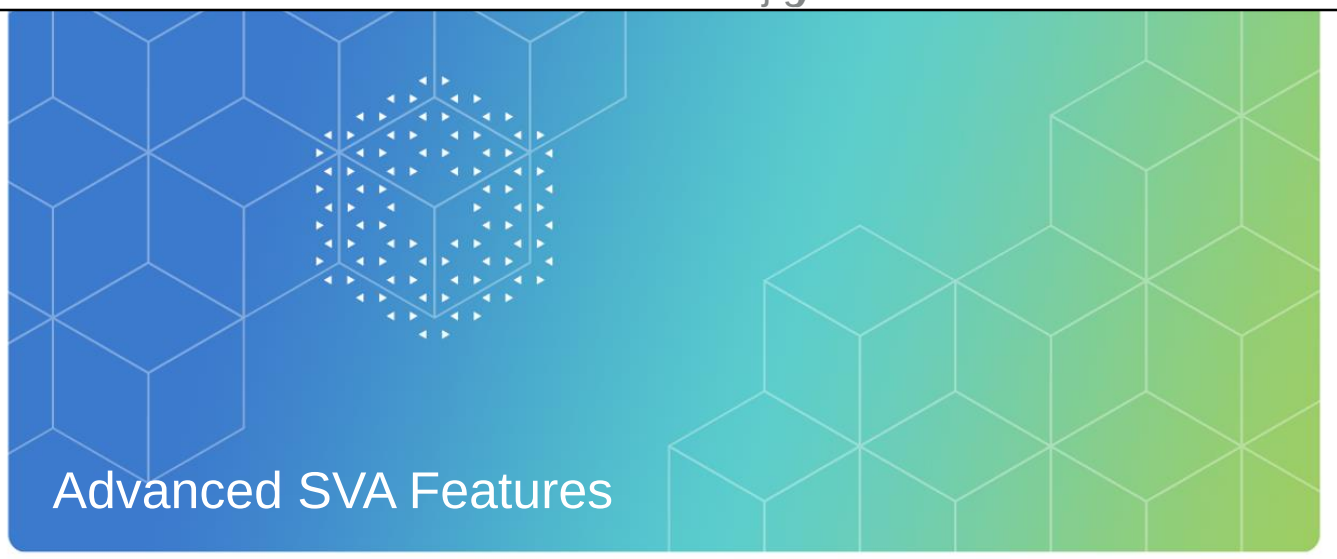
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Module Objectives

Aims

- To explore some of the more advanced features of SystemVerilog sequences and properties

Topics

- Assertion evaluation
- Detecting the end of a sequence
- Action blocks and data manipulation
- Local variables
- Conditional Property Clocking

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Evaluation of Assertions and Properties

Each SystemVerilog simulation timeslot is divided into a series of scheduling regions

A timeslot is a unique simulation time

- Hence, all regions are executed sequentially within the same timeslot

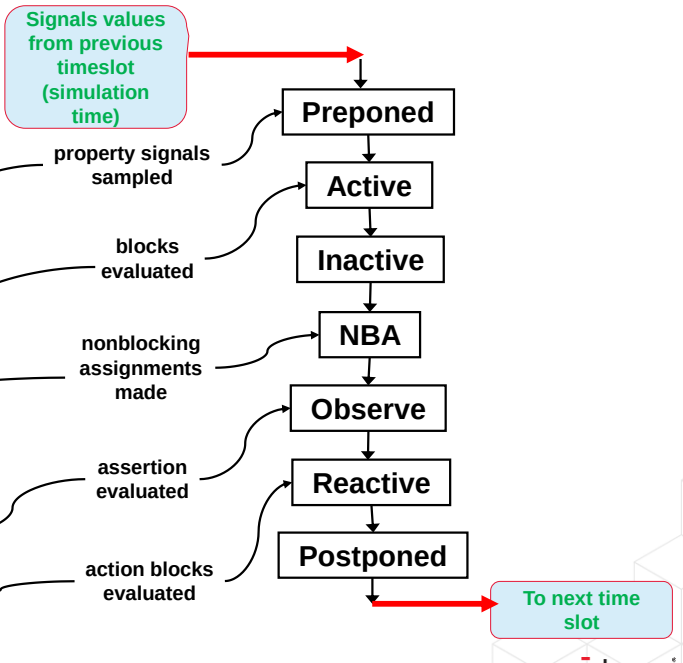
Allows assertions to be evaluated and executed in a predictable way

```
property PIPE;
  @(posedge clk)
  ip -> ##2 op;
end property: PIPE

always @(posedge clk)
  temp <= ip;

always @(posedge clk)
  op <= temp;

A_PIPE : assert property (PIPE)
    $display("pipe passed");
  else
    $warning("pipe failed");
```



170 © Cadence Design Systems, Inc. All rights reserved.

This is a simplified description of the time slots within a SystemVerilog simulation timeslot. There are many more additional scheduling regions between the ones listed above, primarily for PLI callback support.

The key advantage for assertions is that assertions are evaluated at the end of a simulation step, after all assignments have been scheduled, thus avoiding race conditions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Triggered Sequence Method

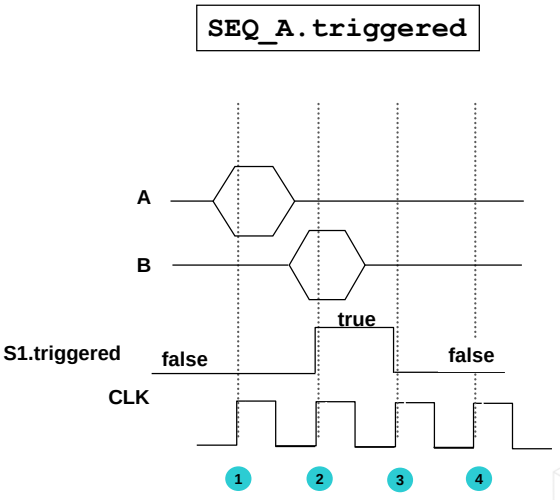
Useful to detect when a sequence completes

- Called the endpoint

Sequence method `triggered` returns true at sequence endpoint

Remains true until simulation time advances

```
sequence S1;
  @(posedge CLK)
  (A ##1 B);
endsequence
```



171 © Cadence Design Systems, Inc. All rights reserved.

Once the endpoint of S1 has been reached, then the triggered method will return true until simulation time advances.

So the waveform shown assumes that no HDL code needs evaluating from S1.triggered being true until the next posedge CLK.

Just as would be the case in a simulation (simulations are event based) which was completely synchronous to the posedge CLK.

In IEEE 1800-2005 there was another sequence method called “ended” which was true only in the observed timeslot of the endpoint of the sequence.

This meant it could not be used in an assertion disable or procedural wait.

This made its usefulness limited, hence why it was deprecated in the IEEE 1800-2009 standard.

To be clear the sequence method “ended” does not exist in the latest System Verilog standard, although you may see it used in legacy code.

Many tool vendors continue to support the sequence method “ended” for backward compatibility.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Triggered Sequence Method Examples

Method `triggered`

- True from observe timeslot until simulation time advances
- Can be used in an assertion
- Can be used to disable an assertion
- Can be used in procedural code

There is also a `sequence.matched()` method, which allows one to synchronize sequences with different clocks

- Refer to the LRM for details

```
'define true 1
sequence EP2;  (req ##1 ack ##2 'true);
endsequence

property SEQ_DIS;
  @(negedge clk) disable iff (EP2.triggered)
  ...
endproperty

always @(posedge clk)
  begin
  wait(EP2.triggered);
  $display("2 cycles after handshake");
  end
property SEQ;
  @(negedge clk) $rose(A) |=> EP2.triggered;
endproperty
```

172 © Cadence Design Systems, Inc. All rights reserved.

cadence

Note that sequences can be extended so that the endpoint is true in later cycles (see sequence `EP2` above).

The triggered sequence method evaluates to true if the given sequence has reached its endpoint in current time step and false otherwise. The triggered status of a sequence is set during the Observe region and persists through the remainder of the time step (i.e., until simulation time advances).

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

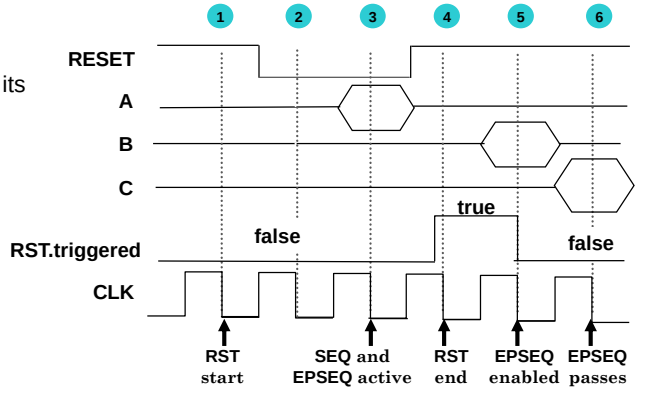
Using Sequence Endpoints

An endpoint can be used in a sequence

An endpoint is **not** equivalent to the sequence of its definition

In `EPSEQ`, `RESET` will be looked for before `A`

In `SEQ`, the `RESET` will only be checked after `A`




```
sequence RST;
  (RESET ##1 !RESET[*2] ##1 RESET);
endsequence

property EPSEQ;
  @(negedge CLK) ((A ##1 RST.triggered ##1 B) |=> (C));
endproperty

property SEQ;
  @(negedge CLK) ((A ##1 RST ##1 B) |=> (C));
endproperty
```

? Question

When does the `SEQ` enabling sequence become incomplete?

173 © Cadence Design Systems, Inc. All rights reserved.

cadence

In property `SEQ`, the `RST` sequence must begin after `A` and be followed by `B`.

In property `EPSEQ`, you expect the end of the `RST` sequence to follow `A`. Therefore, the `RST` sequence must run in parallel with `A`. In fact, because the `RST` sequence is of fixed length, you can fully expand `EPSEQ` as follows:

```
(RESET ##1 ~RESET ##1 ~RESET && A ##1 RESET ##1 B)
```

Endpoint can be useful when describing multiple concurrent or open-ended sequences. Separate sequences can be individually defined and named, and a property declared to check endpoints in the correct order.

The timing diagrams show when `EPSEQ` passes.

The enabling sequence of `SEQ` never occurs. It gets cancelled at the clock falling edge labelled `EPSEQ` enabled, because `RESET` is not `FALSE`.

This shows that an endpoint is not a replacement of the sequence which defines the endpoint.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Action Blocks for Concurrent Assertions

Action blocks can be used with concurrent assertions

- Also in immediate assertions

Statements executed upon assertion pass or fail

- else block executed on assertion failure
- Pass block can be omitted

Severity levels can also be attached

```
property HS;
  @(posedge CLK)
   REQ ##1 ~ACK |-> BUSY;
endproperty

A1 : assert property (HS)
       // pass block
       $display("A1 passed");
     else
       // fail block
       begin
         $warning("A1 failed");
         err_count++;
         -> ack_err_event;
       end
```



Caution **Action block code ignored by synthesis tools**

174 © Cadence Design Systems, Inc. All rights reserved.

Actions blocks allow code to perform additional operations on the pass or failure of an assertion.

Note that the else clause is executed on the failure of the assertion, therefore it is common to omit the pass block and just include the fail block of the else clause.

However an action block can be any of the following:

- Nothing (omitted)
- Pass statements only
- Fail statements only (after else)
- Both pass and fail statements

WARNING: since immediate assertions are ignored by synthesis tools, code in the action block will not be synthesized. Therefore verification code only should be placed in the action blocks.

The optional assertion label can be accessed via %m format specifier.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

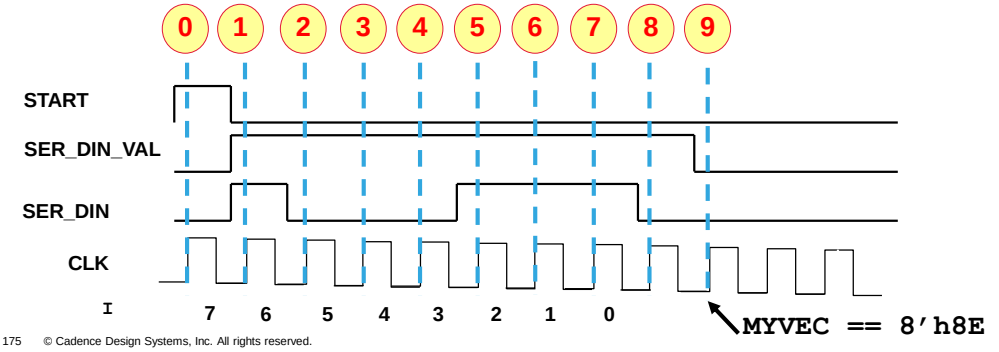
SVA Local Variables to De-Serialize

There is no limit on the number of local variables

This example shows how to collect serial data

Declare local variables inside property

```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
  int I;
  logic [7:0] MYVEC;
     @(posedge clk)
 ( START, I = 7 ) |=>
    (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8] ##1
       MYVEC == EXPECTED_VAL;
endproperty
```



175 © Cadence Design Systems, Inc. All rights reserved.

Remember that each time that the enabled condition occurs (START is true), then a new property instance is spawned and the fulfilling condition is required. Each spawned copy of the property has its own unique set of assertion variables and each has its own fulfilling condition requirement.

It is possible that some behaviour satisfies the fulfilling conditions of multiple property instances at the same time.

In the example, when the enabling condition is met (START is true), then you set the assertion variable i to the highest index number of MYVEC (7).

From the next cycle, you expect to see SER_DIN_VAL true for 8 consecutive cycles ([*8]).

On each of those 8 cycles, do 2 things in this order:

1. Write the current value of SER_DIN to the index of MYVEC indicated by current value of I.
2. Decrement I.

On the cycle after those 8 cycles, you expect the collected value of MYVEC to be equal to some expected value that you have defined elsewhere.

In this case, the definition is the value of a localparam.

In this example, the expected value is 8E hex for the property to pass.

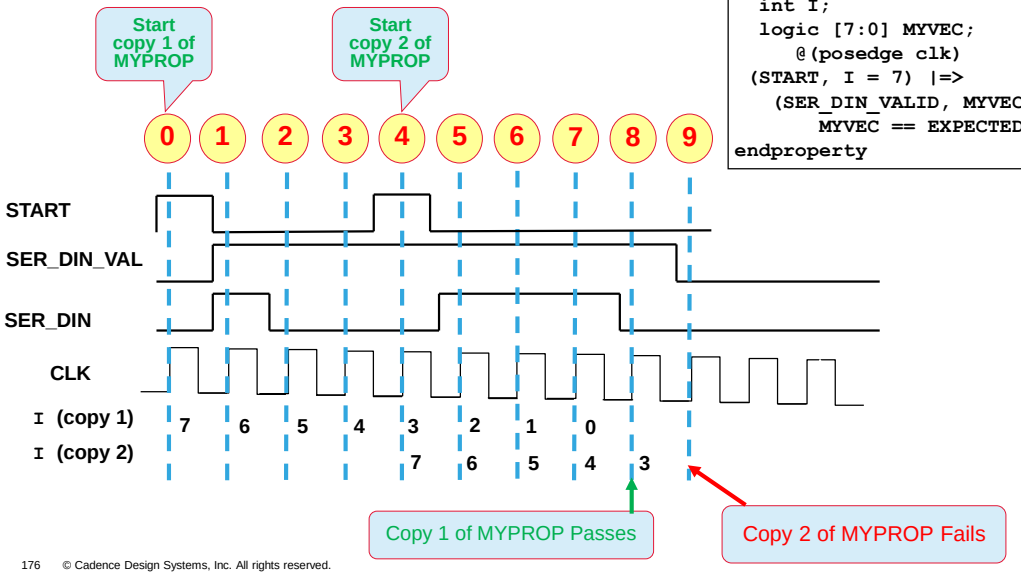
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

What Happens with Overlapping Properties?

Copy1 and Copy 2 cannot communicate their own local value of `I`

```
localparam [7:0] EXPECTED_VAL = 8'h8E;
...
property MYPROP;
  int I;
  logic [7:0] MYVEC;
     @(posedge clk)
 (START, I = 7) |=>
   (SER_DIN_VALID, MYVEC[I]=SER_DIN, I--)[*8] ##1
      MYVEC == EXPECTED_VAL;
endproperty
```



176 © Cadence Design Systems, Inc. All rights reserved.

Remember that each time that the enabled condition occurs (START is true), then a new property instance is spawned and the fulfilling condition is required. Each spawned copy of the property has its own unique set of assertion variables and each has its own fulfilling condition requirement.

It is possible that some behaviour satisfies the fulfilling conditions of multiple property instances at the same time.

In the example, when the enabling condition is met (START is true), then you set the assertion variable i to the highest index number of MYVEC (7).

From the next cycle, you expect to see SER_DIN_VAL true for 8 consecutive cycles ([*8]).

On each of those 8 cycles, do 2 things in this order:

1. Write the current value of SER_DIN to the index of MYVEC indicated by current value of I.
2. Decrement I.

On the cycle after those 8 cycles, you expect the collected value of MYVEC to be equal to some expected value that you have defined elsewhere.

In this case, the definition is the value of a localparam.

In this example, the expected value is 8E hex for the property to pass.

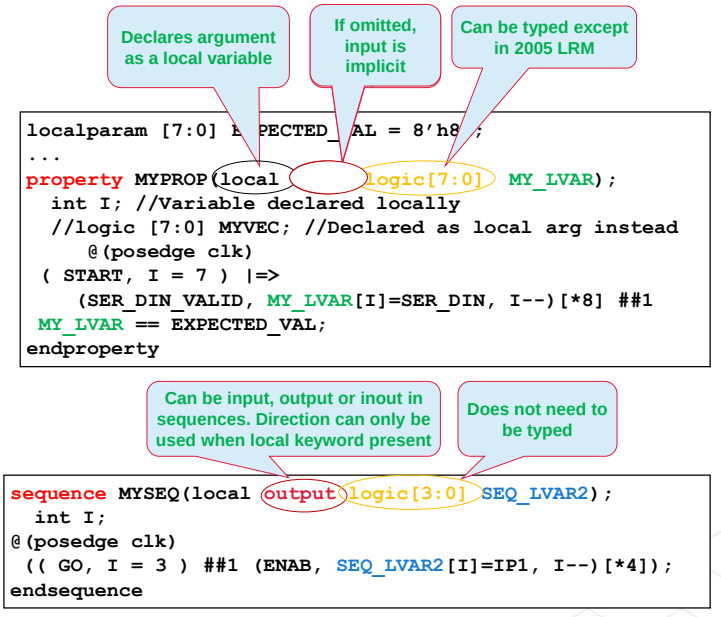
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Local Variables Declared as Arguments to Sequences and Properties

Pragmatically very problematic for these reasons

- What is legal has Language Reference Manual dependencies
- Difficult to debug or be confident your intent is correctly specified
- Sequences and Properties have different rules
- Difficult to be aware of the scheduling issues
- Difficult to comprehend all the behaviors of overlapping threads
  - One property may instance a sequence which in turn instances another sequence, and so on
  - Overlapping threads cannot communicate with each other
- `input` local Var Args are not visible outside of the sequence/property
- Cannot used `triggered()` or `matched()` sequences methods
- Other considerations, e.g., with sequence `and` and sequence `or` operators
- The list of restrictions continues …
  - Advantages don't often outweigh disadvantages



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

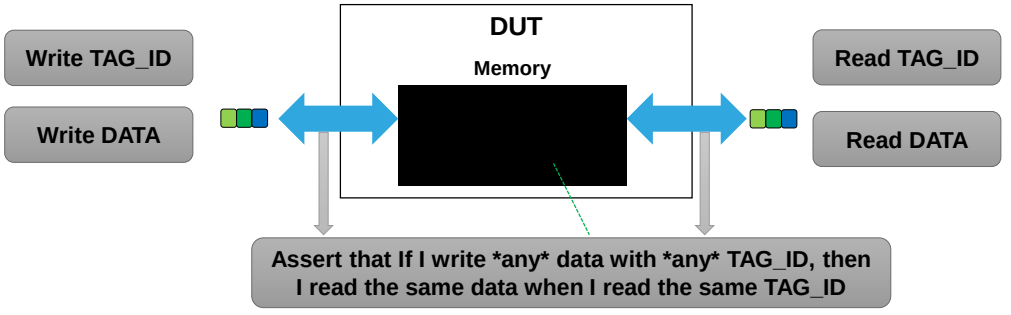
Local Variables to Track Data Values

Data Stored in memory upon write

Data will not be overwritten if it has not been read yet

Memory location freed upon a read, permitting future writes

User responsible for only reading locations with valid data



Assumptions are required so this assert does not produce false negatives

- Only write when TAG_ID location is empty
- Only read when TAG_ID location has valid data, i.e., there has been a valid write previously
- Must eventually read all locations

 © Cadence Design Systems, Inc. All rights reserved.

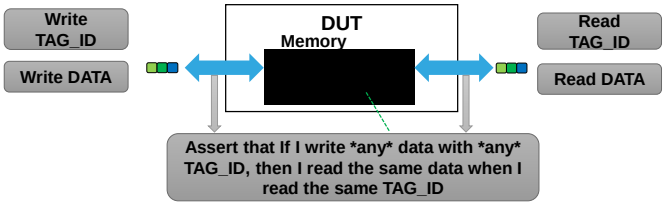
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Local Variables to Track Data Values; Code Examples



Assumptions are required so this assert does not produce false negatives

- Only write when TAG_ID location is empty
- Only read when TAG_ID location has valid data, i.e., there has been a valid write previously
- Must eventually read all locations

```
ASM_ONLY_WRITE_LOCATION_WHEN_MT:   assume property (wr |-> wr_location_mt);

ASM_ONLY_READ_LOCATION_WHEN_VALID: assume property (rd |-> rd_location_valid);

ASM_EVENTUALLY_MUST_RD_VALID_TAG:  assume property (s_eventually empty );

property WR_EVENTUALLY_READ_LVAR;
  logic [TAG_WIDTH-1:0] TAG;  //local vars declaration
  logic [DWIDTH-1:0]    DATA; //local vars declaration
  (wr, TAG = wr_tag_id, DATA = wr_data) |=>
           s_eventually (rd && rd_tag_id == TAG ##1 rd_data ==  DATA);
endproperty

AST_WR_EVENTUALLY_READ_LVAR: assert property( WR_EVENTUALLY_READ_LVAR);
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Multiclocked Properties

Multiclocked properties can use either property implication operator

```
@(negedge CLK1) SEQ1 |=> @(posedge CLK2) SEQ2
```

Synchronizes last cycle of 1st clock to closest next cycle of 2nd clock

```
@(negedge CLK1) SEQ1 |->
  @(negedge CLK1) SEQ2 ##1 @(posedge CLK3) SEQ3
```

Synchronizes last cycle of 1st clock to same delta cycle (if using simulation) of 2nd clock

Otherwise synchronizes last cycle of 1st clock to closest next cycle of 2nd clock

SEQ3 uses next `posedge` CLK3



Caution

Formal tools use a cycle based model hence there is no concept of delta cycles!
Be careful with multi-clocked properties – there are many issues to understand!
Recommend using specific CDC tools rather than trying to code clock domain crossing checks yourself in SVA

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Using multiple clocks with properties is also reasonably straightforward:

- (1) You might need to parenthesize the sequences as shown to help the parser understand that this is property conjunction rather than sequence conjunction. The standard is ambiguous on this issue, as according to the formal syntax, parenthesizing a sequence produces either a sequence expression or a property expression.
- (2) Clocking of the next-cycle implication synchronizes the left and right operands much the same as for multi-clocked sequences.
- (3) For same-cycle implication, the last cycle of the antecedent sequence overlaps the first cycle of the consequent property, so for that cycle both operands must utilize the same clock. They can utilize different clocks for any other of their cycles.
- (4) The if…else construct is also same-cycle implication, so the first cycle of both branches must utilize the same clock used for the test of the conditional expression.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Complex Properties…

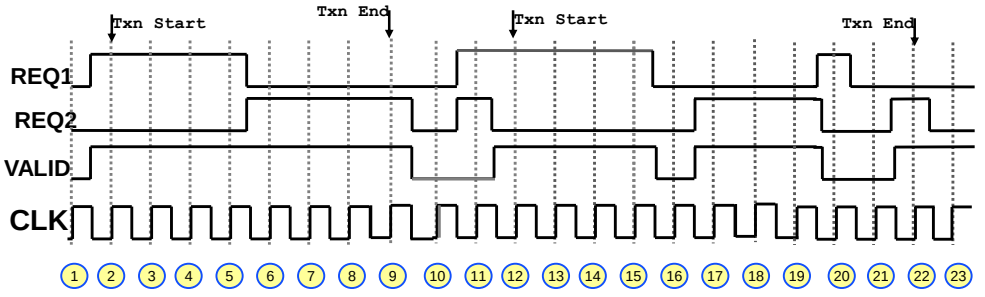
Requirements for a “simple” interface:

- `REQ1` and `REQ2` must not overlap.
- `REQ1` and `REQ2` must both last `exactly` 4 cycles.
- `REQ2` must begin on cycle after `REQ1`.
- If `VALID = '0'`, then it doesn't matter what `REQ1` and `REQ2` are doing.
- `VALID` can go to `'0'` at anytime, for any number of cycles

easy

```
property P1;
(@(posedge CLK)
   not (REQ1 && REQ2 &&
VALID));
endproperty
```

Easy if you do not need to consider **VALID ... but you do**



181 © Cadence Design Systems, Inc. All rights reserved.

The problem is that VALID can go to *0* at any time for any number of cycles.

This greatly complicates what are otherwise simple properties to write.

Remember that you use SVA to do more and better verification in less time.

The waveform diagram shows two correctly completing transactions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prakash of Cadence

… Easy With Complex Clock Expression

Simplify properties with more complex clock expression

Define a qualified sampling clock

- Use **`iff`** clause

REQ1 and REQ2 must not overlap

REQ1 and REQ2 must both last exactly 4 cycles

REQ2 must begin on cycle after REQ1 and then complete

```
property P2;
( @(posedge CLK iff VALID)
   not(REQ1 && REQ2));
endproperty
```

```
property CHK_REQ_LEN (REQ, CLK, VALID, NUM_CYC);
( @(posedge CLK iff VALID)
  ( !REQ ##1 REQ |=> REQ[*NUM_CYC-1] ##1 !REQ );
end property

CHK_REQ1_LEN: assert property(CHK_REQ_LEN(REQ1, CLK, VALID , NUM_CYC));
CHK_REQ2_LEN: assert property(CHK_REQ_LEN(REQ2, CLK, VALID , NUM_CYC));
```

```
sequence REQ_N_CYC(REQ, N)
  REQ[*N];
endsequence

property R2_FOLLOWS_R1
( @(posedge CLK iff VALID)
   REQ_N_CYC(REQ1, 4) |=> REQ_N_CYC(REQ2, 4)
);
endproperty
```

It is typical to not specify a clock for the named sequences

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Using a combination of the 4 simple properties described, check all requirements of the protocol are met. This is a good example of where specifying many simple properties to check that some overall behavior is correct, is easier and less error prone than trying to express the complete behavior in a single property.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Property Clocking Recommendations

If DUT is fully synchronous to clock rising edge

- Use clock rising edge to evaluate properties
  - Enables Formal tools to perform clock optimizations
- Same result but nothing gained by using clock falling edge
  - Gate level simulations probably fail unless you have > ½ cycle slack
- Same result from Dynamic and Formal Verification

Use the most infrequent property clock possible for checking behavior

- Don’t necessarily need to check every property on each active clock

Property clock does not need to be periodic

- For example, you may only want to check a property when `write address` changes

Always take time to understand the issues

- Property clocking is absolutely fundamental
- If you get this part wrong, then everything else will be wrong

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Summary

You now have some powerful constructs for describing complex assertion sequences

- Endpoints
  - Convert the completion of a sequence into a boolean
  - Allow multiple concurrent sequences to be described
- Action blocks
  - Allow procedural code to be executed on the pass or failure of an assertion
- Assertion local variables
  - Easier definition of complex assertions by accessing local dynamic variables on the completion of a sequence or property
- Defining conditional and multi-clocked properties

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Quiz

1. Expand `WHATSEQ`:

```
sequence ONE;   (A ##1 B[*3] ##1 C);   endsequence
sequence TWO;   (K ##1 L ##1 M ##1 N); endsequence
sequence THREE; (X ##2 Y[*2]);         endsequence
sequence WHATSEQ;
  (ONE.triggered ##1 TWO.triggered ##1 THREE.triggered);
endsequence
```

2. In the property below, `end` must go high within 5 and 15 cycles of `start`. For a response of 10 cycles or less, `short` is asserted with `end`, For greater than 10 cycles, `long` is asserted with `end`. Modify the property to check right number of cycles for each alternate response:

```
property count_cycles;
  @(posedge clk)
  $rose(start) |=> !end[*4:14]
  ##1 ((end && short) or (end && long));
endproperty
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Constructs Which Form Properties

Module 12
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Module Objectives

Aims

- Describe the constructs that form properties
  - Namely, constructs which return a property expression
- Understand which of these are of pragmatic use and why

Topics

- Constructs and Operators of Pragmatic Use
  - until family of property constructs
  - always property construct
  - followed-by operators
  - implies operator
- Constructs and Operators NOT of Pragmatic Use
  - if-else Properties
  - Conjunction and Disjunction operators
  - iff operator
  - nexttime family of operators

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Overview of Property Forming Constructs

IEEE 1800-2009 introduced many new operators which return a property expression

Some of these are of pragmatic use and some are not

As standards are decided by committee, it's the harsh reality that bad ideas don't always get rejected

Some of these operators were introduced because they have a practical use and others just because PSL has a similarly named operator

Many of the expressions allow one to form a property expression which can be used where the syntax requires a property expression

- This couldn't have been done previously in some cases

This section is divided into 2 sections, one section having expressions of practical use and the other having expressions little or not of practical use

We show descriptions of each operator and some practical examples

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Constructs Which Form Properties:
Constructs of Some Pragmatic Value

Submodule 12-1
Revision 1.0
Version 5.1

Estimated Time:
- Lecture
- Lab

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

until Property Expression

Syntax: <property expr 1>
until <property expr 2 >

```
P_UNTIL_NOVRLP_W: assert property (
        B |=> (C until D) );
```

This is the weak non-overlapping form of until

If `B` occurs then from the next cycle, `C` is required to be true every cycle until `D` is true

There are other forms of the until operator, combinations of weak/strong and [non-]overlapping forms

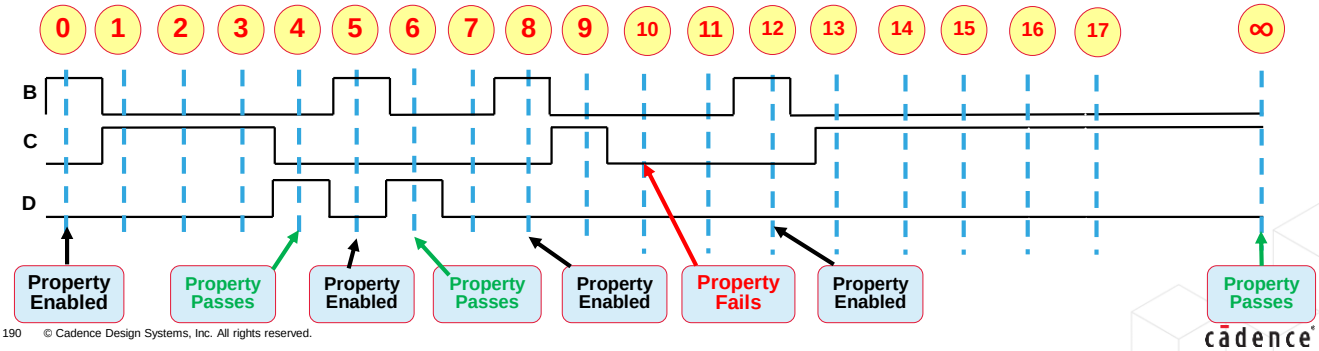
- We discuss these later

`P_UNTIL_NOVRLP_W` is the weak non-overlapping form which means:

- `D` is never required occur, as long as `C` stays true – a weak property
- `C` is not required to be true in the cycle where `D` occurs – non-overlapping

This is the equivalent property described with sequences

```
P_UNTIL_NOVRLP_W_SEQ: assert property (
        B |=> (C[*] ##1 D) );
```



190 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

until_with Property Expression

Syntax: <property expr 1>

until_with <property expr 2 >

```
P_UNTIL_OVRLP_W: assert property (
    B |=> (C until_with D) );
```

This is the weak overlapping form of until

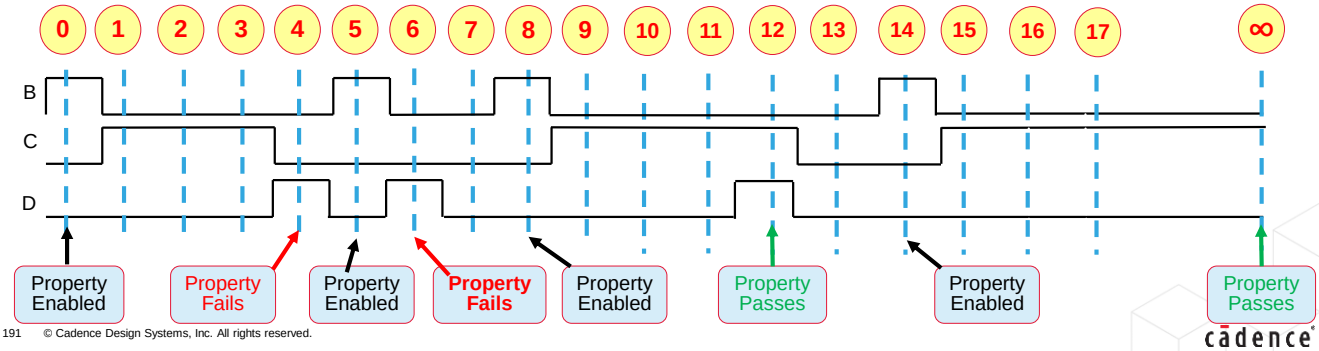
If `B` occurs then from the next cycle, `C` is required to be true every cycle until `D` is true, including the cycle `D` becomes true

`P_UNTIL_OVRLP_W` is the weak overlapping form which means:

- `D` is never required occur, as long as `C` stays true – a weak property
- `C` is required to be true in the cycle where `D` occurs – overlapping

This is the equivalent property described with sequences

```
P_UNTIL_OVRLP_W_SEQ: assert property (
    B |=> (C[+] ##0 D) );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

s_until Property Expression

Syntax: <property expr 1>

s_until <property expr 2 >

If `B` occurs then from the next cycle, `C` is required to be true every cycle until `D` is true

`P_UNTIL_NOVRLP_S` is the strong non-overlapping form which means:

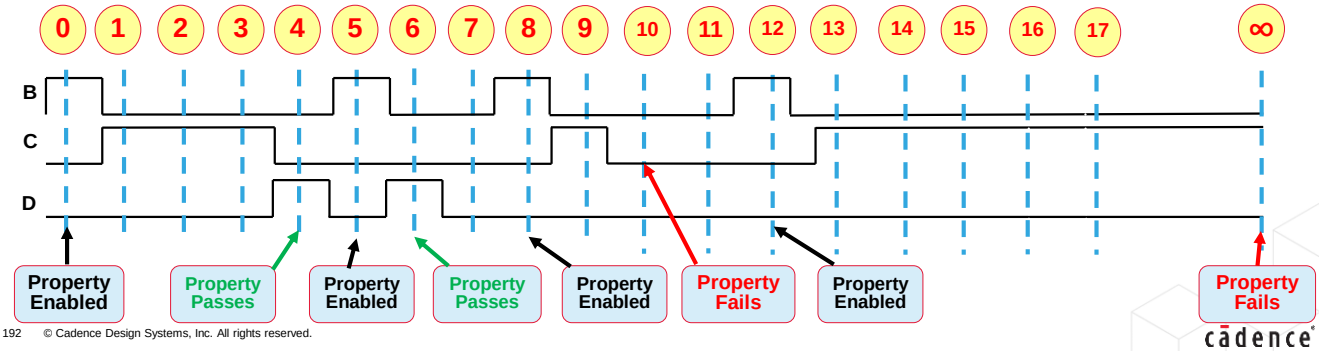
- `D` is required occur – a strong property
- `C` is not required to be true in the cycle where `D` occurs – non-overlapping

```
P_UNTIL_NOVRLP_S: assert property (
        B |=> (C s_until D) );
```

This is the strong non-overlapping form of until

This is the equivalent property described with sequences

```
P_UNTIL_NOVRLP_S_SEQ: assert property (
        B |=> strong(C[*] ##1 D) );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

s_until_with Property Expression

Syntax: <property expr 1>

s_until_with <property expr 2 >

```
P_UNTIL_OVRLP_S: assert property (
    B |=> (C s_until_with D) );
```

This is the strong overlapping form of until

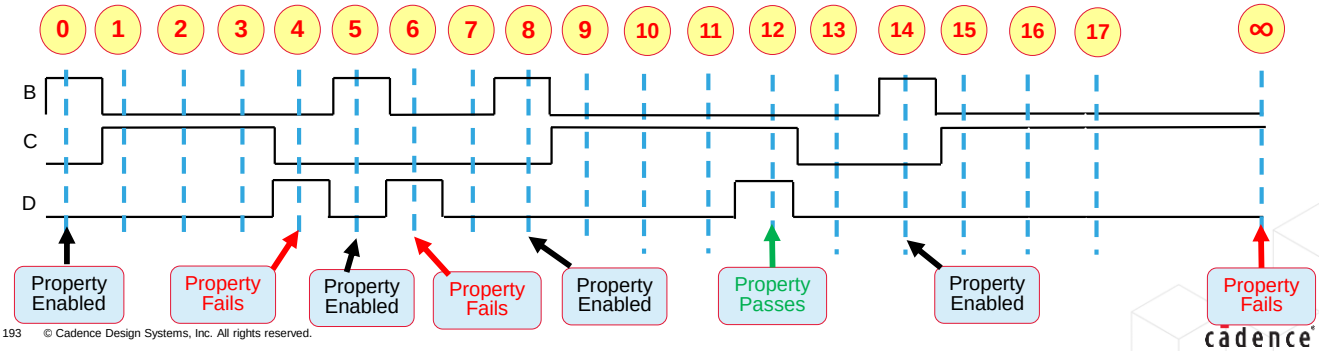
If `B` occurs then from the next cycle, `C` is required to be true every cycle until `D` is true, including the cycle `D` becomes true

`P_UNTIL_OVRLP_S` is the strong overlapping form which means:

- `D` is required occur – a strong property
- `C` is required to be true in the cycle where `D` occurs – overlapping

This is the equivalent property described with sequences

```
P_UNTIL_OVRLP_S_SEQ: assert property (
    B |=> strong(C[+] ##0 D) );
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Should I Use the until Form or the Sequence Form?

`until` form disadvantages:

- With the until forms one has to remember what each of the different forms does
  - This is not convenient, easy to make mistakes
- With sequences there is never any ambiguity or doubt
  - All one has to know is how to define a sequence

`until` form advantage:

- One could reasonably argue that its more readable than the sequence form
  - A weak argument in my view

```
P_UNTIL_NOVRLP_W: assert property (
        B |=> (C until D) );
```

≡

```
P_UNTIL_NOVRLP_W_SEQ: assert property (
          B |=> (C[*] ##1 D) );
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

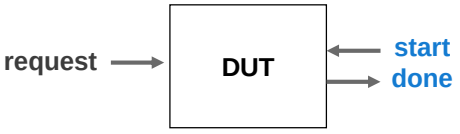
Input Livelocks

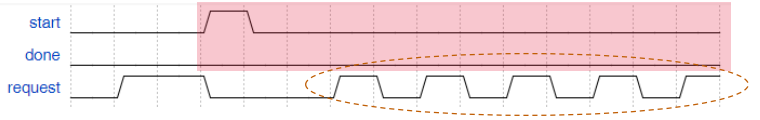
Liveness assertions may fail because inputs keep the design busy

- e.g., new packets continue to enter causing resource never to be available

Example

```
assert property (start |-> s_eventually done);
```





False Failure ☹
done never asserted because requests keep coming

Not sufficient to say that inputs will eventually be idle

- e.g., `assume (s_eventually !request)` does not solve the case above

Need to forbid new input requests/events/transactions during the loop

- See next

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Input Livelocks Solved Using always

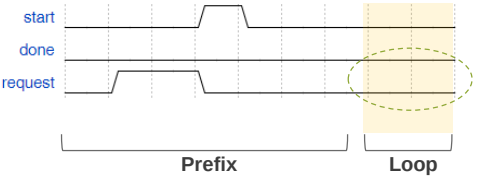
`always` syntax

```
always <property expression>
```

Defines a property which requires <property expression> to be true in the current cycle and all future cycles

Use `s_eventually always` to force events to be always true in loop

```
assume property (
  s_eventually always !request
);
```



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Followed-By Operator

If `sequence` completes then `property` must be true

`sequence` must complete

Example states:

- Sequence `B ##1 C` **must** occur
- Cycle after (non overlapping) which `B ##1 C` occurs then either:
  - `D` occurs followed next cycle by `E`, or
  - `D` does not occur (vacuous)

Overlapping

```
sequence #-# property
```

Nonoverlapping

```
sequence #=# property
```

```
property FB_NC(
    @(posedge clk) (B ##1 C) #=# (D |=> E) );
endproperty
```

- LRM defines:
  - Followed-By operator `#=#` as being equivalent to: **not (*sequence_expr* |=> not *property_expr*)**
  - **Followed-By operator `#-#` as being equivalent to: not (*sequence_expr* |-> not *property_expr*)**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Followed-By Operator

If `sequence` completes then `property` must be true

`sequence` must complete

Example states:

- Sequence `B ##1 C` **must** occur
- Cycle after (non overlapping) which `B ##1 C` occurs then either:
  - `D` occurs followed next cycle by `E`, or
  - `D` does not occur (vacuous)

Overlapping

```
sequence #-# property
```

Nonoverlapping

```
sequence #=# property
```

```
property FB_NC(
    @(posedge clk) (B ##1 C) #=# (D |=> E) );
endproperty
```

- LRM defines:
  - Followed-By operator `#=#` as being equivalent to: **not (*sequence_expr* |=> not *property_expr*)**
  - **Followed-By operator `#-#` as being equivalent to:** **not (*sequence_expr* |-> not *property_expr*)**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

What Does It Mean If We `assert` a Followed-By Operator?

sequence_expr* #-# *property_expr

Let us consider that *`sequence_expr (S)`* and *`property_expr (P)`* *are both booleans*

not (*sequence_expr* |-> not *property_expr*)

P	S	not P	S |-> not P	not (S |-> not P)
0	0	1	True	False
0	1	1	True	False
1	0	0	True	False
1	1	0	False	True

```
FC_SC_ALL_BOOLS: assert property(S #-# P);
```

The only cycles on which this does not fail are when S is proceeding and P is passing

So how can Followed-By be useful?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

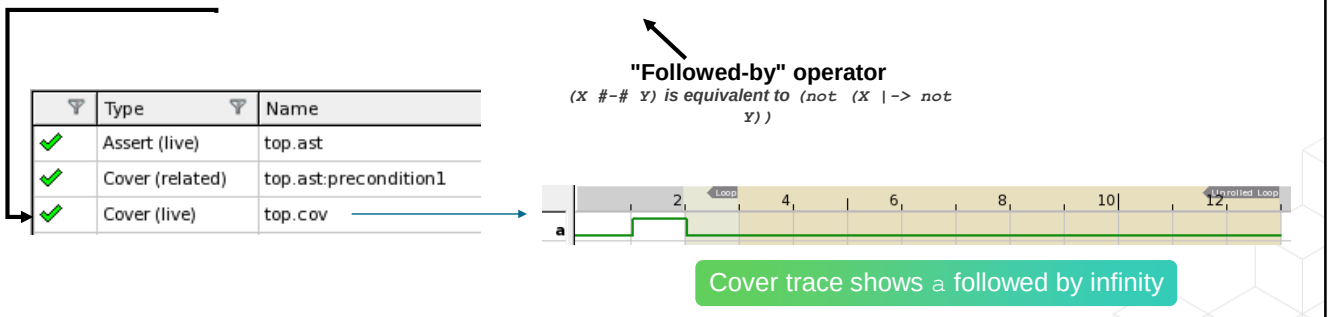
Liveness Covers

Possible to create liveness covers using the `always` keyword

Recommended to have one liveness cover for each assertion

- Make sure precondition can be hit and extended to infinity

```
assert property (a |-> s_eventually b);
cover property (a #-# always 1);
```



 © Cadence Design Systems, Inc. All rights reserved.

Requires `analyze -sv09` and `elaborate -lrm_cover_property`

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

implies Operator

For the expression to pass `prop2` must be true if `prop1` is **`prop1 implies prop2`**

This is a unidirectional relationship

- If `prop1` is not true we don't care about `prop2`

`prop1` and `prop2` are evaluated concurrently

Also useful for sequences

- **Remember that a property expression can be a sequence expression**

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Don't Confuse `implies` with `|=>`

For **`IMPLIES_GOOD`** to pass takes 2 cycles:

- B and D need to be true on the same cycle
- C and E need to be true on the cycle after that

```
property IMPLIES_GOOD(
    ( (B ##1 C) implies (D |=> E) );
endproperty
```

For **`IMP_BAD_NC`** and **`IMP_BAD_M_IMP`** to pass takes 4 cycles:

- B needs to be true, then C true next cycle, then D true next cycle, then E true the cycle after that

```
property IMP_BAD_NC(
    ( (B ##1 C) |=> (D |=> E) );
endproperty
```

```
property IMP_BAD_M_IMP(
    ( B |=> C |=> D |=> E );
endproperty
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Constructs Which Form Properties:
Constructs of Little or No Pragmatic Value

Submodule	**12-2**
Revision	**1.0**
Version	**5.1**

Estimated Time:

- **Lecture**
- **Lab**

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

if-else Property Expression Syntax

Syntax: if (<Boolean>) <property expr 1>
else <property expr 2 >

```
property if_else_NC;
  if (A) B |=> C
   else D |=> E;
endproperty
```

```
property if_else_NC_seq;
  if (A) B |=> C ##1 J
   else D |=> E ##1 K;
endproperty
```

Works with sequences

```
property if_else_complicated;
  if (Y) R |=> S ##1 T
   else (if (F) N |=> P ##1 Q else M |=> V ) ;
endproperty
```

Don't over complicate matters

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

Are They of Pragmatic Use?

Disadvantages

- Complicates coverage analysis
  - Did I execute all branches (might be multiple if-else branches)?
- If the property is asserted and fails, I need to spend time analyzing which branches(s) fail
- In formal, if the property is asserted and passes, I don't know all branches will always pass without more effort and risk
- In formal, one normally decomposes the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- It does not do anything one couldn't easily describe in a less complicated way

```
P1_if_else:
  assert property(
    if (A) B |=> C
    else D |=> E);
```

≡

```
PROP_A_TRUE_IMP :
  assert property( A |-> B |=> C);
PROP_A_FALSE_IMP:
  assert property(!A |-> D |=> E);
```

≡

```
PROP_A_TRUE_BOOL :
  assert property( A && B |=> C);
PROP_A_FALSE_BOOL:
  assert property(!A && D |=> E);
```

Advantages

- I have never seen one since if-else was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

if-else Example with Sequence on RHS




```
P2: assert property ( if (A) B |=> C ##1 J
                      else D |=> E ##1 K);
```

The waveform shows an example (cover) of the property passing (called a witness)

Any signal is only required to be high in the cycle where it is shaded green for `P2` to pass

If `D` were not true at cycle 1 then this does not mean `P2` fails as the LHS would be incomplete (vacuous pass)

If `E` were not true at cycle 2 or `K` were not true at cycle 3 then this means `P2` fails

 © Cadence Design Systems, Inc. All rights reserved.

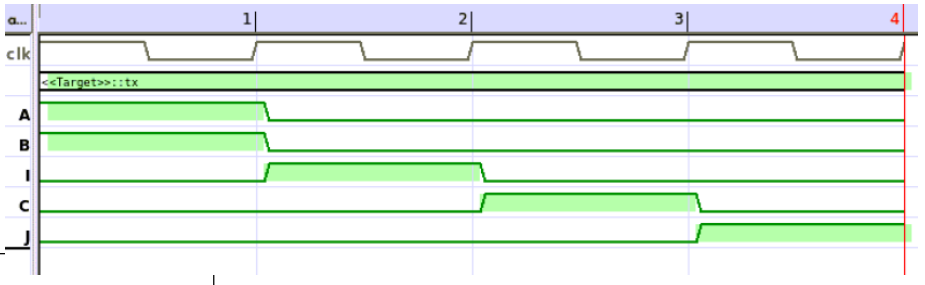
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

if-else Example with Sequence on LHS




```
property if_else_NC_seq_LHS;
  if (A) B ##1 I |=> C ##1 J
   else D ##1 L |=> E ##1 K;
endproperty
P1: assert property (if_else_NC_seq);
```

The waveform shows an example (cover) of the property passing (called a witness)

Any signal is only required to be high in the cycle where it is shaded green for `P1` to pass

If `I` were not true at cycle 2 then this does not mean `P1` fails as the LHS would be incomplete (vacuous pass)

If `C` were not true at cycle 3 or `J` were not true at cycle 4 then this means `P1` fails

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

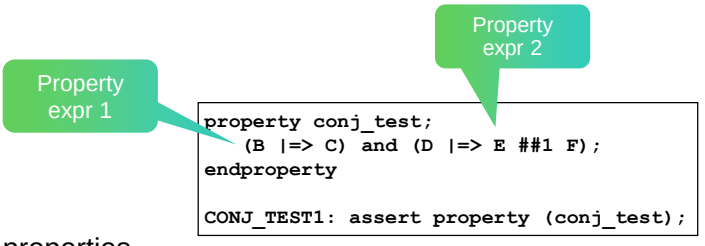
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prakash of Cadence

Conjunction Property Expression Syntax

Syntax: <property expr 1>

and <property expr 2 >



CONJ_TEST1 is a conjunction of 2 properties

CONJ_TEST1 succeeds where both of those 2 properties hold

- Either vacuously or non-vacuously

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Praveen of Cadence

Are They of Pragmatic Use?

Disadvantages

- Complicates coverage analysis
  - Did both properties complete non-vacuously at the same time?
- If the property is asserted and fails, I need to spend time analyzing which property expression(s) failed
- In formal, if the property is asserted and passes, I don't know both property expression(s) will always pass non-vacuously without more effort and risk
- In formal, one normally decomposes the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- It does not do anything one couldn't easily describe in a less complicated way

```
property conj_test;
   (B |=> C) and (D |=> E ##1 F);
endproperty
CONJ_TEST1: assert property (conj_test);
```

≡

```
//Property Set 2
I_B_IMP_C: assert property (B |=> C);
I_D_IMP_EF: assert property (D |=> E ##1 F);
```

Advantages

- I have never seen one since property "and" was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

 © Cadence Design Systems, Inc. All rights reserved.

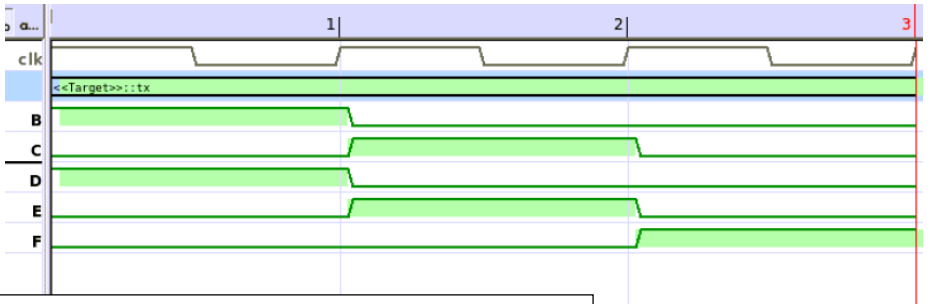
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Conjunction Property Example




```
P2: assert property ((B |=> C) and (D |=> E ##1 F);
```

The waveform shows an example (cover) of the property passing (called a witness)

Any signal is only required to be high in the cycle where it is shaded green for `P2` to pass

If `D` were not true at cycle 1 then this does not mean `P2` fails as the LHS would be incomplete (vacuous pass)

If `E` were not true at cycle 2 or `F` were not true at cycle 3 then this means `P2` fails

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

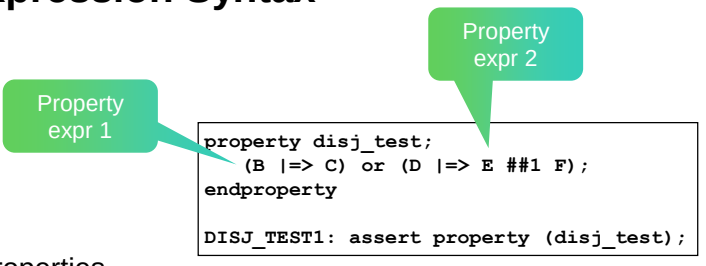
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Disjunction Property Expression Syntax

Syntax: <property expr 1>

or <property expr 2 >



DISJ_TEST1 is a disjunction of 2 properties

DISJ_TEST1 succeeds where either (or both) of those 2 properties hold

- Either vacuously or non-vacuously

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Are They of Pragmatic Use?

Disadvantages

- Complicates coverage analysis
  - Which property(s) complete non-vacuously?
- If the property is asserted and fails, I need to spend time analysing which property expression(s) failed
- In formal, if the property is asserted and passes, I don't know both property expression(s) will always pass non-vacuously without more effort and risk
- In formal, one normally decomposes the problem in order to run proofs in parallel to reduce runtime
  - i.e., have more than one property to describe the equivalent behaviour of the single property
- If it doesn't matter if one property expression fails then why bother writing an assertion in the first place?
  - Non-trivial to write a set of properties equivalent to a "property or"
- Normally better (and the actual user intent) to define a single property with the "sequence or" operator (which is different from the property or)

```
SEQ_OR_EG: assert property ( (B || D) |=> C or (E ##1 F) );
```

Advantages

- I have never seen one since property "or" was first introduced in the 2005 LRM
- I do not think having a single property is a practical advantage

 © Cadence Design Systems, Inc. All rights reserved.

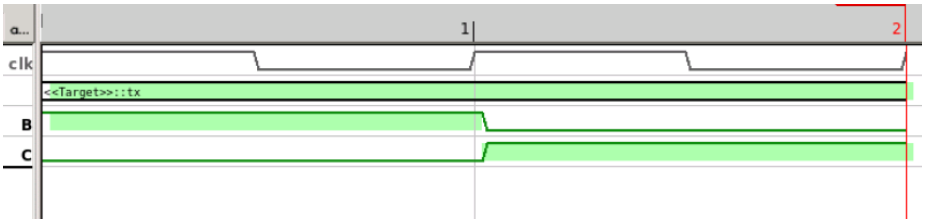
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Disjunction Property Example




```
P2: assert property ((B |=> C) or (D |=> E ##1 F);
```

The waveform shows an example (cover) of the property passing (called a witness)

Any signal is only required to be high in the cycle where it is shaded green for `P2` to pass

Note that signals `D`, `E` and `F` not shown as they are irrelevant for `P2` passing given what values `B` and `C` have

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

iff Operator

Unlike implication (`|=>, |->`) `iff` is bi-directional

```
prop1 iff prop2
```

- **Either both are true or neither of them are**

LRM defines these as **being equivalent**

prop1 `iff` prop2

(prop1 `implies` prop2) and (prop2 `implies` prop1)

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prakash of Cadence

nexttime Property Expression Syntax

Syntax: **nexttime** [index] <property expr 1>

The index is optional

Both of these properties are exactly equivalent

```
AST_D_NEXTT_E_2_F: assert property  (
        D   |-> nexttime[1] E ##1 F );
```

Indexed

```
AST_D_NEXTT_E_2_F: assert property  (
         D   |-> nexttime E ##1 F );
```

Non-indexed

The optional index must be a constant expression

`nexttime` and `nexttime[1]` are equivalent

There are multiple forms of the `nexttime` operator, combinations of weak/strong and [non-]indexed forms

- We discuss these later

They are what I describe as WWYEWTDT properties

- Why would you ever want to do that?

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

nexttime Property Example – Weak Indexed Form

Syntax: **nexttime[index]** <property expr 1 >

```
D_NEXTT_E_2_F_W: assert property  (
      D   |-> nexttime[2] E ##1 F );
```

This is the weak indexed form of nexttime

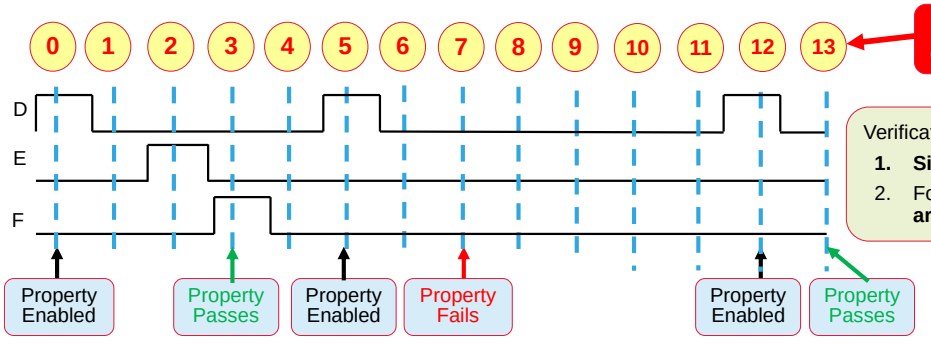
If D occurs then after 2 cycles, E is required to be true and F must be true in the cycle after that

D_NEXTT_E_2_F_W is the weak indexed form which means:

- If D occurs, and verification finishes the next cycle then E or F are not obliged to occur

This is the equivalent property described with sequences

```
D_NEXTT_E_2_F_W_SEQ:
assert property  (
      D   |-> ##2 E ##1 F );
```



Verification ends at cycle 13

Verification ended means:

1. **Simulation – We stopped the simulation**
2. Formal – [over]constraints mean that there **are no further cycles to explore**

216 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Pravesh of Cadence

s_nexttime Property Example – Strong Indexed Form

Syntax: **s_nexttime[index]** <property expr 1 >

```
D_NEXTT_E_2_F_S: assert property  (
      D   |-> s_nexttime[2] E ##1 F );
```

This is the strong indexed form of nexttime

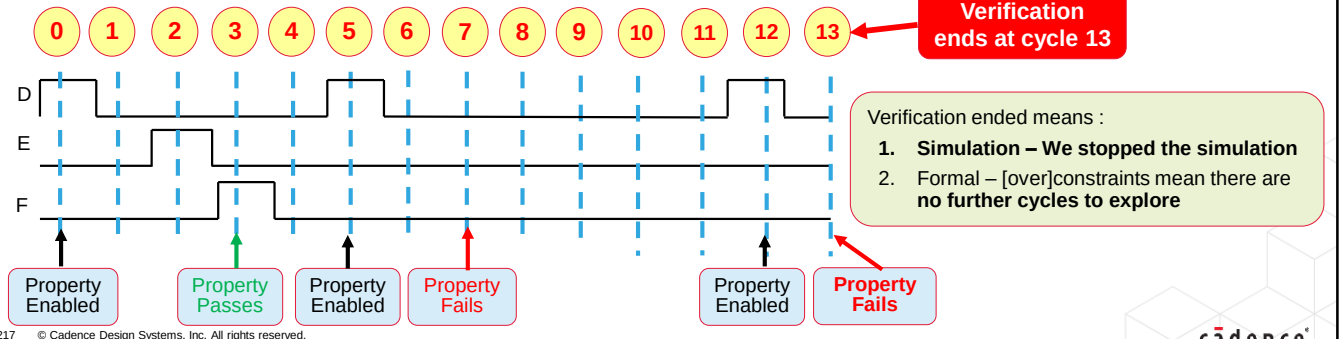
If D occurs then after 2 cycles, E is required to be true and F must be true in the cycle after that

D_NEXTT_E_2_F_S is the strong indexed form which means:

- If D occurs then E and F are obliged to occur

This is the equivalent property described with sequences

```
D_NEXTT_E_2_F_S_SEQ:
assert property  (
      D   |-> strong(##2 E ##1 F) );
```



Verification ended means :

1. **Simulation – We stopped the simulation**
2. Formal – [over]constraints mean there are **no further cycles to explore**

217 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

Are They of Pragmatic Use?

Disadvantages

- One has to remember how they behave, including all of the various forms
  - It invites human error
- It doesn't do anything which one could describe in a simpler and more intuitive way

```
D_NEXTT_E_2_F_W: assert property  (
      D   |-> nexttime[2] E ##1 F );
```



```
D_NEXTT_E_2_F_W_SEQ:
assert property  (
      D   |-> ##2 E ##1 F );
```

Advantages

- I have never seen one since `nexttime` was first introduced in the 2009 LRM
  - I do not think one could even argue that its more readable

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Summary

Not all property forming constructs or operators are useful

With some exceptions, the operators can often be described in a more straightforward and standard way using sequences

- The (especially useful) exceptions being `always` and `followed-by`

Just because something is in the LRM does not mean it is good or the only way of doing something

 © Cadence Design Systems, Inc. All rights reserved.

cadence

Assertions are half the battle – they catch bugs that occur.

Stimulus is the other half of the battle – the right stimulus is necessary to trigger bugs.

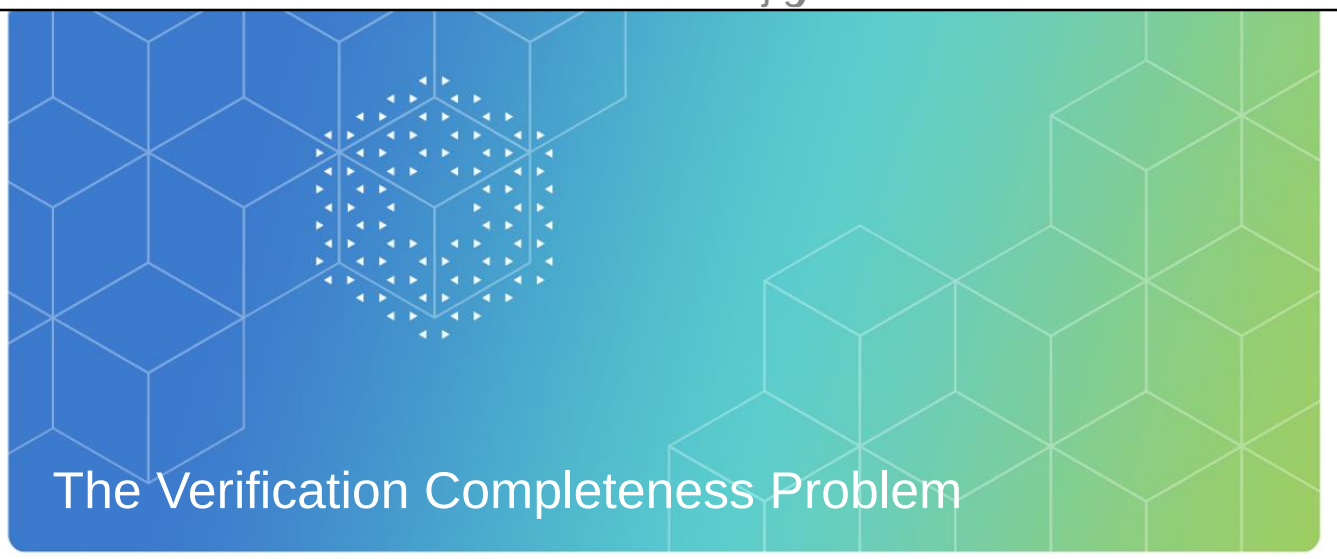
Coverage is a measure of whether we've stimulated a design thoroughly, to trigger all bugs.

Coverage Monitors observe the design during simulation and report what behavior is exercised.

The "cover" directive creates a Coverage Monitor, using the same behavioral description features as used for assertions.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Module Objectives

In this module, you

- Review verification methodologies
- Examine case study highlighting the verification completeness problem
- State techniques for minimising risk of verification holes
- States the merits of each verification technique

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Scale of the Functional Verification Problem

The most complex of processor IC designs may have, for example, 500 thousand flips-flops (FFs)

Those FFs could be in any of 2^{500000} different states at any particular time

The actual number of states will be much less because

- By design not all states will be possible
- There will be many states we don't particularly care about

However, the number of states is still absolutely massive

It is essential that you accept this truth

You cant possibly test everything

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

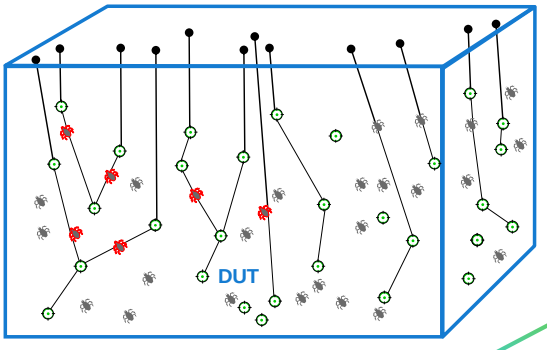
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Most Basic Approach to Verification – Directed Tests

Verification engineer:

- **Defines DUT states to be tested based on specified behavior and corner cases**
- **Writes directed test to visit and verify each item in the test plan**



Redo if design changes

Automation — **Significant manual effort to write all the tests**

Automation — **Work required to verify each goal was reached**

Completeness — **Poor coverage of non-goal scenarios … especially the cases that you didn't "think of"**

 © Cadence Design Systems, Inc. All rights reserved.

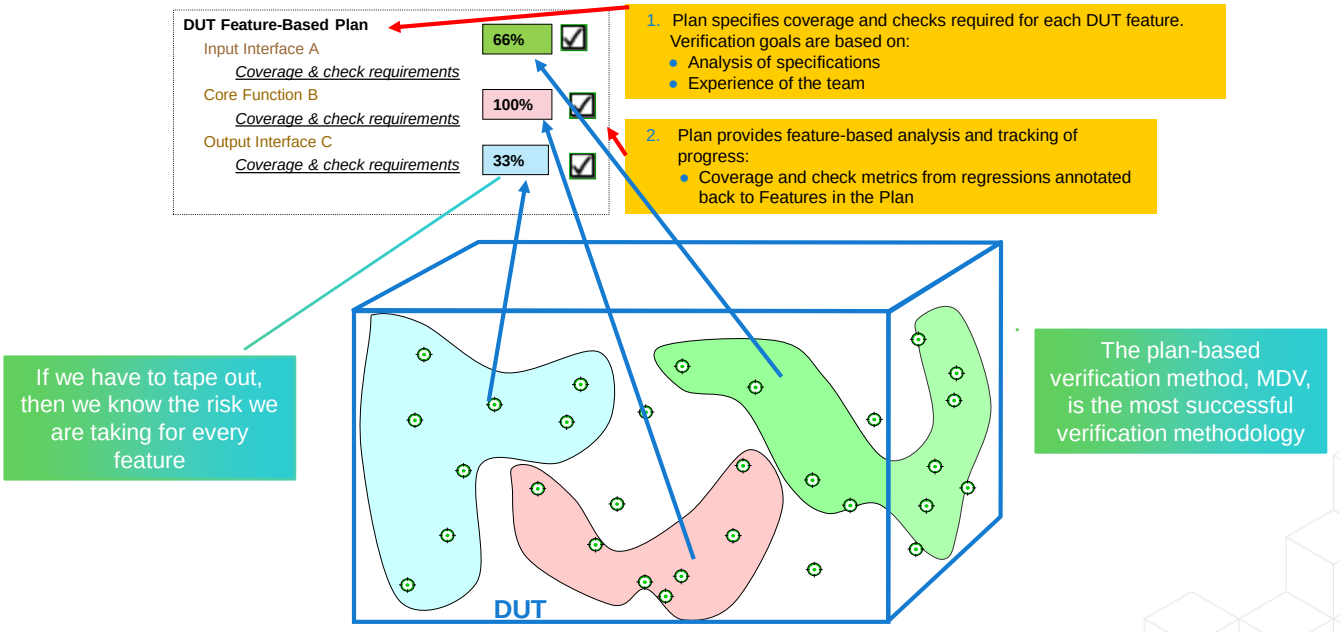
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Metric Driven Verification (MDV)



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Concise and Simplistic Summary of What MDV Is

Define a hierarchical verification plan which

- States the features the design is supposed to have
- Specify the checks to done in order to test those features
- Specify the coverage required in order to be confident that we tested every important scenario the design is expected to work under

We know we have finished verification when we have reached 100% coverage in the context of no checks failing

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

“The Best Laid Plans of Mice and Men”

As the saying goes, “If we fail to plan then we plan to fail”

However, having a plan does not guarantee success

The plan could be wrong

The plan might not be implemented correctly

To show how this can happen for even the simplest of designs we will follow a case study

The schematic we will show defines the functional specification.
The intent is to verify every feature of the design by defining SVA properties.
We will verify these properties using JasperGold® Formal Verification.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Completeness of Property Sets

We obviously define what we think are sufficient properties:

- As identified in the Vplan

This is not just limited to properties or formal verification

- Same argument for a constrained random coverage driven simulation testbench:
  - Did we specify enough coverage?

How can we be certain they cover all required functionality?

A formal tool might tell us that all I/Os and FFs are covered by assertions

This does not mean that we have written properties to cover all functionality

- In the same sense that code coverage does not tell us we have complete functional coverage

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Verification [in]Completeness Example

Shown is a schematic of the functional requirements that are implemented in RTL:

- We write blackbox properties, which we think completely checks all desired functionality
- Human error or oversight might mean that we don't specify all desired behavior

Tool might tell us

- All I/Os and all FFs are included in some assertion:
- There is no deadcode
- All checks pass
- All cover items pass

Should we conclude that verification is complete?

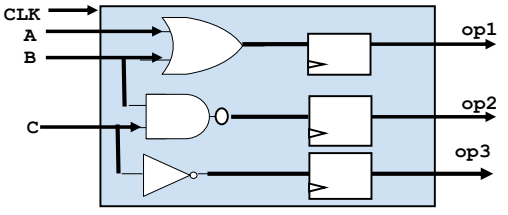
- No way of knowing that we specified enough properties to check all functionality
- Human error or oversight might escape detection in the verification process

At first glance these properties seem sensible, however ….

```
P1: assert property ( (A || B) |=> op1 ); ☑
 P2: assert property( (!A && !B) |=> op2 );
     P3: assert property( !C |=> op3 );
```

Human error, `A` has no effect upon op2 but property still holds

a	b	c	op1`	op2`	op3`
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	1	0	0



 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

Design Mutation

Shown is a schematic whose behavior satisfies our properties

This is a mutant of the desired design

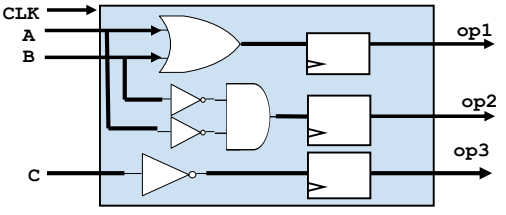
We can see that our verification environment fails to detect this mutation of our desired design

Hence, we did not capture a property set that completely and uniquely defines desired behaviour

Change from desired truth table

a	b	c	op1`	op2`	op3`
0	0	0	0	1	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	0	0
1	1	0	1	0	1
1	1	1	1	0	0

```
P1: assert property ( (A || B) |=> op1 ); ☑
  P2: assert property( (!A && !B) |=> op2 );
      P3: assert property( !C |=> op3 );
```



229 © Cadence Design Systems, Inc. All rights reserved.

In this example, we can (if the properties are sufficient) detect the mutant design.

In practice, the mutation might not propagate an observable difference from the expected behavior to somewhere it could be detected.

This indicates a problem with complete stimulus, that is, we did not provide the correct stimulus to observe the difference in behavior caused by the mutation.

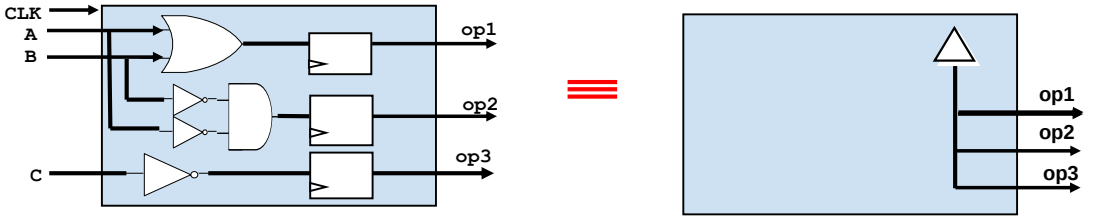
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Prasad of Cadence

We Don’t Even Have the Fundamentals Correct

All these properties pass with the outputs tied to the power rail

```
P1: assert property ( (A || B) |=> op1 );  ☑
    P2: assert property( (!A && !B) |=> op2 ); ☑
    P3: assert property( !C |=> op3 );         ☑
```



The correct assertion to check OP1 is not particularly intuitive

```
P1: assert property( op1 == $past(A || B)); ☑
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

How Can We Catch That Problem?

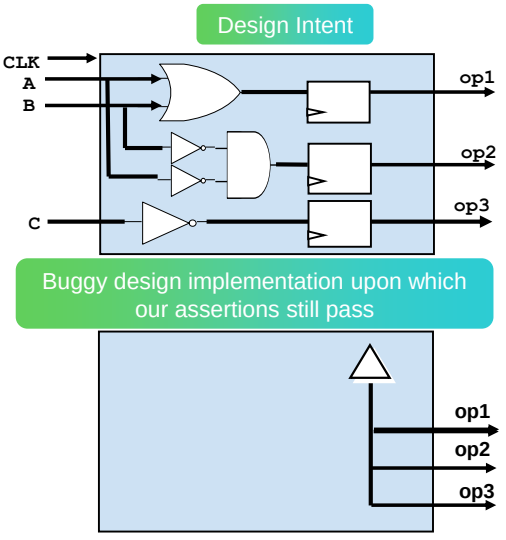
Clearly our assertions didn't check what we intended

Probably, given our design example, we didn't think of any meaningful user defined covers

How can we lower the probability of this bug escaping verification?

We have already seen JasperGold features which would find these problems

- Superlint Auto Formal Toggle check
- JasperGold Coverage App
  - None of the design would be in the proof core




```
P1: assert property ( (A || B) |=> op1 );        ☑
    P2: assert property( (!A && !B) |=> op2 );   ☑
    P3: assert property( !C |=> op3 );           ☑
```

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuku Vijay Prasad of Cadence

Locality and the Dangers of Overoptimism

If one made JG report elements in the COI of the assertion then it would be both the grey and green parts

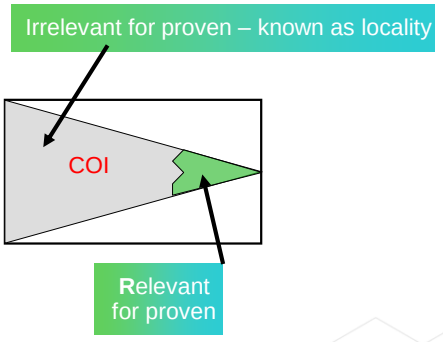
- i.e., the static fanin cone of all signals in the assertion

Question: Does this mean we have covered/verified/exercised all elements in the COI of the property?

There is the danger of overoptimism on coverage

The grey part is irrelevant for the proof

We learned nothing about the grey part



232 © Cadence Design Systems, Inc. All rights reserved.

cadence

Knowing how much of the design was meaningfully exercised by JasperGold® is a very important metric for just about every customer.

The coverage app can collect design metrics on:

- Stimuli coverage
- COI coverage
- Proof coverage
- Bound coverage
- Code coverage
  - Branch
  - Statement
- Functional coverage
  - SVA/PSL covers

Contact your local AE to discuss the JasperGold Coverage App.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Can We Achieve Completeness?

Is it impossible for a tool to read our minds?

Probably, but we can get closer using tool features

No current tools can tell you when you have a complete property set

There is no substitute for:

- Understanding the requirements
- Intelligent planning
- Attention to detail
- Hard work

No EDA tool can tell you that your verification is complete.
All the EDA tools can tell us is if we didn't test all of the things that we said we would, namely if we are incomplete.

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Assessing Completeness

Defining your verification objectives upfront is the best way to assess when you are done

Beyond this, there are other ways to assess incompleteness:

- Do all the outputs of the block have assertions describing their behavior?
- JasperGold Coverage App can help assess what portion of the design your assertions are exercising:
- Cannot tell you when you are done, but when you are not done
- There are no tools that can tell you when you have done enough verification at present

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vijay Praveen of Cadence

Verification Execution Completeness

This is something that we can know

The goal is the same in both dynamic (simulation) and formal

All check items pass/proved in context of coverage items *equals* verification complete

Execution Method	Execution Item	Completeness
Dynamic	Check	All checks are executed with no violation (checks pass)
	Coverage	All coverage items are reached (pass); **driver** + DUV dependent

Execution Method	Execution Item	Completeness
Formal	Check	All checks are proven with no violation (checks proved)
	Coverage	All coverage items are reachable; **constraints** + DUV dependent

 © Cadence Design Systems, Inc. All rights reserved.

cadence

We can know for sure that all checks we have defined have passed and we have reached all cover items.

The holy grail of verification is knowing whether those checks and cover items defined all behaviours which we want to verify.

This might be possible in the future but not at the present time.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

What Can Be Automated?

Its probably more important to ask the question in reverse, what can't be automated:

- Writing the functional specification
- Writing the Verification Plan
- Interpreting the verification plan to implement the checks and coverage
- Interpreting the results from verification runs
- Understanding the limitations of the verification technique we use

EDA is about automating all that can be, for example:

- Managing huge amounts of data
- Compilation/recompilation
- Running and rerunning verification tools after bug fixes
- Analysing which tests are the most valuable in terms of coverage achieved
- Tracking bug rates/coverage accumulation rates

 © Cadence Design Systems, Inc. All rights reserved.

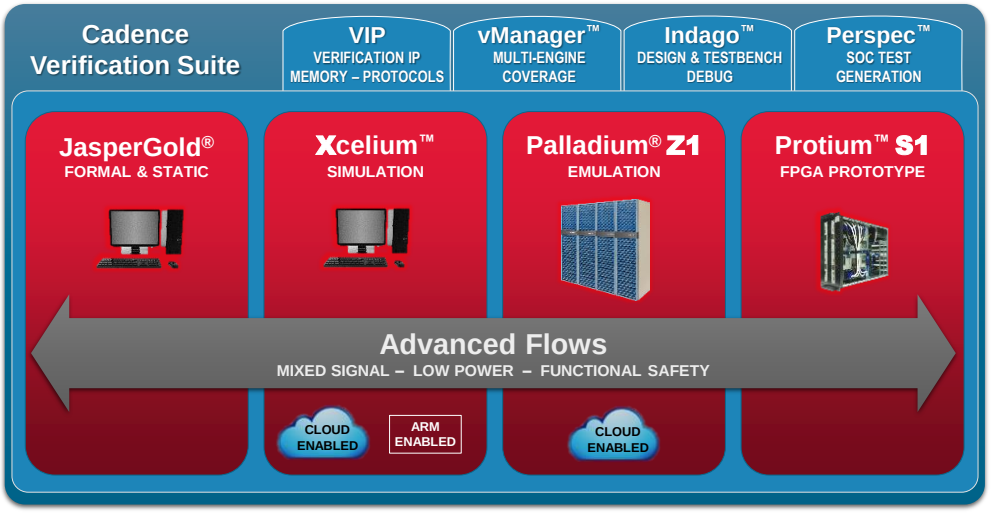
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Cadence Verification Solution



 © Cadence Design Systems, Inc. All rights reserved.

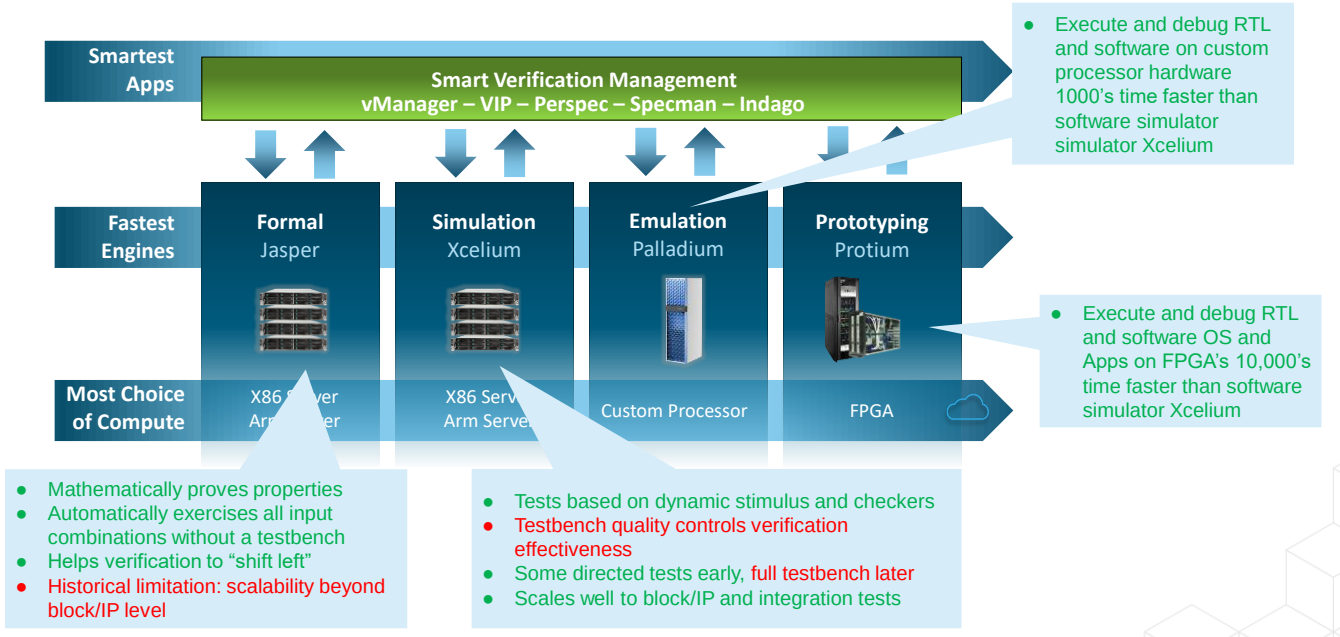
cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Where Each Technique Apply



 © Cadence Design Systems, Inc. All rights reserved.

cadence

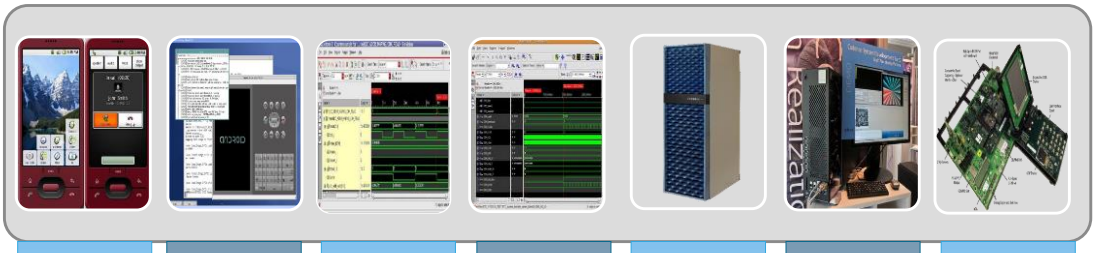
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

There Is No "One Size Fits All"

Verification and software platforms need to interoperate



SDK OS Simulation	Virtual Platform	Formal Analysis	HDL Simulation	Acceleration Emulation	FPGA Prototype	Prototyping Board
• Highest speed • Earliest in the flow • Ignore hardware	• Almost at speed • Less accurate (or slower) • Before RTL • Great to debug (but less detail) • Easy replication	• Non-scalable • Exhaustive • Early RTL • Great for IP • No SW execution	• KHz range • Accurate • Excellent HW debug • Broadly available • Mixed-abstractions • Limited SW execution	• MHz Range • RTL accurate • After RTL is available • Good to debug with full detail • Expensive to replicate	• 10's of MHz • RTL accurate • After stable RTL is available • OK to debug • More expensive than software to replicate	• Real time speed • Fully accurate • Post Silicon • Difficult to debug • Sometimes hard to replicate

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Summary

- Functional Verification is about risk management not verifying everything
- We need to minimize the chances of a critical bug escaping the verification process
- There are many tools and methodologies to help us do this
- We cannot abdicate the responsibility for thorough verification to automation
- Automation only helps with manging data, predictable repetitive task and computing
- Functional Verification is still a human process vulnerable to human error
- There is no single verification methodology which worst best for every circumstance all of the time

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

SVA Course Part 2 (Simulation Users Only) Conclusion

Module	**14**
Revision	**1.0**
Version	**5.1**

Estimated Time:

- **Lecture**
- **Lab**

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

What Have We Learned in Part 1 (Review)?

Motivation and methodology of using ABV

How to define Sequential and Boolean properties

Language features and methodologies for property reuse

- Including from formal to simulation and vice-versa

Motivation and methodology of defining coverage with SVA

How to correctly define Liveness and Safety properties

Motivation and methodology of using Auxiliary (HDL helper) code

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

SVA Issues (Review)

SVA syntax looks easy but the application is not.

- The syntax is “sugared” and meant to look easy
- The concepts are hard

Not all common behaviors can be checked with SVA alone

Aux code often required to:

- Make properties simpler to express
- Do things which assertion languages cannot express
- Reduce “complexity” for Formal

Special Techniques can be applied when using Formal

- These are not compatible with Simulation

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vijay Praveen of Cadence

SVA Part 2 (Simulation Users Only) Summary

We showed how we can describe complex sequence relationships with the sequence operators

However, being able to describe complex sequences is a double edged sword

- There are hard to define correctly and debug
- For formal they create proof complexity issues

We saw some advanced SVA features to describe complex behaviour, e.g., local variables and sequence endpoints

SVA has many operators and constructs which return a property expression

Not all of them are of pragmatic use

- They often overcomplicate things which can already be achieved using sequences in a simpler way

We saw how easy it is to achieve, and know you have achieved, a complete property set for verification

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Where Next?

SystemVerilog Language Reference Manual

- `www.ieee.org`
- `IEEE 1800-2012`

Textbooks

- Formal Verification: An Essential Toolkit for Modern VLSI Design
  - Erik Seligman
- SystemVerilog Assertions Handbook
  - Cohen *et al*

 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

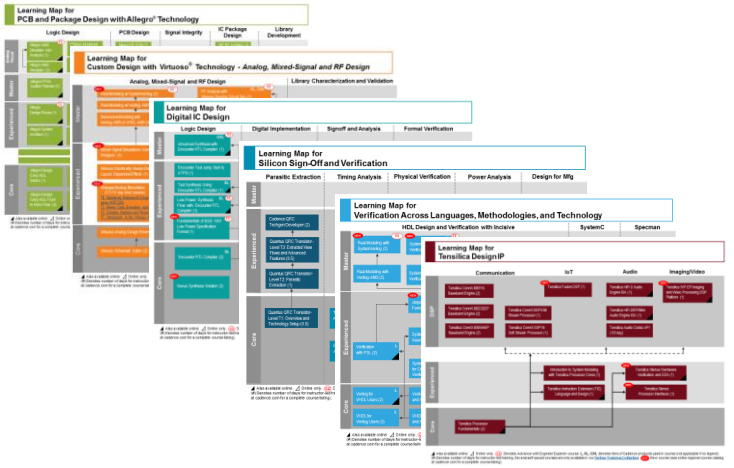
© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

Click here to see all our courses in each technology area and the recommended order in which to take them.



 © Cadence Design Systems, Inc. All rights reserved.

cadence

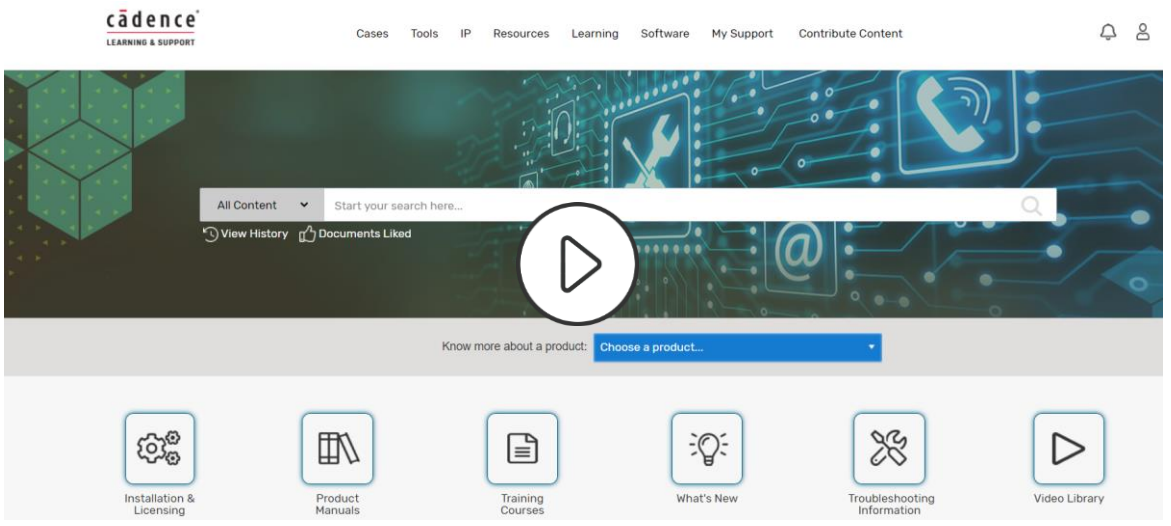
Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawashe of Cadence

Cadence Learning and Support



Customer Support now includes over 2000 product/language/methodology videos ("Training Bytes")!

247 © Cadence Design Systems, Inc. All rights reserved.

Click the play button in the figure on this slide to view the demo of Cadence Learning and Support.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion and a Digital Badge

Thank you!

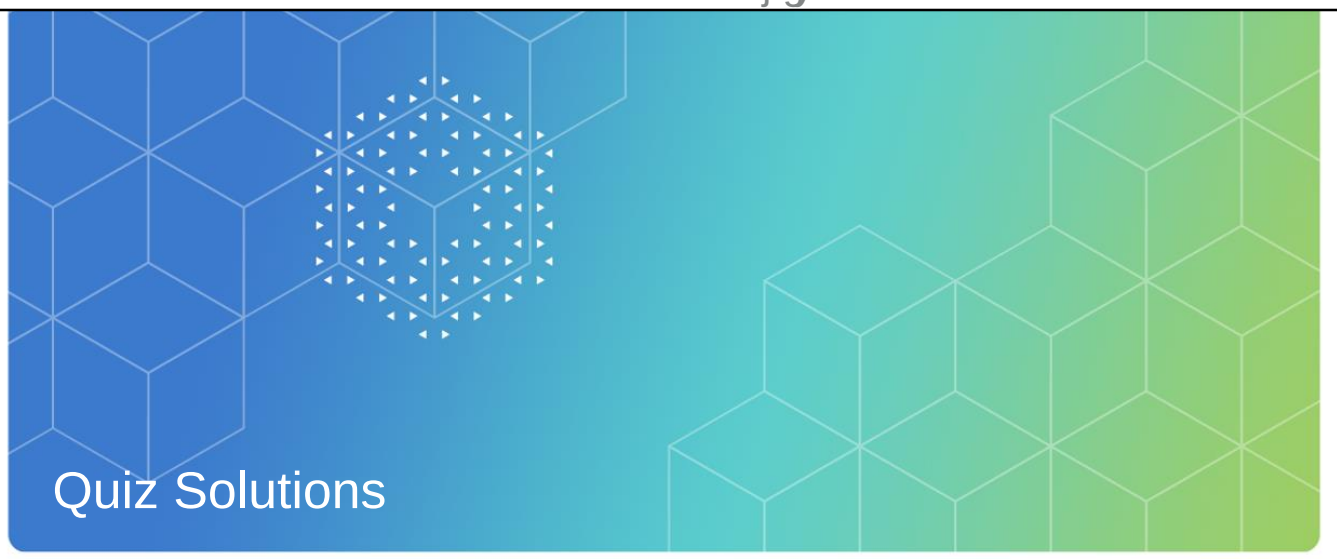
 © Cadence Design Systems, Inc. All rights reserved.

cadence

This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



Appendix **A**
Revision **1.0**
Version **5.1**

Estimated Time:
- **Lecture**
- **Lab**

cadence

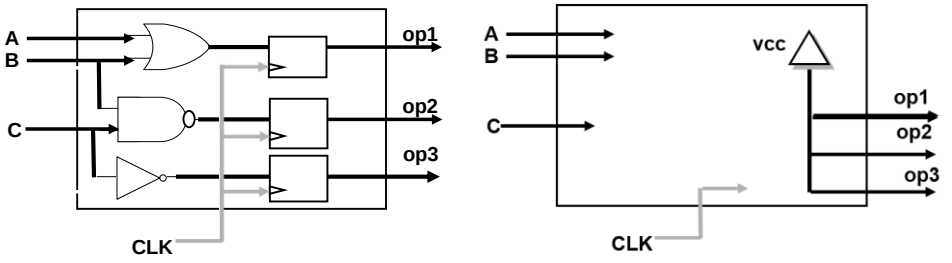
This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Boolean Assertions: Quiz

Write assertion(s) to check the behavior of the circuit:




```
P1 : assert property ( @(posedge CLK) A || B |=> op1 );
P2 : assert property ( @(posedge CLK) !(B && C) |=> op2 );
P3 : assert property ( @(posedge CLK) !C |=> op3 );
```

These assertions check some of the desired behavior

However, op1, op2 and op3 might be stuck at logic 1 every cycle!

You would need coverage to check that op1 can be zero

- Assuming that you care about the value of op1 if A || B was false on the previous cycle

```
P1_B : assert property ( @(posedge CLK) op1 |-> $past( A || B) );
```

```
P1_B : assert property ( @(posedge CLK) op1 == $past( A || B) );
```

250 © Cadence Design Systems, Inc. All rights reserved.

Hopefully this quiz gets you in the mindset of writing assertions. Don't worry if it looks harder than you thought, this is the feeling most people have initially.

It's recommended to try and forget about procedural code like Verilog, C/C++, VHDL.

Knowing when you have written enough assertions to cover all desired behavior is a very difficult task. In fact, it is so hard that tools cannot do this for you automatically. This is called the *completeness* problem and is applicable to verification in general, not just using properties or formal verification. This is covered in more detail later.

Often with assertions to specify completely the behavior which you want to be checked, you may need to check the output value in context of previous inputs and the next output value in context of the current inputs.

Can you think of a circuit failure scenario where the properties below cannot detect all possible bad behaviors?

P1_B : assert property (@(posedge CLK) op1 |-> $past(A || B));

P2 _B: assert property (@(posedge CLK) op2 |-> $past(!(B && C)));

P3 _B: assert property (@(posedge CLK) op3 |-> $past(!C));

What about if op1, op2 and op3 were all stuck at 0?

You need coverage to check that op1, op2 and op3 did occur during verification.

Just as with Coverage Driven Verification, OVM, Specman, Constrained random simulations:

"Checking (assertions) without coverage is of no use"

Having said all of that, let's revisit the properties in the slides.

op3 could be stuck at logic 1 and the assertions still all pass.

You must also consider that, for example, perhaps you don't care what value op1 is if A || B was not true on the previous cycle.

When writing assertions, don't copy the HDL implementation, you define the desired rules of behavior.

If the desired behavior was that op1 must be true in the cycle after A || B, then maybe you don't care what op1 is doing at any other time.

If you do care about the value of op1 at other times, then you need to explicitly state what behavior you want to see.

© Cadence Design Systems, Inc. All rights reserved.

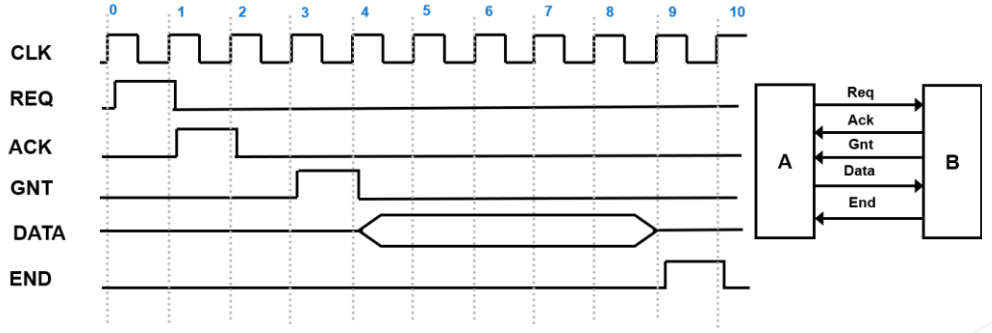
This document is for the sole use of Kuhn Vijay Praveen of Cadence

Sequences: Quiz

After REQ in one cycle followed by ACK in the next:

- GNT must be low in the same cycle as ACK, and goes high between 1 and 2 cycles *after* ACK goes high (cycles 3 or 4)
- END must remain low for 5 CLK cycles after GNT as DATA clocked in
- END then goes high

```
@(posedge CLK) (req ##1 ack) |-> !gnt[*1:2] ##1 gnt ##1 !end[*5] ##1 end;
```



251 © Cadence Design Systems, Inc. All rights reserved.

Enabling sequence is (REQ ##1 ACK). You use the same cycle implication for the fulfilling sequence as GNT must be low in the same cycle as ACK. GNT must be low for 1 to 2 cycles before going high, (!GNT[*1:2] ##1 GNT). After GNT, look for 5 cycles of END low followed by END high (!END[*5] ##1 END). So the full property is:

```
property P1;
  @(posedge CLK)
  (REQ ##1 ACK) |-> !GNT[*1:2] ##1 GNT ##1 !END[*5] ##1 END;
endproperty
A1 : assert property (P1);
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuhn Vajay Praveen of Cadence

Sequence Operators: Quiz

1. Write sequences to describe the following properties:
   A. 4 nonconsecutive occurrences of `PULSE` in 10 cycles

```
(PULSE[=4]) intersect (1'b1[*10])
```

   B. `GO` is followed by either `ERR` or 3 to 7 consecutive cycles of `ENB`

```
GO ##1 (ERR) or (ENB[*3:7])
```

2. Write an assertion for the following specification:

A sequence beginning with STRT, followed by 6 consecutive occurrences of PUSH, during which KILL does not occur, must be followed by 6 nonconsecutive occurrences of POP before STOP can occur.

```
property FRAME; @(posedge CLK)
  STRT ##1 (PUSH && !KILL)[*6] |=>
                      !STOP throughout POP[->6];
endproperty
A1 : assert property (FRAME);
```

252 © Cadence Design Systems, Inc. All rights reserved.

cadence

Question 2: There are many possible ways of describing this property! The enabling sequence can be defined as:

```
STRT ##1 (PUSH && !KILL)[*6]
```

i.e. `STRT` followed by 6 consecutive occurrences of `PUSH` and not `KILL`.

The fulfilling sequence could be defined as:

```
!STOP throughout POP[->6]
```

i.e., 6 nonconsecutive occurrences of `POP`, terminating on the 6th occurrence, during which `STOP` is low. Note the use of go-to repetition for `POP`, so that after the 6th occurrence of `POP`, the sequence immediately terminates. There is no check to see if STOP does occur.

```
property FRAME; @(posedge CLK)
  STRT ##1 (PUSH && !KILL)[*6] |=>
  !STOP throughout POP[->6];
endproperty
A1 : assert property (FRAME);
```

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence

Advanced Features: Quiz

1. Expand `WHATSEQ`:

```
sequence ONE;   (A ##1 B[*3] ##1 C);   endsequence
sequence TWO;   (K ##1 L ##1 M ##1 N); endsequence
sequence THREE; (X ##2 Y[*2]);         endsequence
sequence WHATSEQ;
  (ONE.triggered ##1 TWO.triggered ##1 THREE.triggered);
endsequence
```

```
sequence WHATSEQ_EXPANDED;
  A ##1 B ##1 (B && K) ##1 (B && L && X) ##1
 (C && M) ##1 (N && Y) ##1 Y;
endsequence;
```

2. In the property below, `end` must go high within 5 and 15 cycles of `start`. For a response of 10 cycles or less, `short` is asserted with `end`, For greater than 10 cycles, `long` is asserted with `end`. Modify the property to check right number of cycles for each alternate response:

```
property count_cycles;
  @(posedge clk)
  $rose(start) |=> !end[*4:14] ##1 ((end && short) or (end && long));
endproperty
```

```
property count_cycles;
int cyc;
  @(posedge clk)
  ($rose(start), cyc=0) |=> (!end, cyc=cyc+1)[*4:14] ##1
  ((end && short && cyc<= 10) or (end && long && cyc > 10));
endproperty
```

253 © Cadence Design Systems, Inc. All rights reserved.

cadence

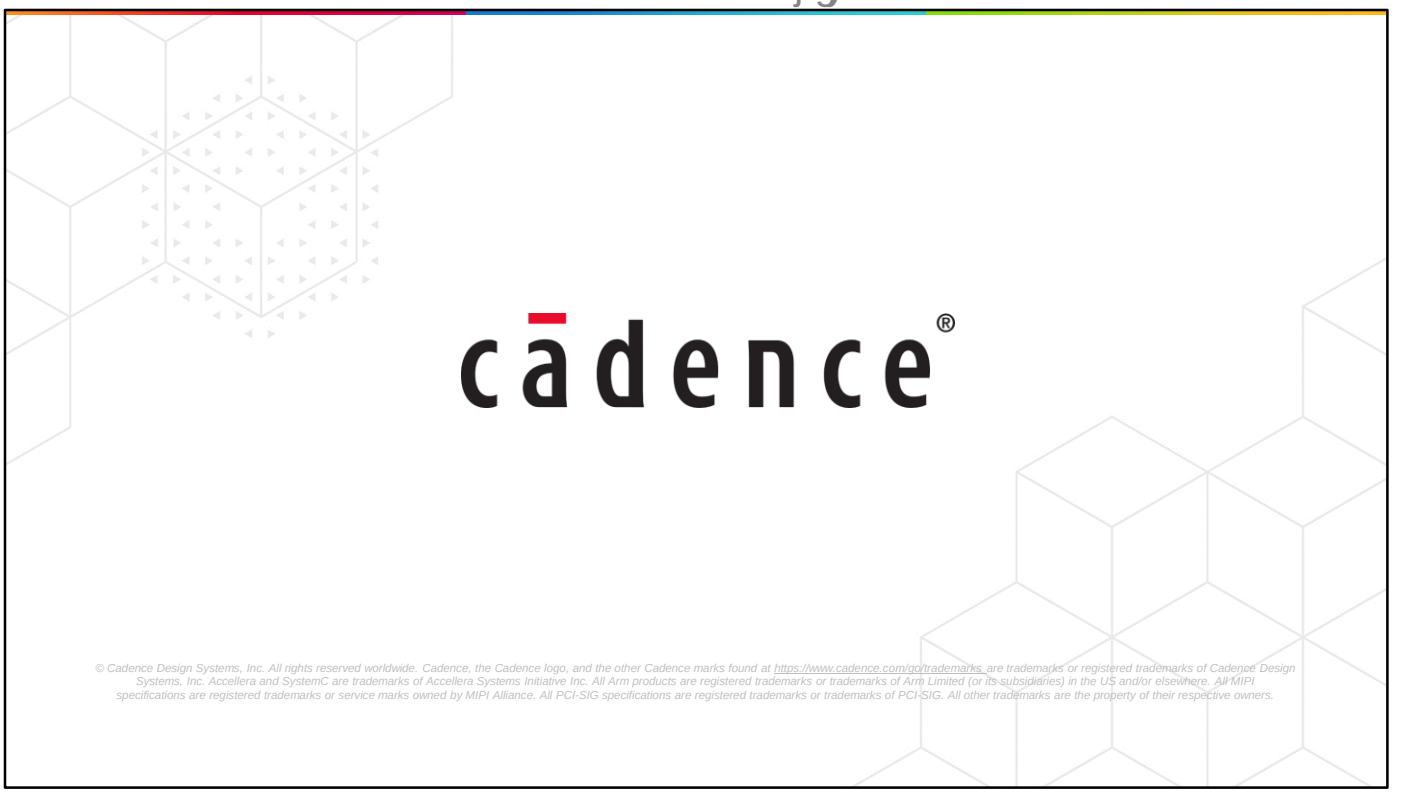
```
sequence WHATSEQ_EXPANDED;
  A ##1 B ##1 (B && K) ##1 (B && L && X) ##1
 (C && M) ##1 (N && Y) ##1 Y;
endsequence;
```

```
property count_cycles;
int cyc;
  @(posedge clk)
  ($rose(start), cyc=0) |=> (!end, cyc=cyc+1)[*4:14] ##1
  ((end && short && cyc<= 10) or (end && long && cyc > 10));
endproperty
```

Initialize a local variable `cyc` on the enabling condition. Increment `cyc` on every occurrence of `end` low. Use the value of `cyc` as an additional boolean check in the two alternatives.

© Cadence Design Systems, Inc. All rights reserved.

This document is for the sole use of Kuldeep Vijay Pawaskar of Cadence



This page does not contain notes.

© Cadence Design Systems, Inc. All rights reserved.